CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

Chandubhai S. Patel Institute of Technology (CSPIT)

K.D. Patel Department of Information Technology • Academic Year 2026–2027

Name : Anshul Bhalala	**ID :** 25IT007	**Branch :** IT
DEPT : CSPIT	**SUBJECT :** OOP (CEUE203)	**Semester :** 3rd Semester

INDEX OF PRACTICAL IMPLEMENTATIONS

Prac	Title / Practical Problem Statement	Page No.	Date	Sign
01	Vending Machine: Coin enums & switch expressions Toll Booth: Vehicle records & frequency counter RPSLS: Five-move game (Advanced Learners)	**2 – 7**	07/07/2026	
02	Smart Thermostat: Encapsulation & constructor chaining Cinema Show: Transactional entity & state tracking Parking Lot: Capacity and occupancy (Advanced Learners)	**8 – 13**	13/07/2026	
03	Distinct Points: equals() & hashCode() Duplicate Card Check: Composite equality & search Fractions: Fraction reduction & comparison (Advanced Learners)	**14 – 19**	20/07/2026	
04	Password Strength: Regex-based security validation Chat-Log Filter: Case-insensitive filtering & StringBuilder Template Filler: Regex placeholders (Advanced Learners)	**20 – 25**	29/07/2026	
05	Shape Areas: Abstract classes & dynamic method dispatch Payroll System: Superclass delegation & subclass polymorphism Media Late Fees: Abstract media hierarchy (Advanced Learners)	**26 – 31**	03/08/2026	
06	Remote Control: Interfaces, default methods & lambda predicates Notification Senders: Functional interfaces & urgent markers Discount Engine: Lambda-based discount rules (Advanced Learners)	**32 – 37**	17/08/2026	
07	Form Validator: Declarative annotations & reflection Mini Test Runner: Dynamic method discovery & @Run execution Column Mapper: Runtime annotation field mapping (Advanced Learners)	**38 – 43**	27/10/2026	
08	Guarded Calculator: DivideByZeroException & multi-catch recovery Stock Issue: Contextual OutOfStockException & inventory handling AutoCloseable Resource: Resource lifecycle (Advanced Learners)	**44 – 49**	16/09/2026	
09	Counter Race: Thread synchronization & race-condition mitigation Seat Booking Race: Monitor locking against ticket overselling Parallel Array Sum: Shared-total race and fixes (Advanced Learners)	**50 – 55**	21/09/2026	
10	Thread Pool Runner: ExecutorService worker reuse & lifecycle Producer–Consumer: Guarded wait()/notify() coordination Deadlock Demonstration: Opposite locks & consistent lock order (Advanced Learners)	**56 – 61**	05/10/2026	
11	Save/Load Serialization: Binary persistence & transient fields Log Analyzer: NIO.2 stream pipelines & file attributes NIO Directory Tree Report: File metadata report (Advanced Learners)	**62 – 67**	12/10/2026	
12	Word Frequency: HashMap counting & Comparator sorting Unique Visitors: Data deduplication via HashSet & LinkedHashSet Leaderboard: Map-based top-N ranking (Advanced Learners)	**68 – 73**	19/10/2026	
13	Generic Containers: Box<T>, Pair<A,B> & generic methods Bounded Stats: Upper-bounded type parameter <N extends Number> Generic Stack: Array-backed generic container (Advanced Learners)	**74 – 79**	26/10/2026	
14	Assemble and Test: Layered application & unit tests Modular Packaging: Packages, runnable JAR & persistence test (Advanced Learners)	**80 – 83**	27/10/2026	

Total Practicals Completed: 14 / 14 **Faculty In-Charge Signature:** _____ **Date:** _____

Part-A

1 to 14 Practical

Start from here

PRACTICAL 1-A

Object Oriented Programming

AIM:

To implement and understand the concepts specified in Practical 1 of the CEUE203 Object Oriented Programming practical list, using Java. The work covers Part A: Vending Machine, Toll Booth, RPSLS and Part B: MiniBank menu shell.

CODE SCREENSHOT:

RPSLS.java 1

Part A > Practical 1-A > RPSLS.java > ...

```java
import java.util.*;

public class RPSLS {
    enum Move {
        ROCK, PAPER, SCISSORS, LIZARD, SPOCK
    }

    static int winner(Move a, Move b) {
        if (a == b)
            return 0;
        boolean beats = switch (a) {
            case ROCK -> b == Move.SCISSORS || b == Move.LIZARD;
            case PAPER -> b == Move.ROCK || b == Move.SPOCK;
            case SCISSORS -> b == Move.PAPER || b == Move.LIZARD;
            case LIZARD -> b == Move.SPOCK || b == Move.PAPER;
            case SPOCK -> b == Move.SCISSORS || b == Move.ROCK;
        };
        return beats ? 1 : -1;
    }

    Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Random random = new Random();
        int player = 0, computer = 0;
        for (int i = 1; i <= 5; i++) {
            System.out.print("Round " + i + " (ROCK/PAPER/SCISSORS/LIZARD/SPOCK): ");
            Move p = Move.valueOf(sc.next().toUpperCase());
            Move c = Move.values()[random.nextInt(Move.values().length)];
            int result = winner(p, c);
            if (result > 0)
                player++;
            else if (result < 0)
                computer++;
            System.out.println("You: " + p + ", Computer: " + c + " -> "
                    + (result == 0 ? "Tie" : result > 0 ? "You win" : "Computer wins"));
        }
        System.out.println(player > computer ? "You win " + player + "" + computer
                : player < computer ? "Computer wins " + computer + "" + player
                        : "Overall tie " + player + "" + computer);
    }
}

```

Code Screenshot 1

TollBooth.java 1

Part A > Practical 1-A > TollBooth.java > ...

```java
import java.util.*;

public class TollBooth {
    record Vehicle(String number, String type) {
    }

    Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int total = 0, bike = 0, car = 0, truck = 0;
        while (true) {
            System.out.print("Vehicle number (done to finish): ");
            String number = sc.next().trim();
            if (number.equalsIgnoreCase("done"))
                break;
            System.out.print("Type (bike/car/truck): ");
            String type = sc.next().trim().toLowerCase();
            Vehicle v = new Vehicle(number, type);
            int toll = switch (v.type()) {
                case "bike" -> 20;
                case "car" -> 50;
                case "truck" -> 150;
                default -> 0;
            };
            if (toll == 0) {
                System.out.println("Unknown vehicle type");
                continue;
            }
            total += toll;
            switch (type) {
                case "bike" -> bike++;
                case "car" -> car++;
                case "truck" -> truck++;
            }
        }
        String frequent = bike >= car && bike >= truck ? "bike" : car >= truck ? "car" : "truck";
        System.out.println("Total toll: " + total);
        System.out.println("Most frequent: " + frequent);
    }
}

```

Code Screenshot 2

VendingMachine.java 1

Part A > Practical 1-A > VendingMachine.java > VendingMachine > main(String[])

```java
import java.util.Scanner;

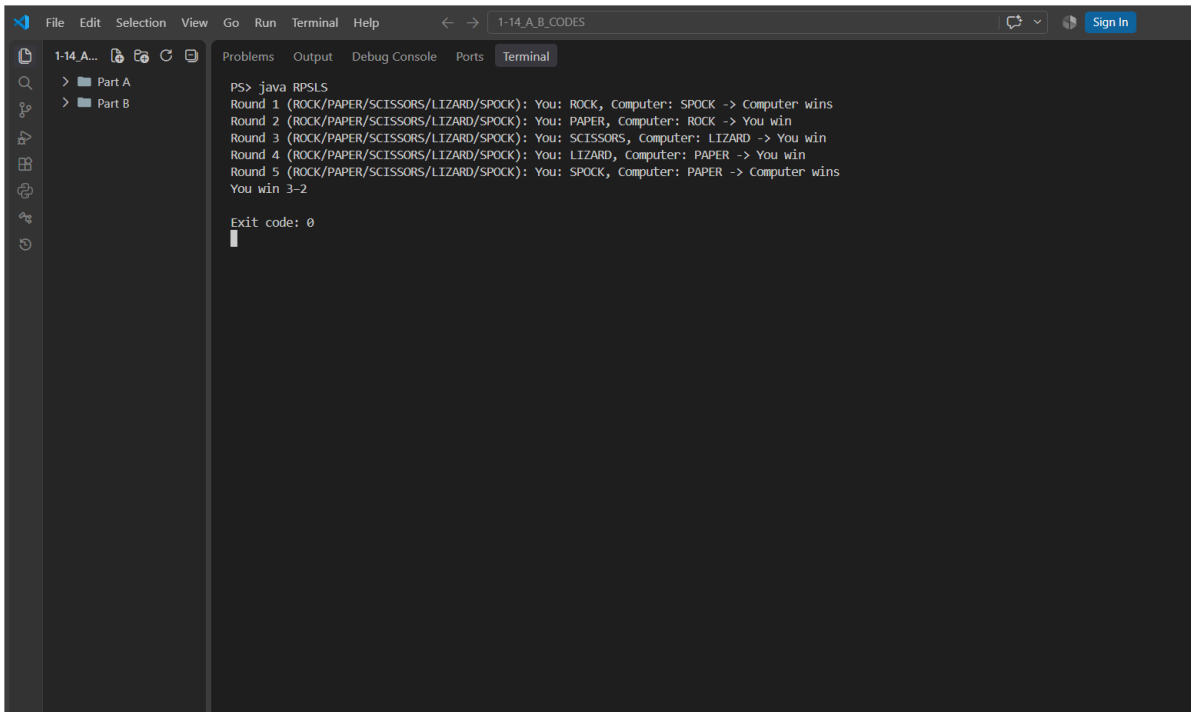
public class VendingMachine {
    enum Coin {
        ONE, TWO, FIVE, TEN
    }

    Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int total = 0, price = 15;
        while (total < price) {
            System.out.print("Enter coin (ONE, TWO, FIVE, TEN): ");
            Coin coin = Coin.valueOf(sc.next().trim().toUpperCase());
            total += switch (coin) {
                case ONE -> 1;
                case TWO -> 2;
                case FIVE -> 5;
                case TEN -> 10;
            };
            System.out.println("Total so far: " + total);
        }
        System.out.println("Paid. Change: " + (total - price));
    }
}

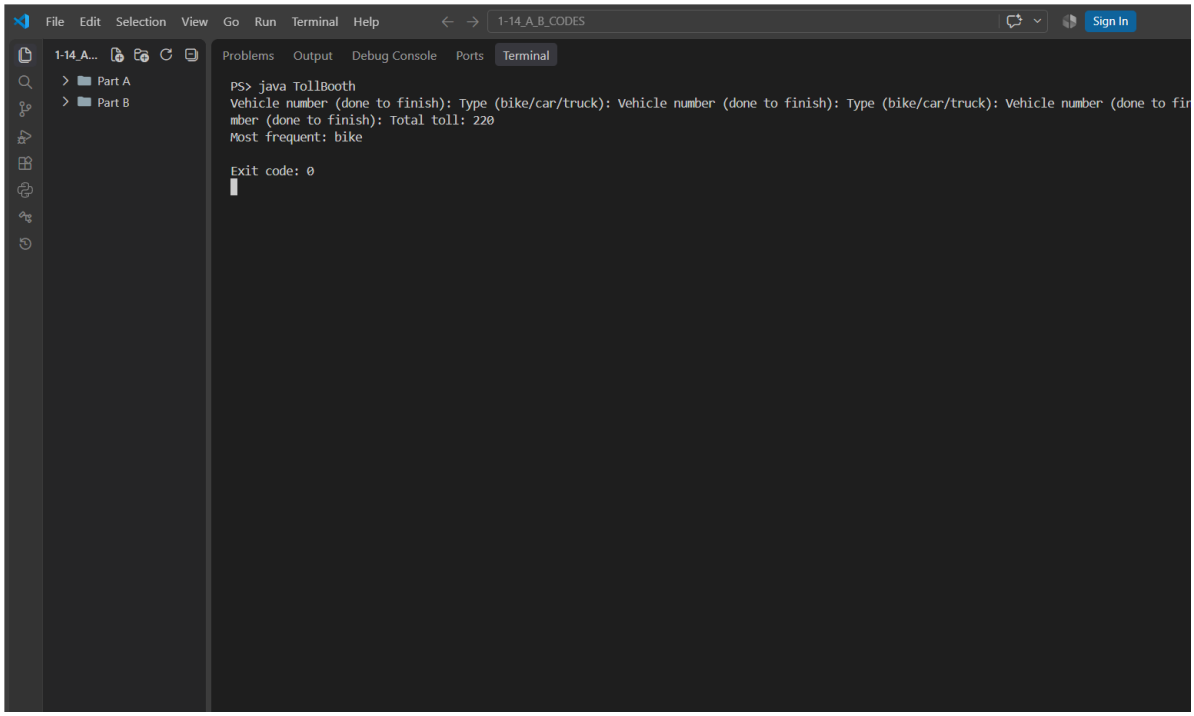
```

Code Screenshot 3

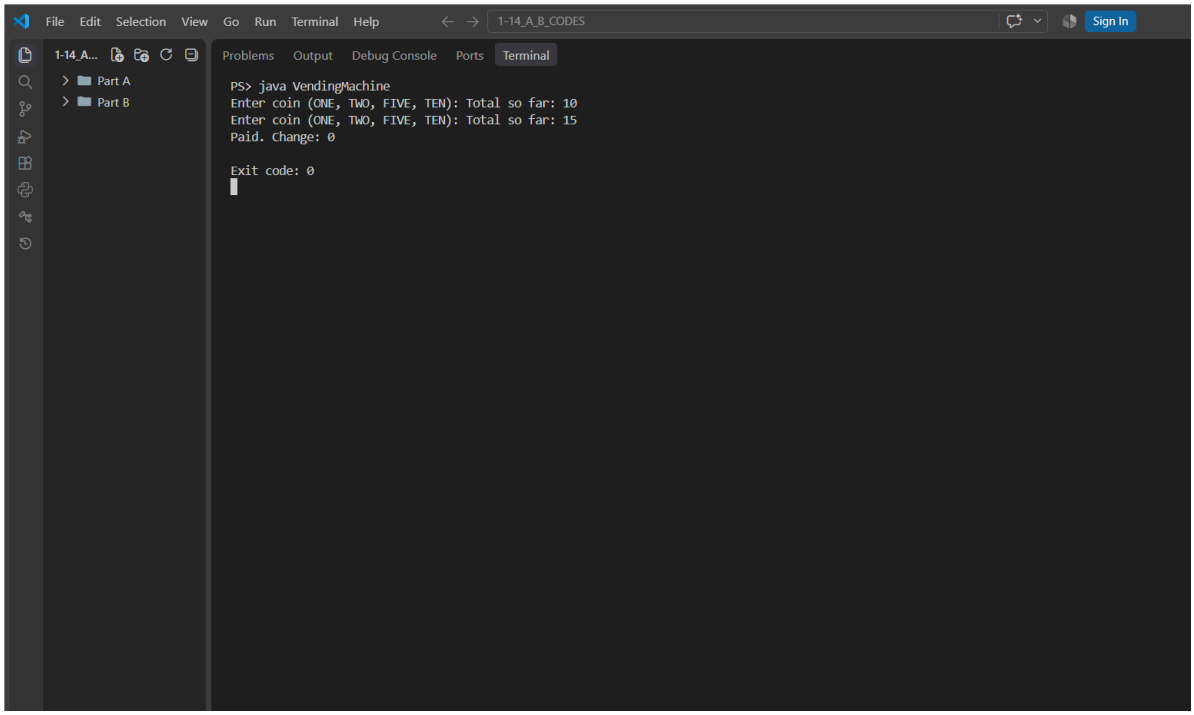
OUTPUT SCREENSHOT:



Output Screenshot 1



Output Screenshot 2



Output Screenshot 3

LEARNING OUTCOMES:

Develop a basic interactive Java console application and use enums, records, switch expressions, and input handling.

PRACTICAL 2-A

Object Oriented Programming

AIM:

To implement and understand the concepts specified in Practical 2 of the CEUE203 Object Oriented Programming practical list, using Java. The work covers Part A: Thermostat, Cinema Show, Parking Lot and Part B: Customer and Account encapsulation.

CODE SCREENSHOT:

CinemaShow.java

Part A > Practical 2-A > CinemaShow.java > ...

```java
public class CinemaShow {
    private String title;
    private int seatsAvailable;
    private final int capacity;
    private static int totalBooked = 0;

    public CinemaShow(String title, int capacity) {
        this.title = title;
        this.capacity = capacity;
        seatsAvailable = capacity;
    }

    public CinemaShow(String title) {
        this(title, capacity: 100);
    }

    public boolean book(int n) {
        if (n > 0 && n <= seatsAvailable) {
            seatsAvailable -= n;
            totalBooked += n;
            return true;
        }
        return false;
    }

    public void cancel(int n) {
        if (n > 0)
            seatsAvailable = Math.min(capacity, seatsAvailable + n);
    }

    public int getSeatsAvailable() {
        return seatsAvailable;
    }

    public static int getTotalBooked() {
        return totalBooked;
    }

    Run | Debug
    public static void main(String[] args) {
        CinemaShow show = new CinemaShow(title: "Java Premiere", capacity: 50);
        System.out.println("Book 20: " + show.book(n: 20) + ", seats: " + show.getSeatsAvailable());
        System.out.println("Book 40: " + show.book(n: 40) + ", seats: " + show.getSeatsAvailable());
        show.cancel(n: 5);
        System.out.println("After cancellation, seats: " + show.getSeatsAvailable());
        System.out.println("Total booked: " + getTotalBooked());
    }
}
```

Code Screenshot 1

ParkingLot.java

Part A > Practical 2-A > ParkingLot.java > ...

```java
public class ParkingLot {
    private int twoWheelers = 0, fourWheelers = 0;
    private final int twoCap, fourCap;
    private static long revenue = 0;

    public ParkingLot(int twoCap, int fourCap) {
        this.twoCap = twoCap;
        this.fourCap = fourCap;
    }

    public boolean park(String type) {
        if (type.equalsIgnoreCase("two") && twoWheelers < twoCap) {
            twoWheelers++;
            revenue += 20;
            return true;
        }
        if (type.equalsIgnoreCase("four") && fourWheelers < fourCap) {
            fourWheelers++;
            revenue += 40;
            return true;
        }
        System.out.println("Full");
        return false;
    }

    public void leave(String type) {
        if (type.equalsIgnoreCase("two") && twoWheelers > 0)
            twoWheelers--;
        else if (type.equalsIgnoreCase("four") && fourWheelers > 0)
            fourWheelers--;
    }

    Run | Debug
    public static void main(String[] args) {
        ParkingLot p = new ParkingLot(twoCap: 2, fourCap: 1);
        p.park(type: "two");
        p.park(type: "four");
        p.park(type: "two");
        p.park(type: "two");
        p.leave(type: "two");
        System.out.println(
                "Two-wheelers: " + p.twoWheelers + ", Four-wheelers: " + p.fourWheelers + ", Revenue: " + revenue);
    }
}

```

Code Screenshot 2

Thermostat.java 1

Part A > Practical 2-A > Thermostat.java > ...

```java
public class Thermostat {
    private String location;
    private int temperature;
    private static final int MIN = 16, MAX = 30;
    private static int activeCount = 0;

    public Thermostat(String location, int startTemp) {
        this.location = location;
        temperature = (startTemp >= MIN && startTemp <= MAX) ? startTemp : 22;
        activeCount++;
    }

    public Thermostat(String location) {
        this(location, startTemp: 22);
    }

    public void raise() {
        if (temperature < MAX)
            temperature++;
        else
            System.out.println("Already at maximum (30)");
    }

    public void lower() {
        if (temperature > MIN)
            temperature--;
        else
            System.out.println("Already at minimum (16)");
    }

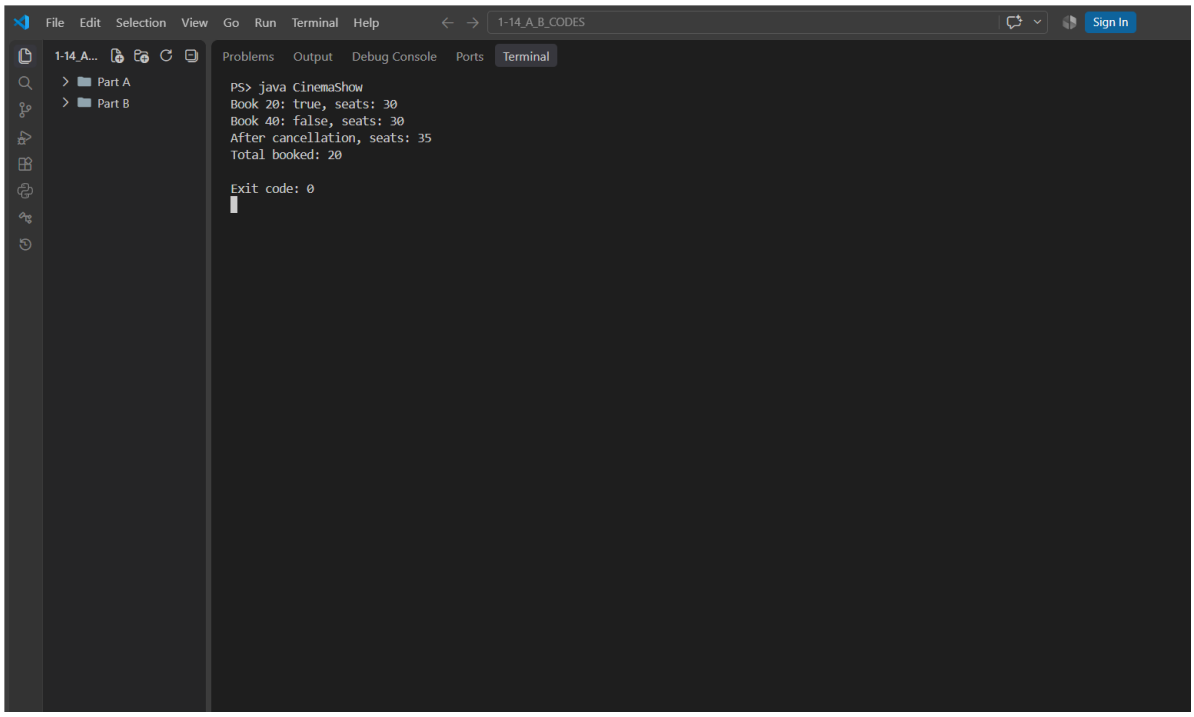
    public int getTemperature() {
        return temperature;
    }

    public static int getActiveCount() {
        return activeCount;
    }

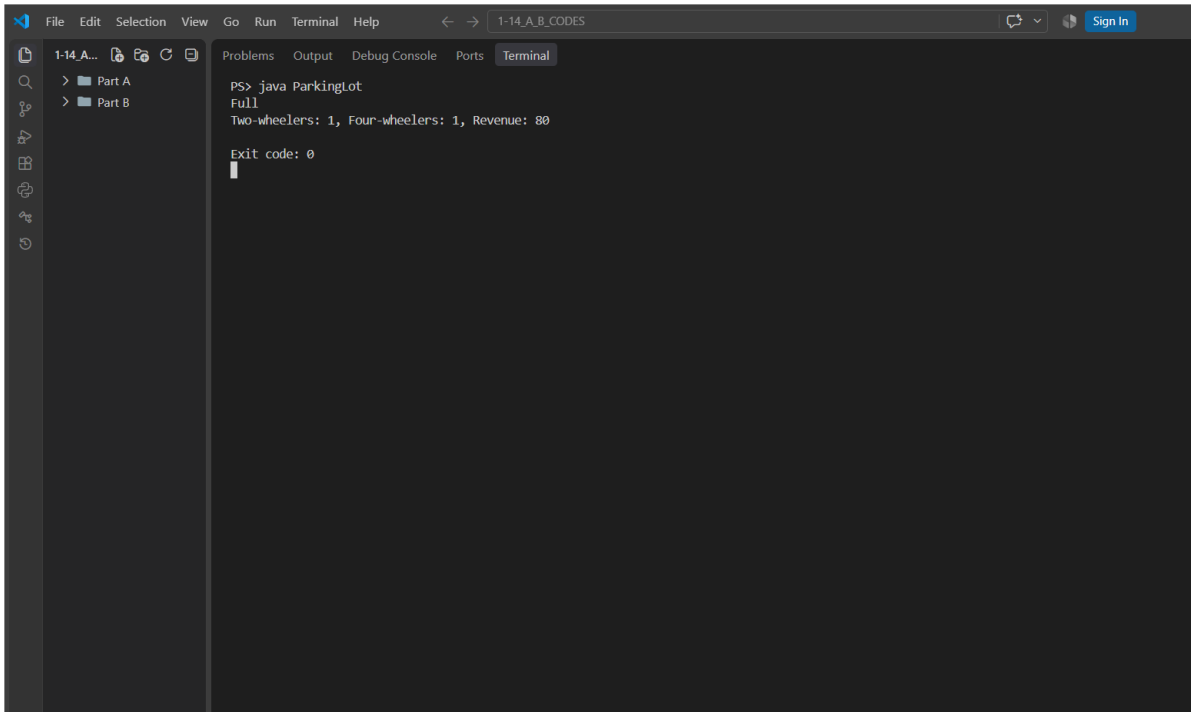
    Run | Debug
    public static void main(String[] args) {
        Thermostat a = new Thermostat(location: "Lab", startTemp: 20), b = new Thermostat(location: "Office");
        for (int i = 0; i < 10; i++) {
            a.raise();
            System.out.println(a.location + ": " + a.getTemperature());
        }
        for (int i = 0; i < 20; i++) {
            a.lower();
            System.out.println(a.location + ": " + a.getTemperature());
```

Code Screenshot 3

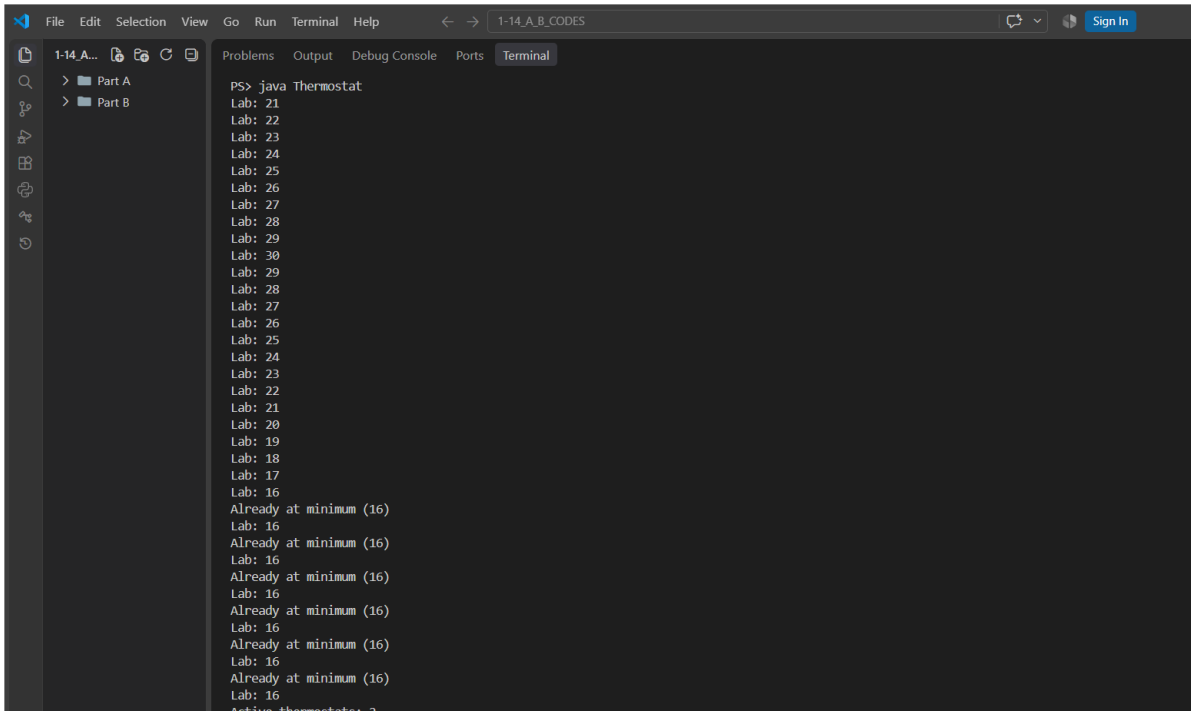
OUTPUT SCREENSHOT:



Output Screenshot 1



Output Screenshot 2



Output Screenshot 3

LEARNING OUTCOMES:

Apply classes, constructors, encapsulation, static members, and object arrays to model real-world entities.

PRACTICAL 3-A

Object Oriented Programming

AIM:

To implement and understand the concepts specified in Practical 3 of the CEUE203 Object Oriented Programming practical list, using Java. The work covers Part A: Distinct Points, Duplicate Cards, Fractions and Part B: Object methods and nested Address.

CODE SCREENSHOT:

Card.java

Part A > Practical 3-A > Card.java > ...

```java
import java.util.Objects;

public class Card {
    private final String rank, suit;
nshul\Downloads\OOP_Practicals_1-13_A_B\Part A\Practical 3-A\Card.java
    t) {
        this.rank = rank;
        this.suit = suit;
    }

    @Override
    public String toString() {
        return rank + " of " + suit;
    }

    @Override
    public boolean equals(Object o) {
        return o instanceof Card c && rank.equals(c.rank) && suit.equals(c.suit);
    }

    @Override
    public int hashCode() {
        return Objects.hash(rank, suit);
    }
}
```

Code Screenshot 1

CardDriver.java

Part A > Practical 3-A > CardDriver.java > ...

```java
import java.util.*;

public class CardDriver {
    Run | Debug
    public static void main(String[] args) {
        Card[] cards = { new Card(rank: "Ace", suit: "Spades"), new Card(rank: "King", suit: "Hearts"), new Card(rank: "Ace", suit: "Spades") };
        List<Card> seen = new ArrayList<>();
        for (Card c : cards) {
            if (seen.contains(c)) {
                System.out.println("Duplicate found: " + c);
                break;
            }
            seen.add(c);
        }
    }
}
```

Code Screenshot 2

Driver.java

Part A > Practical 3-A > Driver.java > ...

```java
import java.util.*;

public class Driver {
    Run | Debug
    public static void main(String[] args) {
        Point[] points = { new Point(x: 1, y: 2), new Point(x: 2, y: 3), new Point(x: 1, y: 2), new Point(x: 4, y: 5), new Point(x: 2, y: 3) };
        List<Point> distinct = new ArrayList<>();
        for (Point p : points)
            if (!distinct.contains(p))
                distinct.add(p);
        System.out.println("Distinct: " + distinct.size());
    }
}

```

Code Screenshot 3

Fraction.java

Part A > Practical 3-A > Fraction.java > ...

```java
import java.util.Objects;

public class Fraction {
    private final int num, den;

    public Fraction(int num, int den) {
        if (den == 0)
            throw new IllegalArgumentException("Denominator cannot be zero");
        if (den < 0) {
            num = -num;
            den = -den;
        }
        int g = gcd(Math.abs(num), den);
        this.num = num / g;
        this.den = den / g;
    }

    private static int gcd(int a, int b) {
        while (b != 0) {
            int t = a % b;
            a = b;
            b = t;
        }
        return a == 0 ? 1 : a;
    }

    @Override
    public String toString() {
        return num + "/" + den;
    }

    @Override
    public boolean equals(Object o) {
        return o instanceof Fraction f && num == f.num && den == f.den;
    }

    @Override
    public int hashCode() {
        return Objects.hash(num, den);
    }

    Run | Debug
    public static void main(String[] args) {
        Fraction a = new Fraction(num: 2, den: 4), b = new Fraction(num: 1, den: 2);
        System.out.println(a);
        System.out.println("Equal: " + a.equals(b));
    }
}
```

Code Screenshot 4

Point.java

Part A > Practical 3-A > Point.java > ...

```java
import java.util.Objects;

public class Point {
    private final int x, y;

    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }

    @Override
    public String toString() {
        return "(" + x + ", " + y + ")";
    }

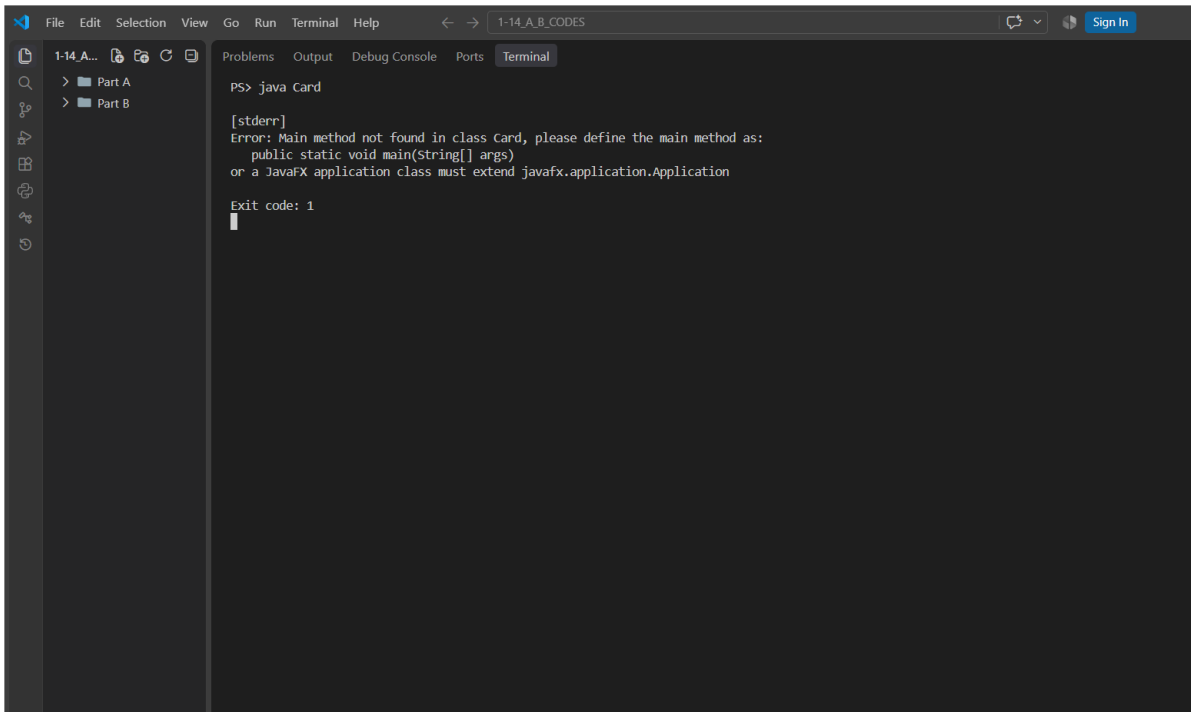
    @Override
    public boolean equals(Object o) {
        return o instanceof Point p && x == p.x && y == p.y;
    }

    @Override
    public int hashCode() {
        return Objects.hash(x, y);
    }
}

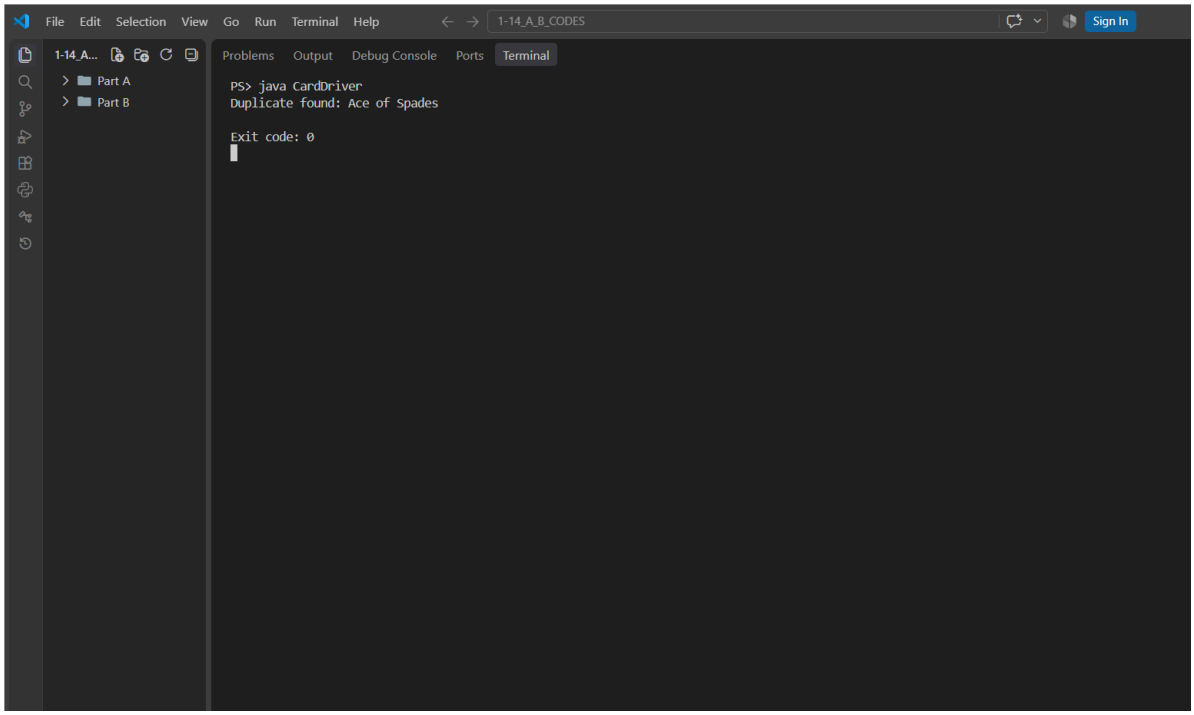
```

Code Screenshot 5

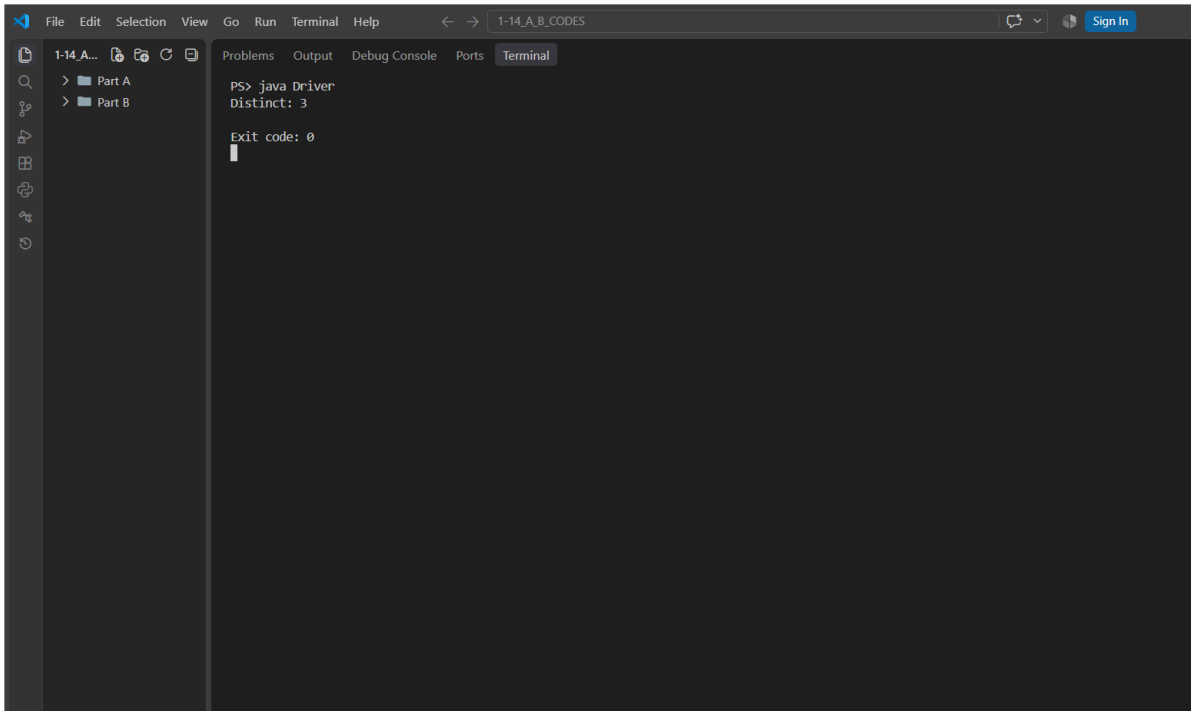
OUTPUT SCREENSHOT:



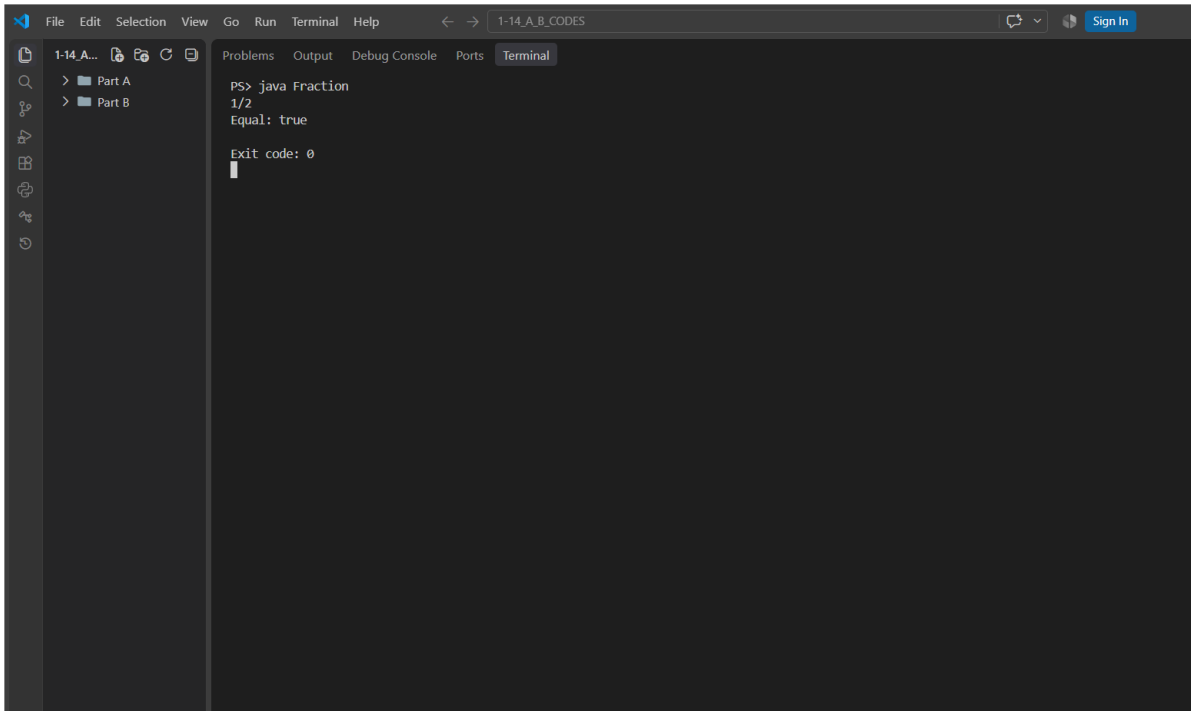
Output Screenshot 1



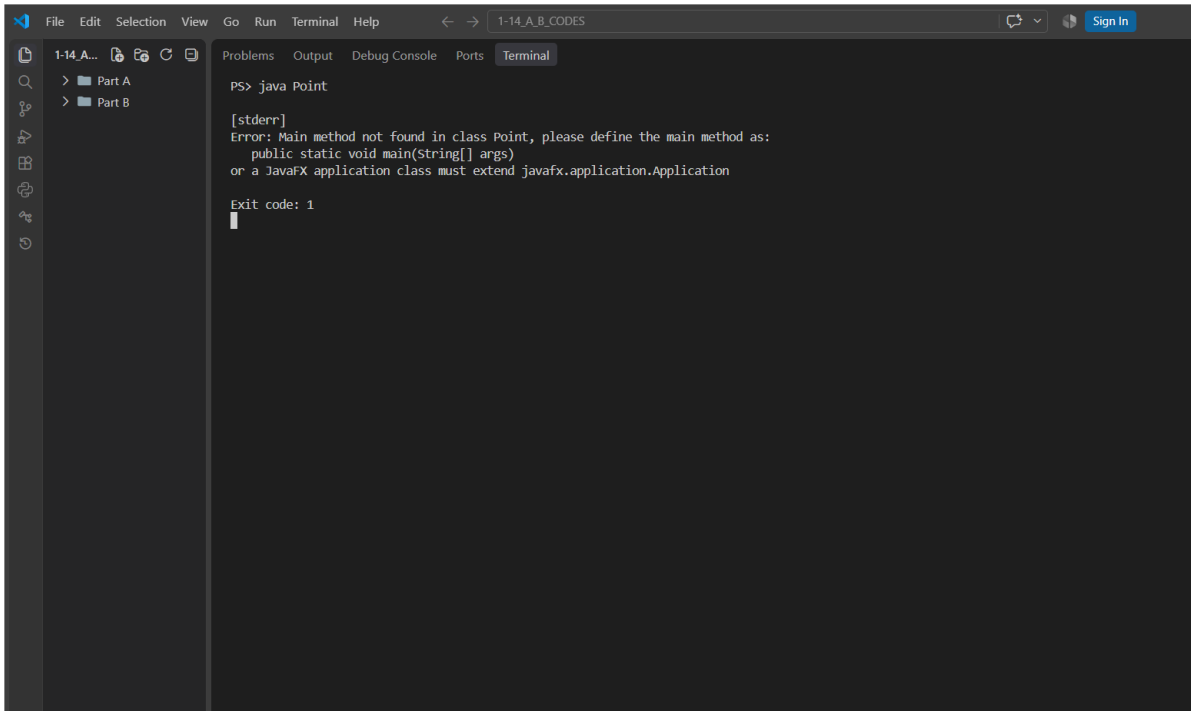
Output Screenshot 2



Output Screenshot 3



Output Screenshot 4



Output Screenshot 5

LEARNING OUTCOMES:

Override Object methods correctly and use equality, hashing, nested classes, and object identity checks.

PRACTICAL 4-A

Object Oriented Programming

AIM:

To implement and understand the concepts specified in Practical 4 of the CEUE203 Object Oriented Programming practical list, using Java. The work covers Part A: Password Checker, Chat Filter, Template Filler and Part B: Validators, Command Parser, Statement Formatter.

CODE SCREENSHOT:

ChatFilter.java 1

Part A > Practical 4-A > ChatFilter.java > ...

```java
import java.util.*;

public class ChatFilter {
    Run | Debug
    public static void main(String[] args) {
        String[] logs = { "10:05 alice Hello there", "10:06 bob How are you", "malformed", "10:07 cara HELLO again" };
        Scanner sc = new Scanner(System.in);
        System.out.print("Keyword: ");
        String key = sc.nextLine().toLowerCase();
        StringBuilder report = new StringBuilder();
        int matches = 0;
        for (String line : logs) {
            String[] parts = line.split(" ", 3);
            if (parts.length < 3)
                continue;
            if (parts[2].toLowerCase().contains(key)) {
                matches++;
                report.append(parts[0]).append(" ").append(parts[1]).append(": ").append(parts[2])
                        .append(System.lineSeparator());
            }
        }
        System.out.println("Matches: " + matches);
        System.out.print(report);
    }
}

```

Code Screenshot 1

PasswordChecker.java

Part A > Practical 4-A > PasswordChecker.java > ...

```java
public class PasswordChecker {
    public static String strength(String pw) {
        int score = 0;
        if (pw.length() >= 8)
            score++;
        if (pw.matches(".*[A-Z].*"))
            score++;
        if (pw.matches(".*\\d.*"))
            score++;
        if (pw.matches(".*[^A-Za-z0-9].*"))
            score++;
        return score <= 1 ? "Weak" : score <= 3 ? "Medium" : "Strong";
    }

    Run | Debug
    public static void main(String[] args) {
        for (String p : new String[] { "abc", "Abcd1234!", "hello123", "HELLO!!" })
            System.out.println(p + " -> " + strength(p));
    }
}

```

Code Screenshot 2

TemplateFiller.java

Part A > Practical 4-A > TemplateFiller.java > ...

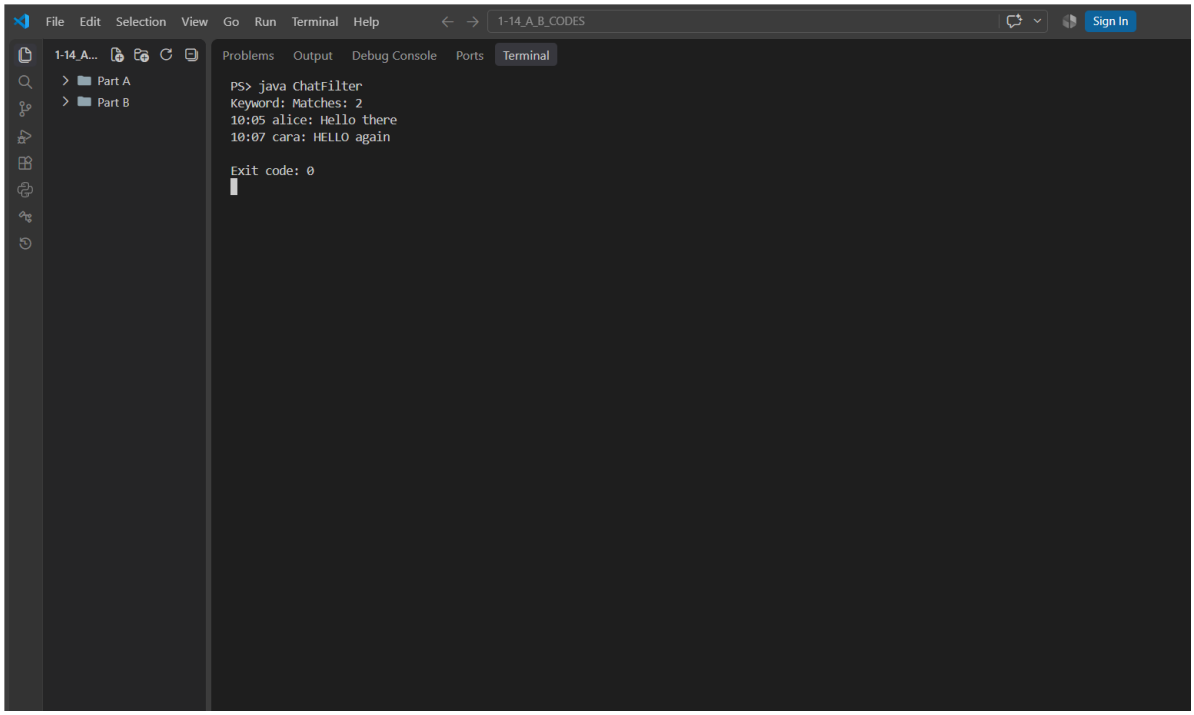
```java
import java.util.regex.*;

public class TemplateFiller {
    Run | Debug
    public static void main(String[] args) {
        String template = "Dear {name}, order {id} ships {date}.";
        String[] names = { "name", "id" }, values = { "Riya", "A07" };
        Matcher m = Pattern.compile("\\{(\\w+)\\}").matcher(template);
        StringBuilder out = new StringBuilder();
        int last = 0;
        while (m.find()) {
            out.append(template, last, m.start());
            String key = m.group(1), value = "[?]";
            for (int i = 0; i < names.length; i++)
                if (names[i].equals(key))
                    value = values[i];
            out.append(value);
            last = m.end();
        }
        out.append(template.substring(last));
        System.out.println(out);
    }
}

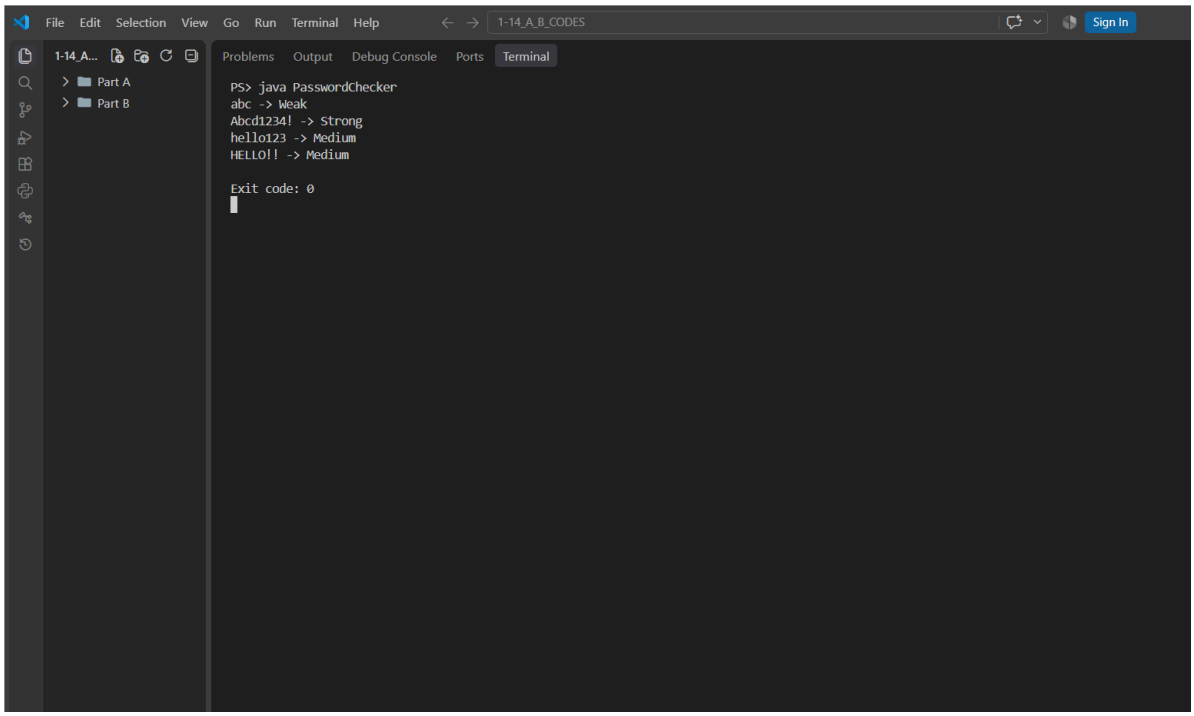
```

Code Screenshot 3

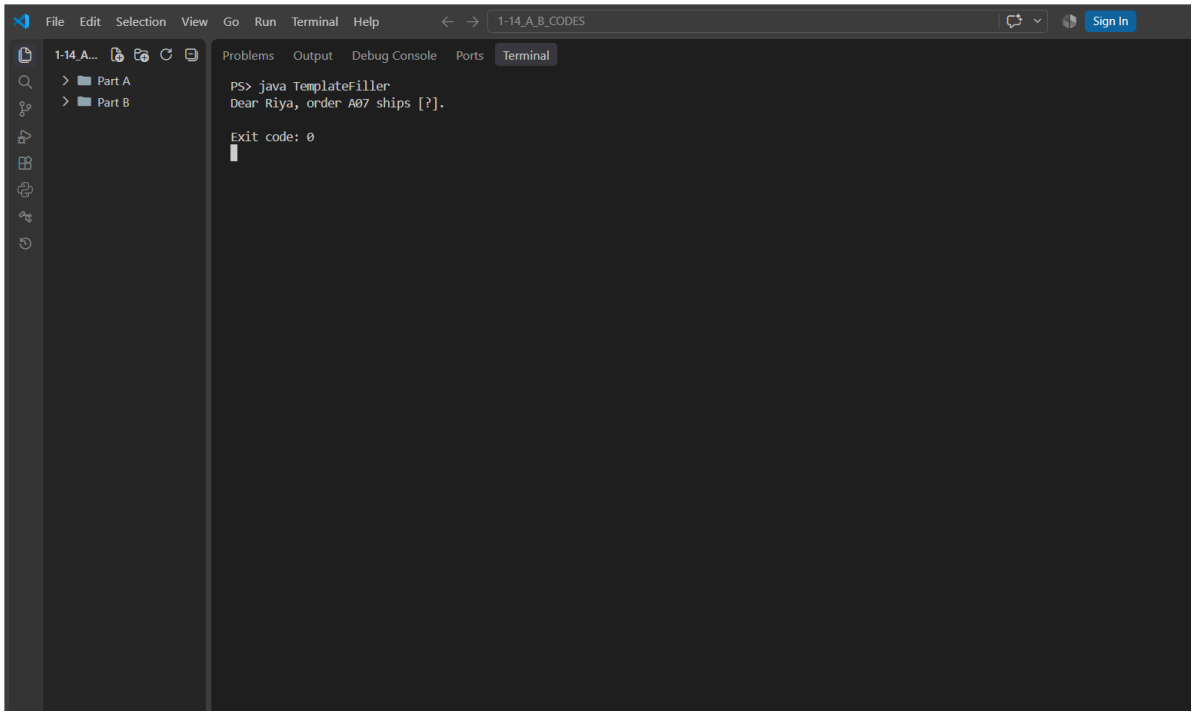
OUTPUT SCREENSHOT:



Output Screenshot 1



Output Screenshot 2



Output Screenshot 3

LEARNING OUTCOMES:

Validate and parse text using String methods, StringBuilder, regular expressions, and reusable helper classes.

PRACTICAL 5-A

Object Oriented Programming

AIM:

To implement and understand the concepts specified in Practical 5 of the CEUE203 Object Oriented Programming practical list, using Java. The work covers Part A: Shape Areas, Payroll, Media Late Fees and Part B: Abstract Account subclasses and polymorphism.

CODE SCREENSHOT:



Code Screenshot 1

Payroll.java

Part A > Practical 5-A > Payroll.java > ...

```java
abstract class Employee {
    String name, id;

    Employee(String n, String i) {
        name = n;
        id = i;
    }

    abstract double monthlySalary();
}

class FullTime extends Employee {
    double salary;

    FullTime(String n, String i, double s) {
        super(n, i);
        salary = s;
    }

    double monthlySalary() {
        return salary;
    }
}

class PartTime extends Employee {
    int hours;
    double rate;

    PartTime(String n, String i, int h, double r) {
        super(n, i);
        hours = h;
        rate = r;
    }

    double monthlySalary() {
        return hours * rate;
    }
}

class Intern extends Employee {
    double stipend;

    Intern(String n, String i, double s) {
        super(n, i);
        stipend = s;
    }

    double monthlySalary() {
```

Code Screenshot 2

ShapeDemo.java

Part A > Practical 5-A > ShapeDemo.java > ...

```java
abstract class Shape {
    abstract double area();
}

class Circle extends Shape {
    double r;

    Circle(double r) {
        this.r = r;
    }

    double area() {
        return Math.PI * r * r;
    }
}

class Rectangle extends Shape {
    double w, h;

    Rectangle(double w, double h) {
        this.w = w;
        this.h = h;
    }

    double area() {
        return w * h;
    }
}

class Triangle extends Shape {
    double b, h;

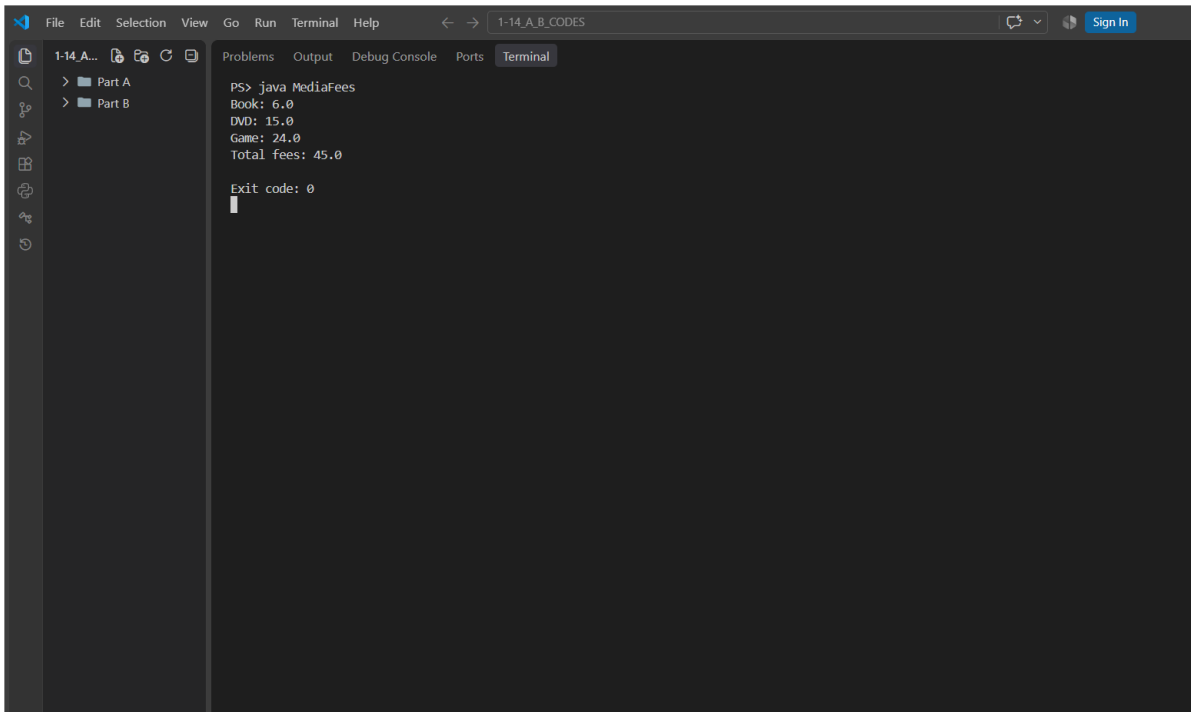
    Triangle(double b, double h) {
        this.b = b;
        this.h = h;
    }

    double area() {
        return b * h / 2;
    }
}

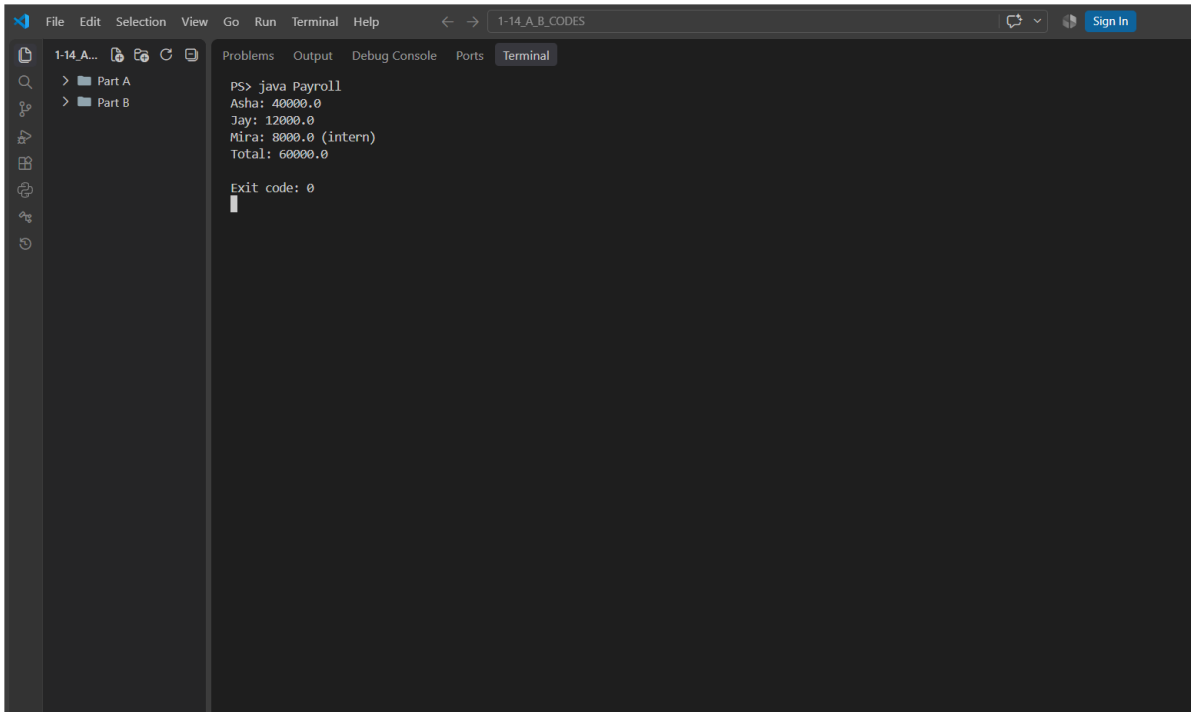
public class ShapeDemo {
    Run | Debug
    public static void main(String[] args) {
        Shape[] shapes = { new Circle(r: 2), new Rectangle(w: 3, h: 4), new Triangle(b: 3, h: 5) };
        double total = 0, largest = 0;
        for (Shape s : shapes) {
```

Code Screenshot 3

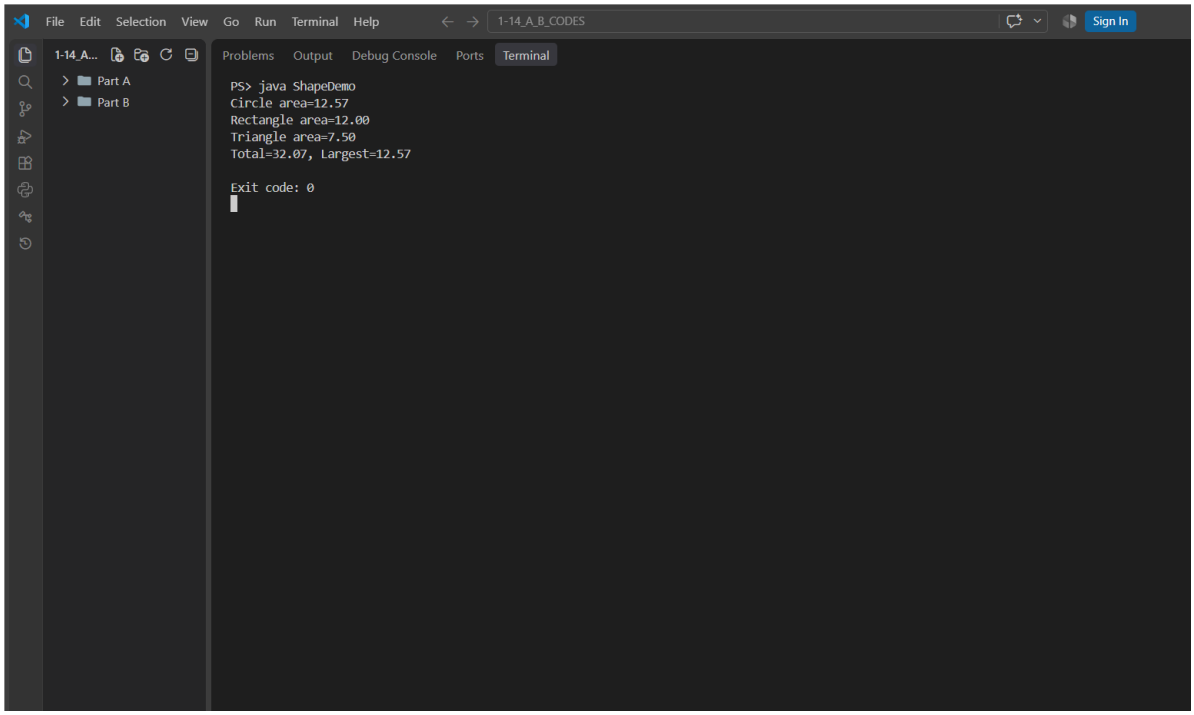
OUTPUT SCREENSHOT:



Output Screenshot 1



Output Screenshot 2



Output Screenshot 3

LEARNING OUTCOMES:

Use inheritance, abstract classes, method overriding, and runtime polymorphism to model related types.

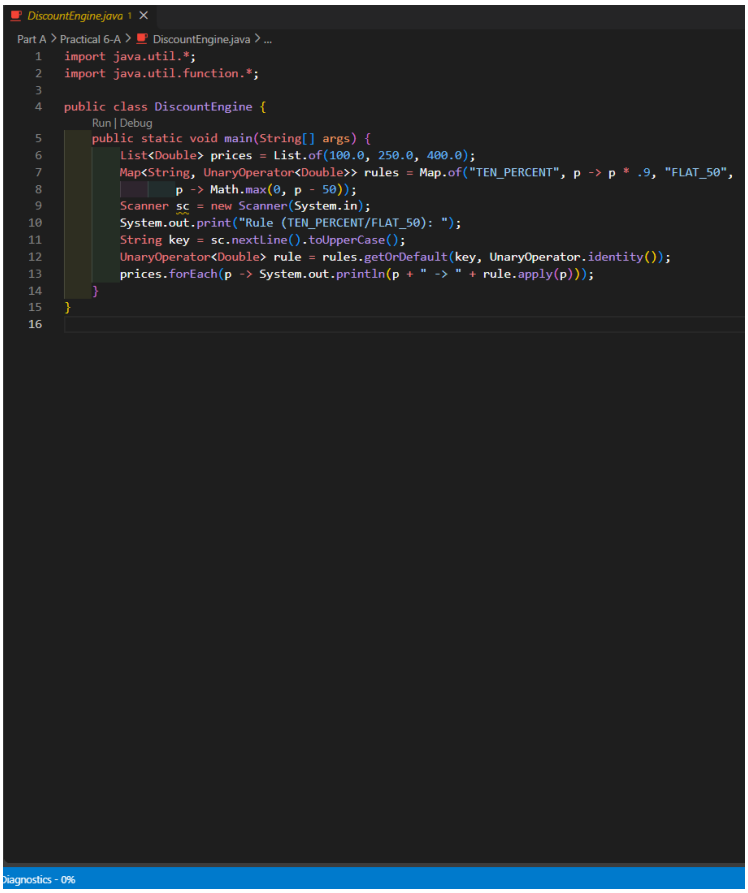
PRACTICAL 6-A

Object Oriented Programming

AIM:

To implement and understand the concepts specified in Practical 6 of the CEUE203 Object Oriented Programming practical list, using Java. The work covers Part A: Remote Control, Notification Senders, Discount Engine and Part B: Interfaces, lambda rule, packages and JAR concepts.

CODE SCREENSHOT:



Code Screenshot 1



```java
@FunctionalInterface
interface Notifier {
    void send(String message);
}

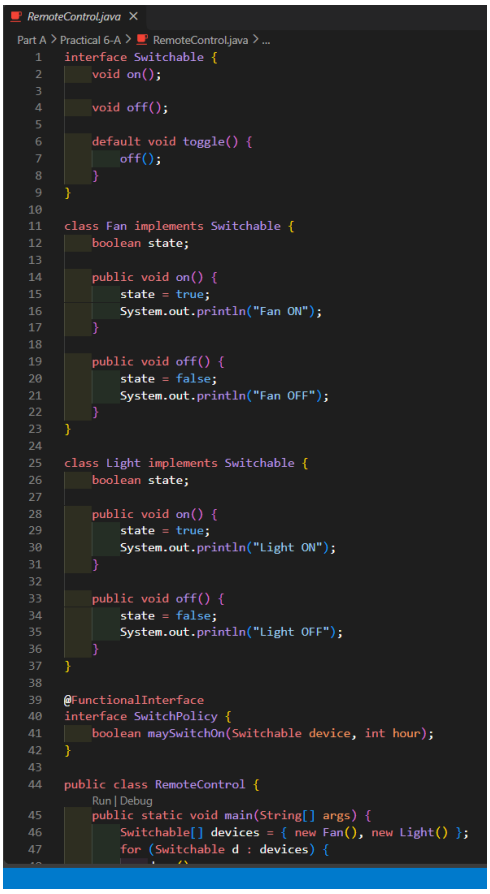
interface Urgent {
}

class EmailNotifier implements Notifier {
    public void send(String m) {
        System.out.println("Email: " + m);
    }
}

class SmsNotifier implements Notifier, Urgent {
    public void send(String m) {
        System.out.println("SMS: " + m);
    }
}

public class NotificationSenders {
    Run | Debug
    public static void main(String[] args) {
        Notifier[] senders = { new EmailNotifier(), new SmsNotifier() };
        for (Notifier n : senders) {
            n.send(message: "System maintenance at 8 PM");
            if (n instanceof Urgent)
                n.send(message: "URGENT: System maintenance at 8 PM");
        }
    }
}
```

Code Screenshot 2



```java
interface Switchable {
    void on();

    void off();

    default void toggle() {
        off();
    }
}

class Fan implements Switchable {
    boolean state;

    public void on() {
        state = true;
        System.out.println("Fan ON");
    }

    public void off() {
        state = false;
        System.out.println("Fan OFF");
    }
}

class Light implements Switchable {
    boolean state;

    public void on() {
        state = true;
        System.out.println("Light ON");
    }

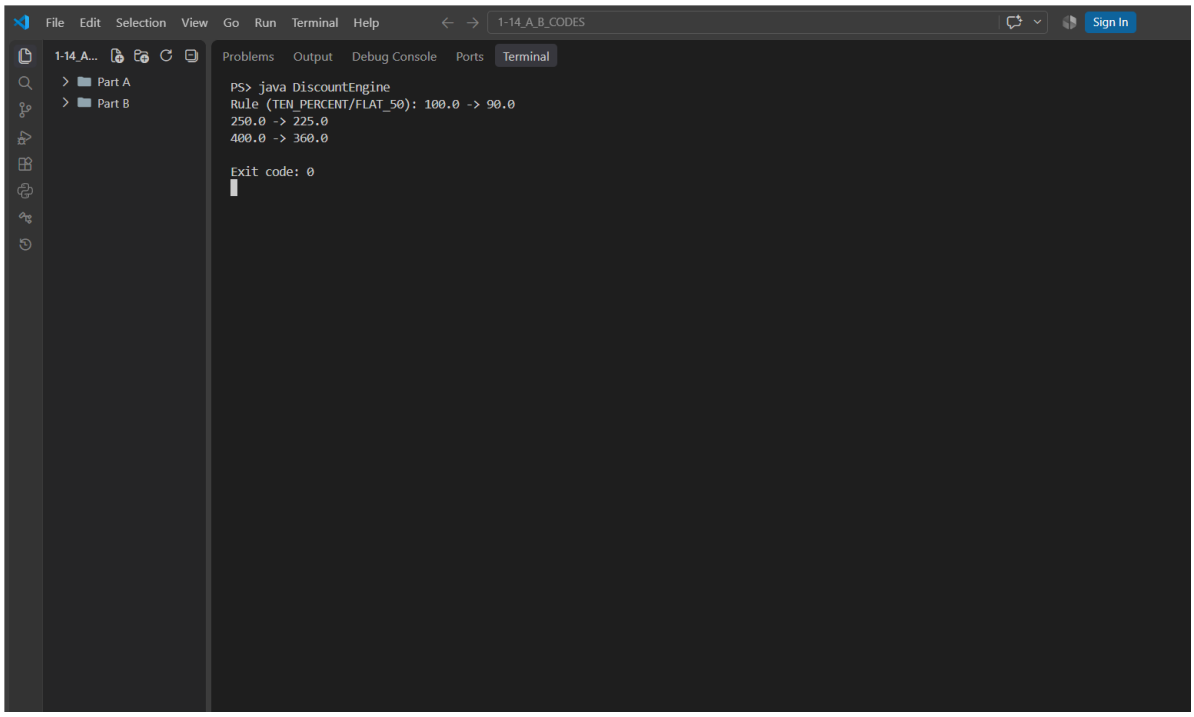
    public void off() {
        state = false;
        System.out.println("Light OFF");
    }
}

@FunctionalInterface
interface SwitchPolicy {
    boolean maySwitchOn(Switchable device, int hour);
}

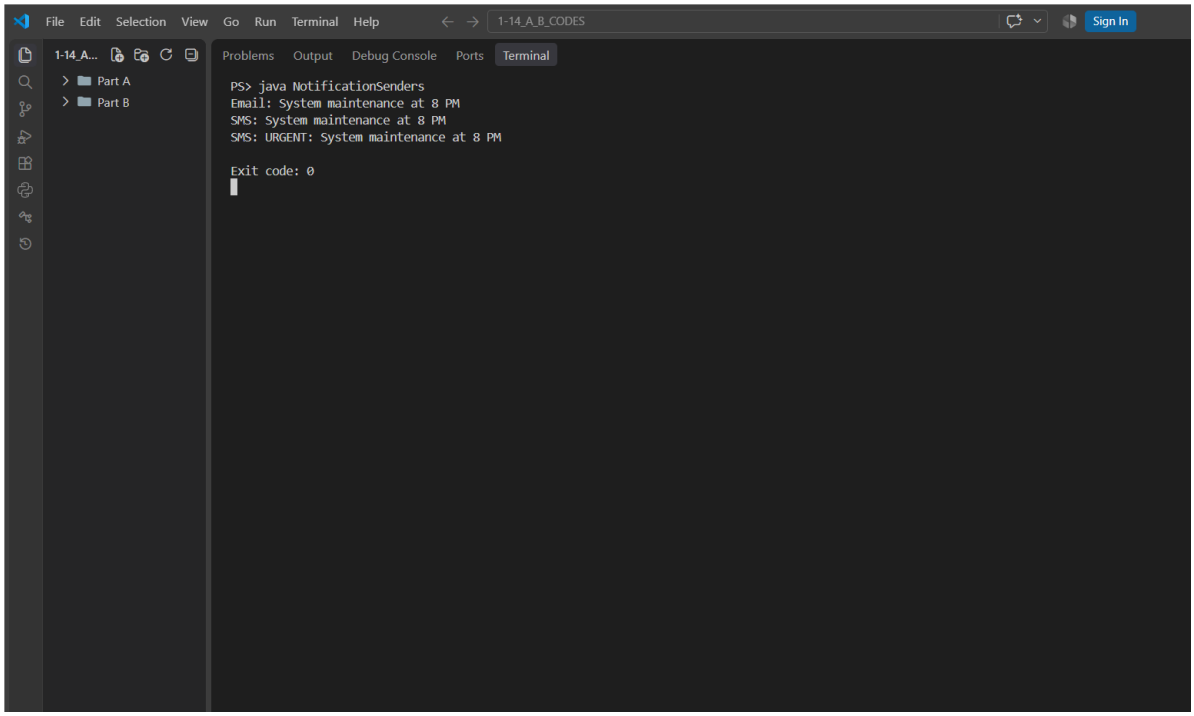
public class RemoteControl {
    Run | Debug
    public static void main(String[] args) {
        Switchable[] devices = { new Fan(), new Light() };
        for (Switchable d : devices) {
```

Code Screenshot 3

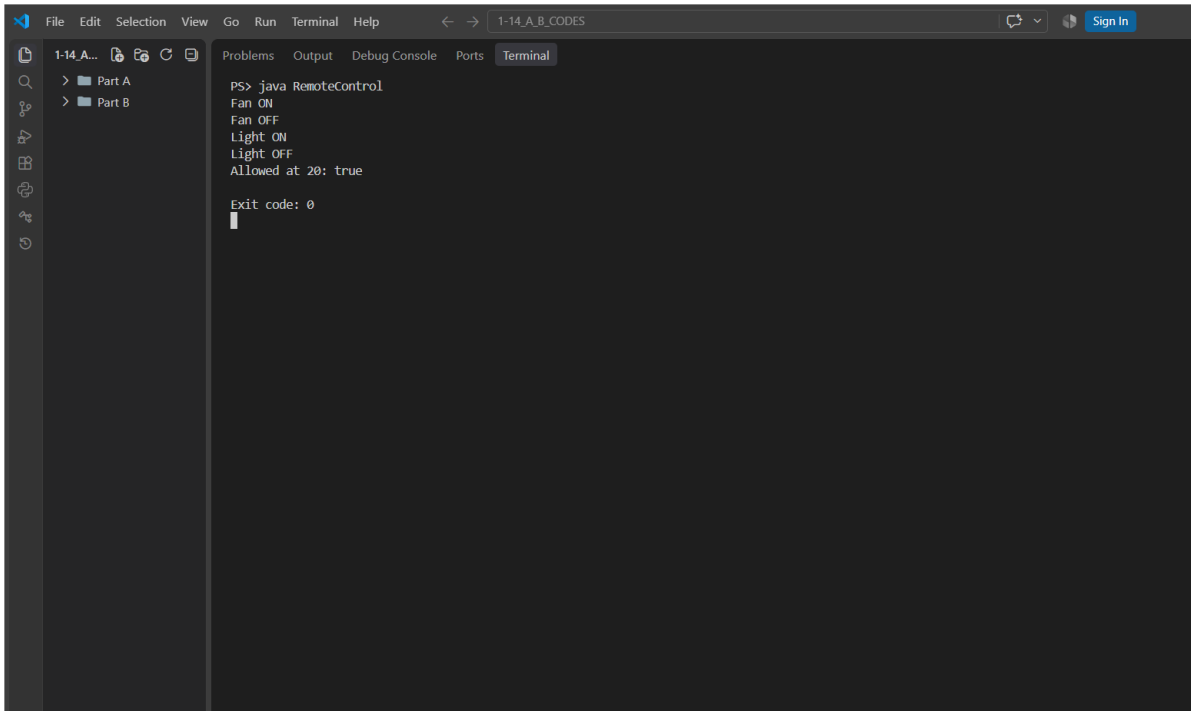
OUTPUT SCREENSHOT:



Output Screenshot 1



Output Screenshot 2



Output Screenshot 3

LEARNING OUTCOMES:

Design programs with interfaces, functional interfaces, lambda expressions, and default methods.

PRACTICAL 7-A

Object Oriented Programming

AIM:

To implement and understand the concepts specified in Practical 7 of the CEUE203 Object Oriented Programming practical list, using Java. The work covers Part A: Annotation Form Validator, Mini Test Runner, Column Mapper and Part B: Custom annotations and reflection validator.

CODE SCREENSHOT:

ColumnMapper.java

Part A > Practical 7-A > ColumnMapper.java > ...

```java
import java.lang.annotation.*;
import java.lang.reflect.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface Column {
    String name();
}

class Person {
    @Column(name = "name")
    String name;
    @Column(name = "age")
    int age;

    public String toString() {
        return name + " (" + age + ")";
    }
}

public class ColumnMapper {
    Run | Debug
    public static void main(String[] args) throws Exception {
        String[] header = { "age", "name" }, row = { "20", "Anshul" };
        Person p = new Person();
        for (Field f : Person.class.getDeclaredFields()) {
            Column c = f.getAnnotation(Column.class);
            for (int i = 0; i < header.length; i++)
                if (c.name().equals(header[i])) {
                    f.setAccessible(true);
                    if (f.getType() == int.class)
                        f.setInt(p, Integer.parseInt(row[i]));
                    else
                        f.set(p, row[i]);
                }
        }
        System.out.println(p);
    }
}

```

Code Screenshot 1

FormValidator.java

Part A > Practical 7-A > FormValidator.java > ...

```java
import java.lang.annotation.*;
import java.lang.reflect.*;
import java.util.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface NotBlank {
}

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface MaxLength {
    int value();
}

class SignupForm {
    @NotBlank
    String name = " ";
    @MaxLength(10)
    String username = "verylongusername";

    SignupForm(String n, String u) {
        name = n;
        username = u;
    }
}

public class FormValidator {
    Run | Debug
    public static void main(String[] args) throws Exception {
        SignupForm form = new SignupForm(n: " ", u: "verylongusername");
        List<String> errors = new ArrayList<>();
        for (Field f : form.getClass().getDeclaredFields()) {
            f.setAccessible(true);
            Object v = f.get(form);
            if (f.isAnnotationPresent(NotBlank.class) && (!(v instanceof String) || ((String) v).isBlank()))
                errors.add(f.getName() + " must not be blank");
            MaxLength max = f.getAnnotation(MaxLength.class);
            if (max != null && v instanceof String s && s.length() > max.value())
                errors.add(f.getName() + " exceeds " + max.value());
        }
        System.out.println(errors);
    }
}

```

Code Screenshot 2

MiniTestRunner.java

Part A > Practical 7-A > MiniTestRunner.java > ...

```java
import java.lang.annotation.*;
import java.lang.reflect.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface Run {
}

class DemoTests {
    @Run
    void testOne() {
        System.out.println("testOne passed");
    }

    @Run
    void testTwo() {
        System.out.println("testTwo passed");
    }

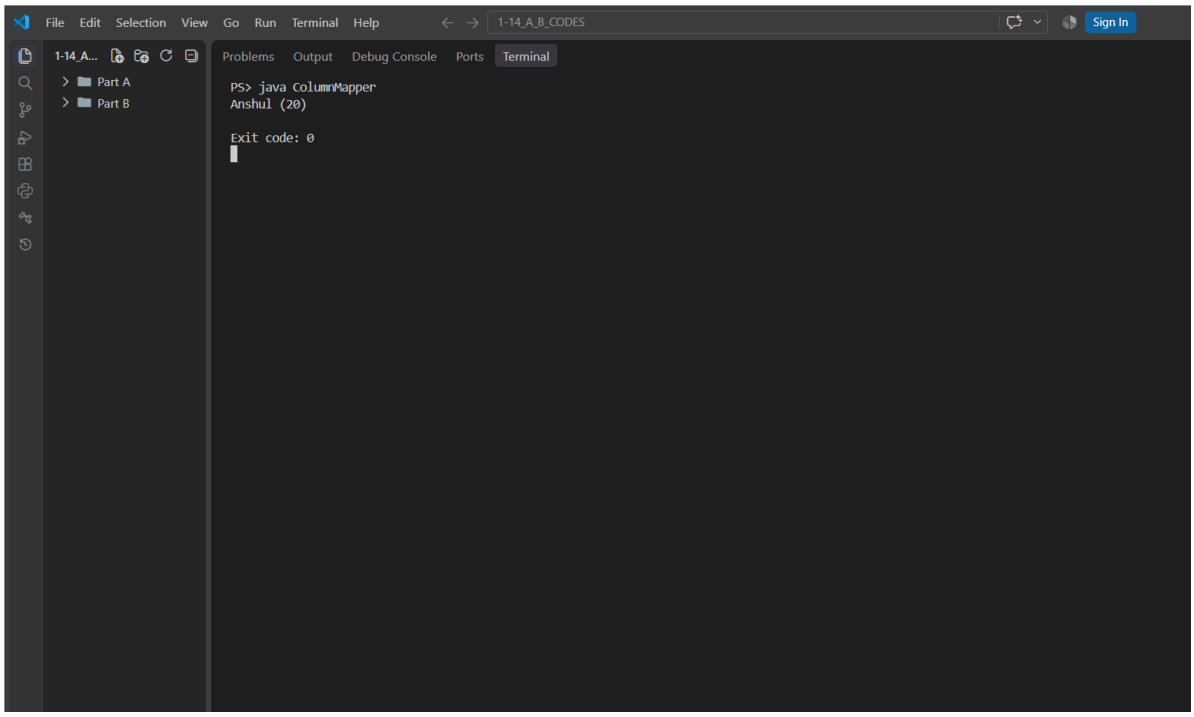
    void helper() {
        System.out.println("ignored");
    }
}

public class MiniTestRunner {
    Run | Debug
    public static void main(String[] args) throws Exception {
        Object test = new DemoTests();
        int count = 0;
        for (Method m : test.getClass().getDeclaredMethods())
            if (m.isAnnotationPresent(Run.class)) {
                m.invoke(test);
                count++;
            }
        System.out.println("Tests run: " + count);
    }
}

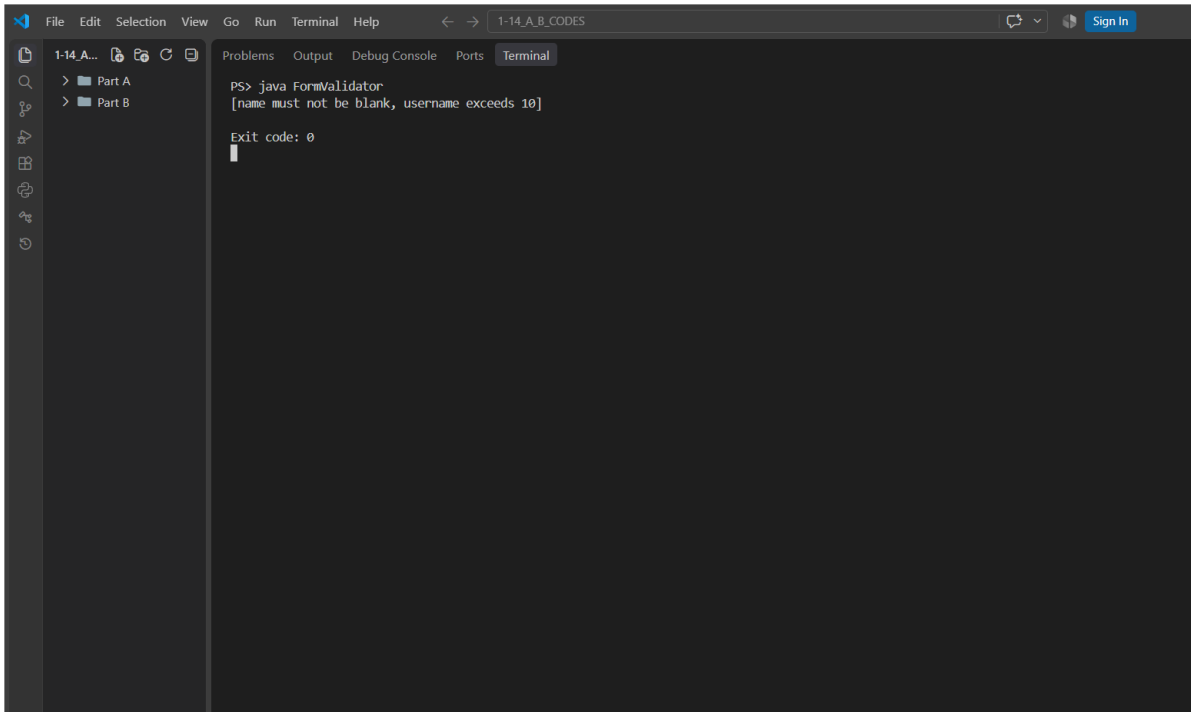
```

Code Screenshot 3

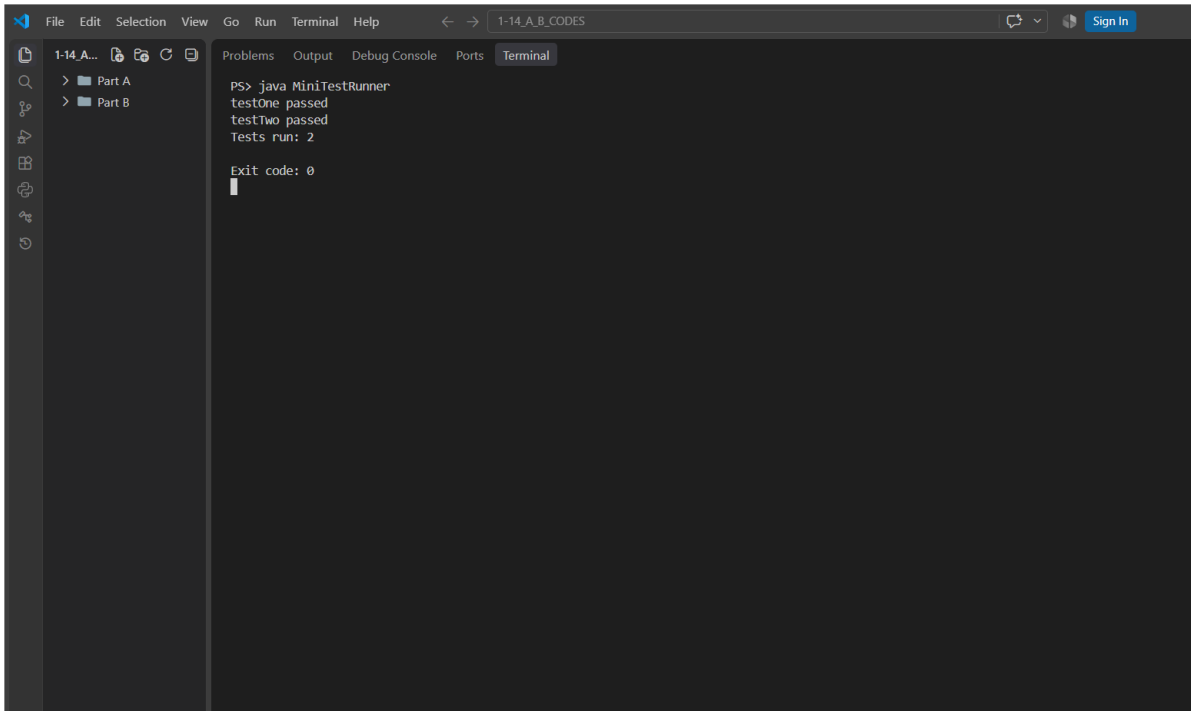
OUTPUT SCREENSHOT:



Output Screenshot 1



Output Screenshot 2



Output Screenshot 3

LEARNING OUTCOMES:

Create runtime annotations and inspect object fields through reflection for validation.

PRACTICAL 8-A

Object Oriented Programming

AIM:

To implement and understand the concepts specified in Practical 8 of the CEUE203 Object Oriented Programming practical list, using Java. The work covers Part A: Guarded Calculator, Warehouse Exceptions, AutoCloseable and Part B: Bank exception hierarchy and transfer handling.

CODE SCREENSHOT:

GuardedCalculator.java 1

Part A > Practical 8-A > GuardedCalculator.java > ...

```java
import java.util.*;

class DivideByZeroException extends Exception {
    DivideByZeroException(String m) {
        super(m);
    }
}

public class GuardedCalculator {
    Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        while (true) {
            try {
                System.out.print("First number (q to quit): ");
                String x = sc.next();
                if (x.equalsIgnoreCase("q"))
                    break;
                double a = Double.parseDouble(x);
                System.out.print("Operator and second number: ");
                String op = sc.next();
                double b = Double.parseDouble(sc.next());
                double result;
                switch (op) {
                    case "+" -> result = a + b;
                    case "-" -> result = a - b;
                    case "*" -> result = a * b;
                    case "/" -> {
                        if (b == 0)
                            throw new DivideByZeroException(m: "Cannot divide by zero");
                        result = a / b;
                    }
                    default -> throw new IllegalArgumentException("Unknown operator");
                }
                System.out.println("Result: " + result);
                break;
            } catch (NumberFormatException e) {
                System.out.println("Invalid number");
            } catch (DivideByZeroException | IllegalArgumentException e) {
                System.out.println(e.getMessage());
            } finally {
                System.out.println("Attempt logged");
            }
        }
    }
}
```

Code Screenshot 1

ResourceDemo.java

Part A > Practical 8-A > ResourceDemo.java > ...

```java
class SampleResource implements AutoCloseable {
    public void use() {
        System.out.println("Using resource");
    }

    public void close() {
        System.out.println("Resource closed");
    }
}

public class ResourceDemo {
    Run | Debug
    public static void main(String[] args) {
        try (SampleResource r = new SampleResource()) {
            r.use();
            if (args.length > 0)
                throw new RuntimeException("Demo failure");
        } catch (RuntimeException e) {
            System.out.println("Reported: " + e.getMessage());
        }
    }
}

```

Code Screenshot 2

Warehouse.java

Part A > Practical 8-A > Warehouse.java > ...

```java
import java.util.*;

class OutOfStockException extends Exception {
    final int shortfall;

    OutOfStockException(int s) {
        super("Out of stock by " + s);
        shortfall = s;
    }
}

class InvalidQuantityException extends Exception {
    InvalidQuantityException() {
        super("Quantity must be positive");
    }
}

public class Warehouse {
    private final Map<String, Integer> stock = new HashMap<>(Map.of("pen", 5, "book", 2));

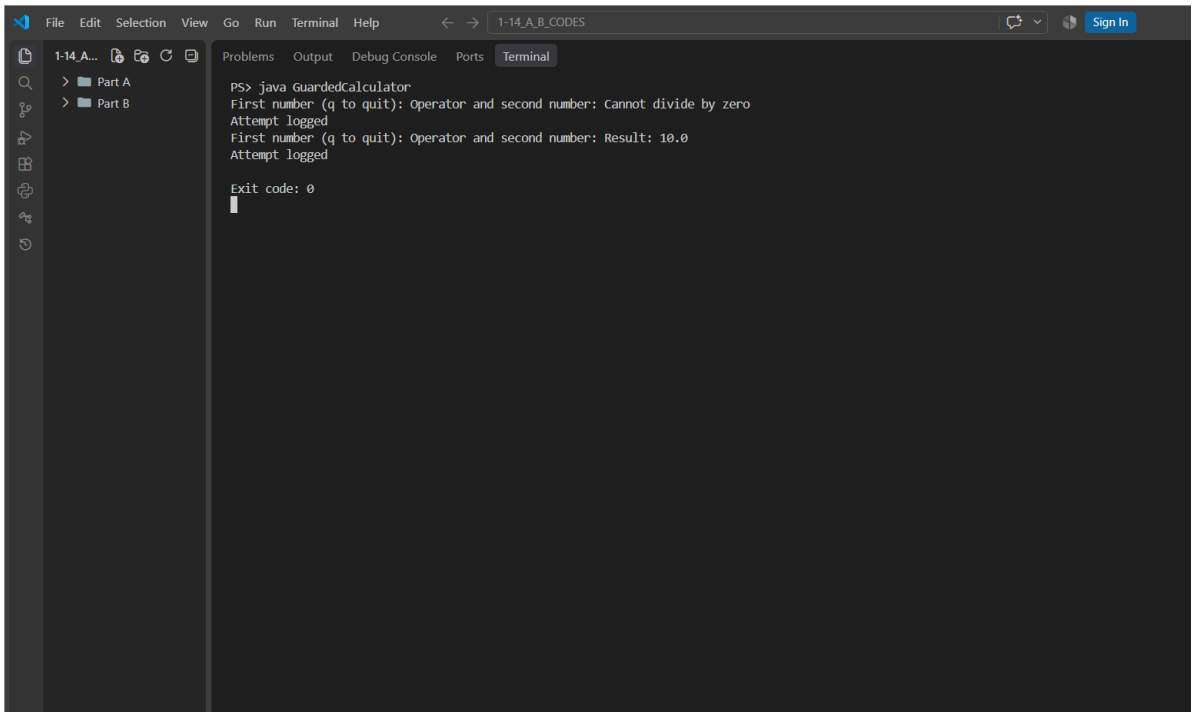
    public void issue(String item, int qty) throws OutOfStockException, InvalidQuantityException {
        if (qty <= 0)
            throw new InvalidQuantityException();
        int available = stock.getOrDefault(item, 0);
        if (qty > available)
            throw new OutOfStockException(qty - available);
        stock.put(item, available - qty);
        System.out.println("Issued " + qty + " " + item);
    }

    Run | Debug
    public static void main(String[] args) {
        Warehouse w = new Warehouse();
        Object[][] req = { { "pen", 2 }, { "book", 5 }, { "pen", 0 } };
        for (Object[] r : req)
            try {
                w.issue((String) r[0], (int) r[1]);
            } catch (Exception e) {
                System.out.println(e.getMessage());
            }
    }
}

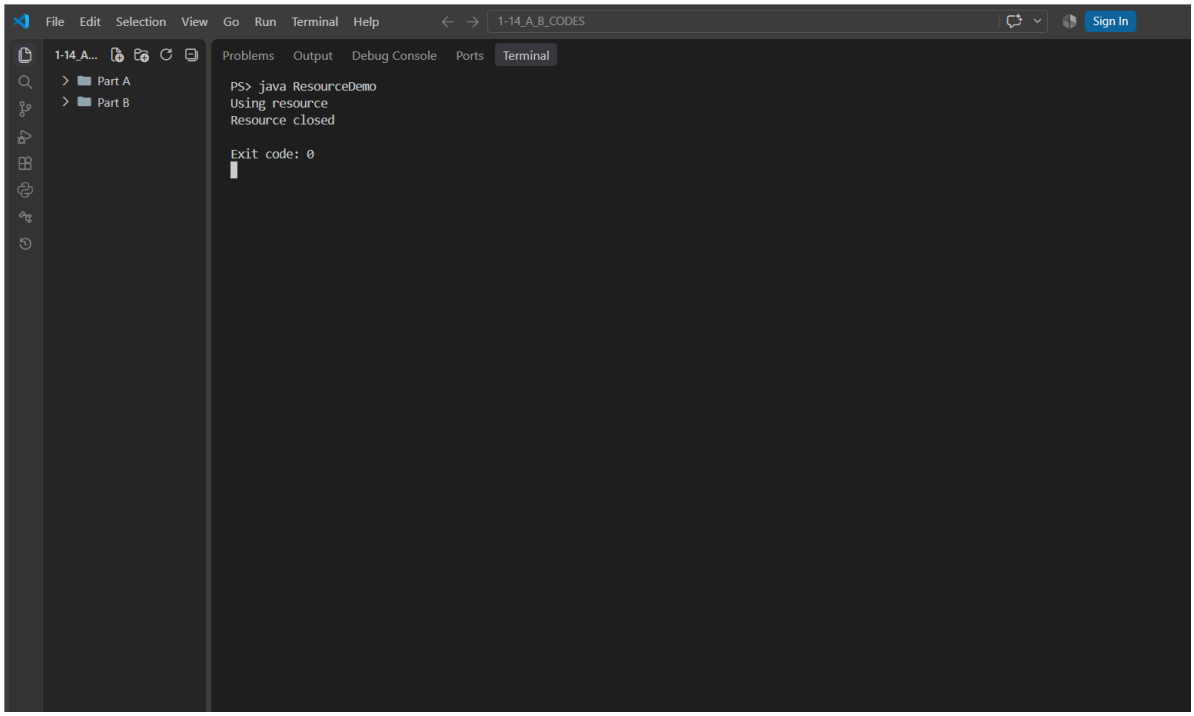
```

Code Screenshot 3

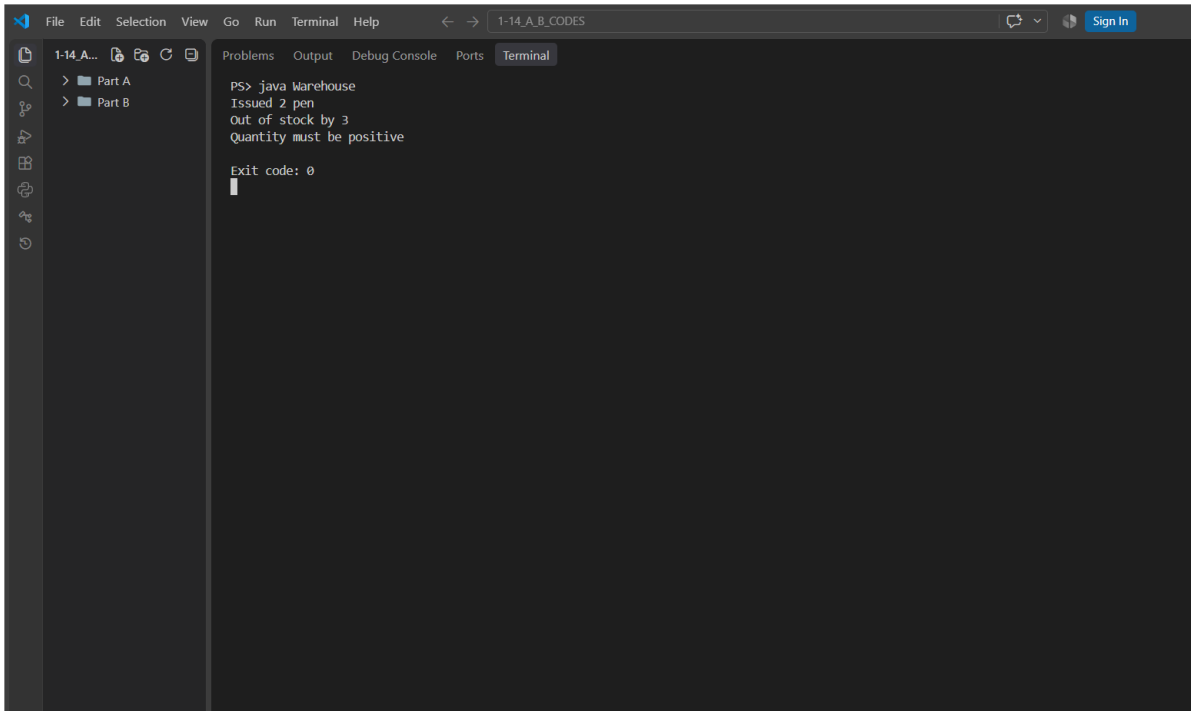
OUTPUT SCREENSHOT:



Output Screenshot 1



Output Screenshot 2



Output Screenshot 3

LEARNING OUTCOMES:

Handle invalid operations safely with custom exceptions, try-catch-finally, and try-with-resources.

PRACTICAL 9-A

Object Oriented Programming

AIM:

To implement and understand the concepts specified in Practical 9 of the CEUE203 Object Oriented Programming practical list, using Java. The work covers Part A: Counter Race, Seat Booking Race, Parallel Sum and Part B: Runnable workers and synchronized account operations.

CODE SCREENSHOT:

CounterRace.java

Part A > Practical 9-A > CounterRace.java > ...

```java
public class CounterRace {
    static class Counter {
        int value = 0;

        synchronized void increment() {
            value++;
        }
    }

    Run | Debug
    public static void main(String[] args) throws InterruptedException {
        Counter c = new Counter();
        Thread[] ts = new Thread[10];
        for (int i = 0; i < ts.length; i++) {
            ts[i] = new Thread(() -> {
                for (int j = 0; j < 1000; j++)
                    c.increment();
            });
            ts[i].start();
        }
        for (Thread t : ts)
            t.join();
        System.out.println("Final counter: " + c.value + " (expected 10000)");
    }
}

```

Code Screenshot 1

ParallelSum.java

Part A > Practical 9-A > ParallelSum.java > ...

```java
import java.util.*;

public class ParallelSum {
    Run | Debug
    public static void main(String[] args) throws InterruptedException {
        int[] values = new int[10000];
        Arrays.fill(values, 1);
        long[] sums = new long[4];
        Thread[] ts = new Thread[4];
        for (int i = 0; i < 4; i++) {
            final int k = i;
            ts[i] = new Thread(() -> {
                for (int j = k * 2500; j < (k + 1) * 2500; j++)
                    sums[k] += values[j];
            });
            ts[i].start();
        }
        for (Thread t : ts)
            t.join();
        System.out.println("Parallel sum: " + Arrays.stream(sums).sum());
    }
}

```

Code Screenshot 2

SeatBooking.java

Part A > Practical 9-A > SeatBooking.java > ...

```java
public class SeatBooking {
    static class Cinema {
        int seatsLeft = 5;

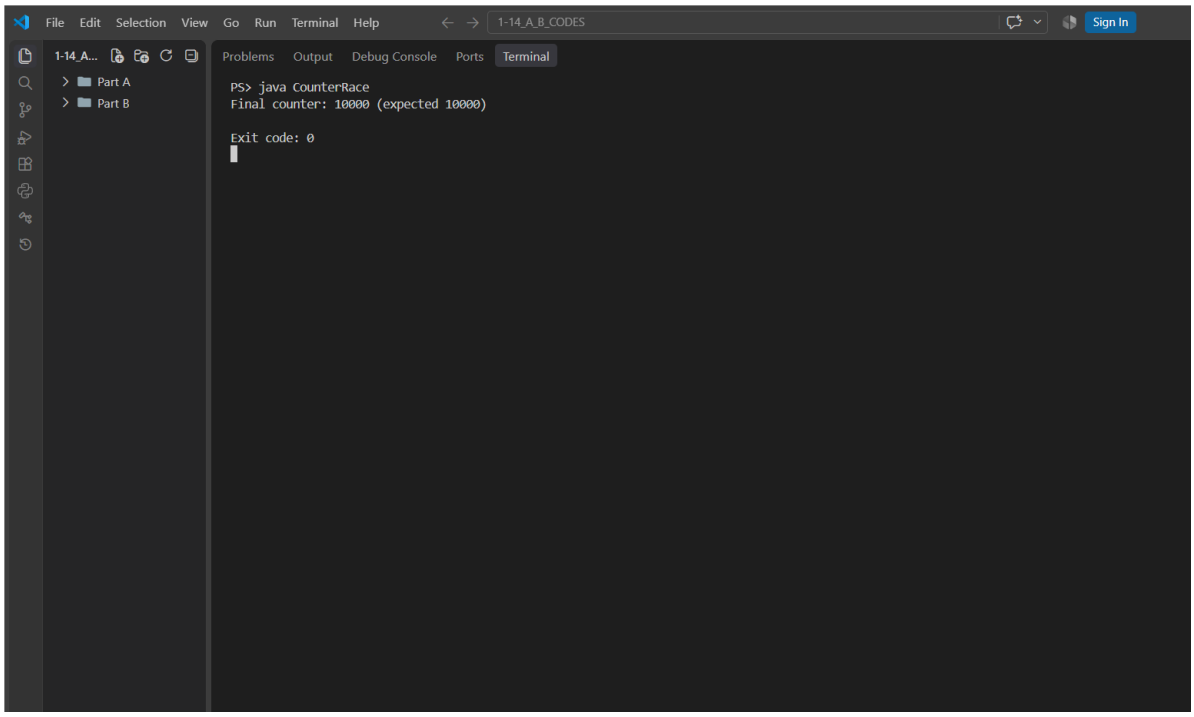
        synchronized boolean book() {
            if (seatsLeft > 0) {
                seatsLeft--;
                return true;
            }
            return false;
        }
    }

    Run | Debug
    public static void main(String[] args) throws InterruptedException {
        Cinema c = new Cinema();
        int[] booked = { 0 };
        Thread[] ts = new Thread[10];
        for (int i = 0; i < 10; i++) {
            ts[i] = new Thread(() -> {
                if (c.book())
                    synchronized (booked) {
                        booked[0]++;
                    }
            });
            ts[i].start();
        }
        for (Thread t : ts)
            t.join();
        System.out.println("Booked: " + booked[0] + ", seats left: " + c.seatsLeft);
    }
}

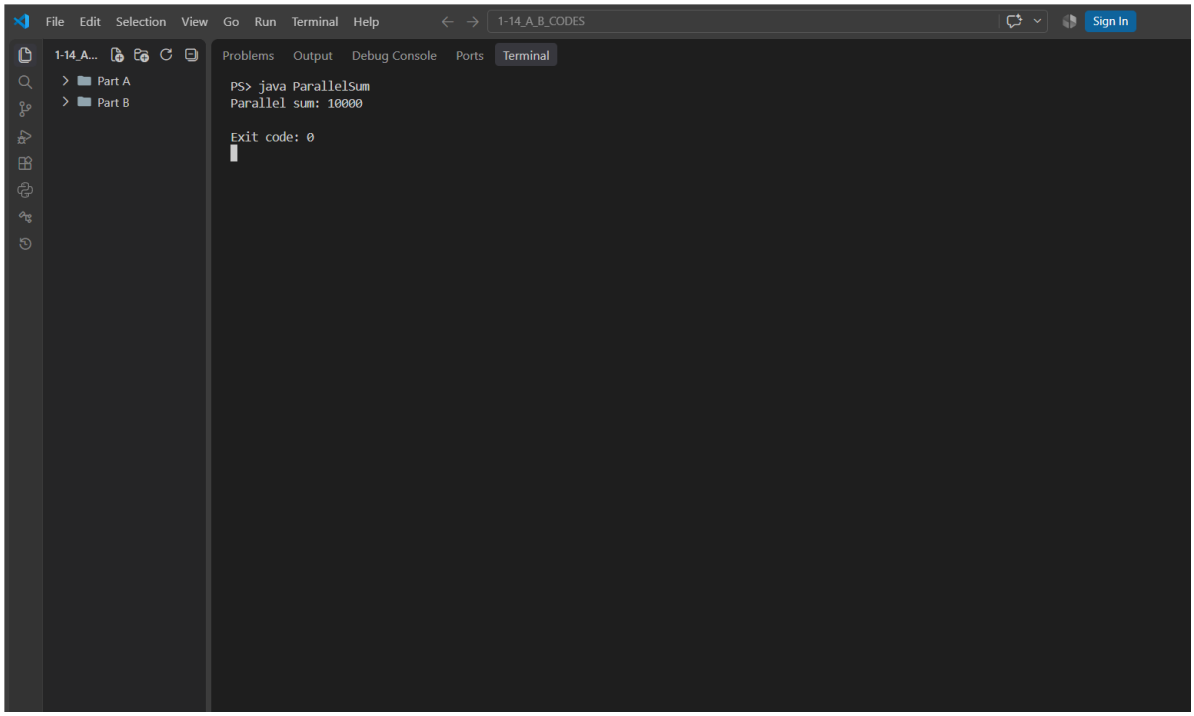
```

Code Screenshot 3

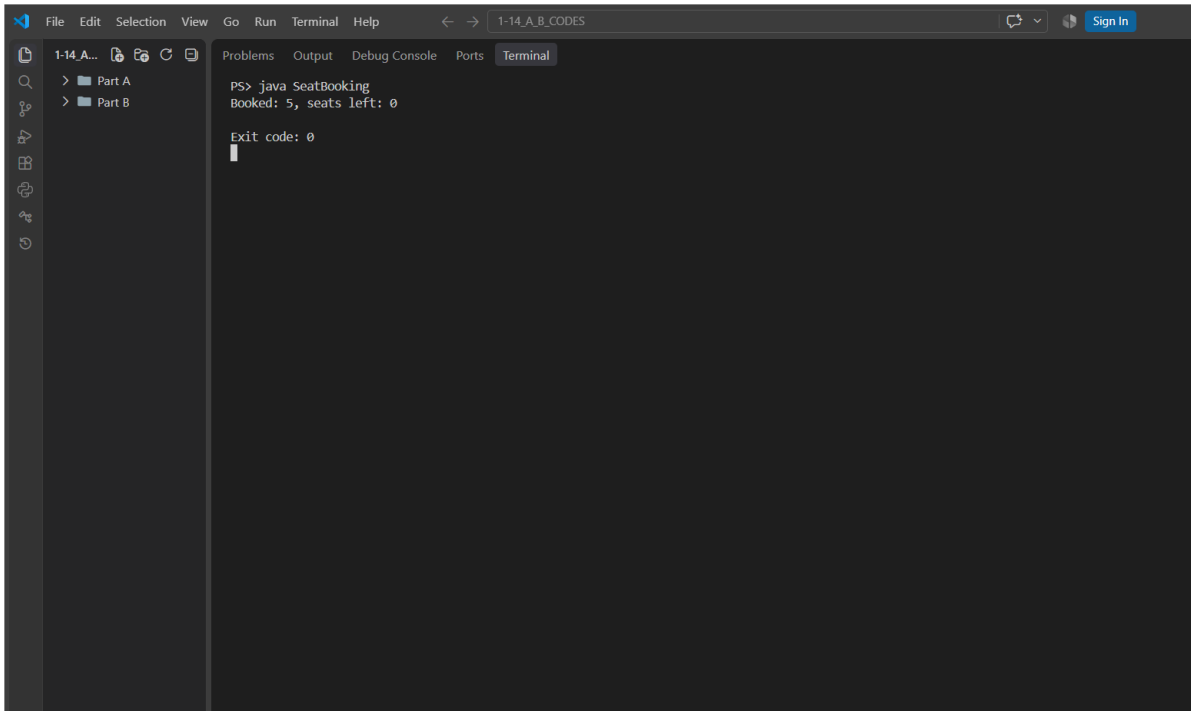
OUTPUT SCREENSHOT:



Output Screenshot 1



Output Screenshot 2



Output Screenshot 3

LEARNING OUTCOMES:

Explain race conditions and protect shared mutable state with synchronization and thread coordination.

PRACTICAL 10-A

Object Oriented Programming

AIM:

To implement and understand the concepts specified in Practical 10 of the CEUE203 Object Oriented Programming practical list, using Java. The work covers Part A: Thread Pool, Producer-Consumer, Deadlock Demo and Part B: TransactionProcessor and batch operations.

CODE SCREENSHOT:

DeadlockDemo.java

Part A > Practical 10-A > DeadlockDemo.java > DeadlockDemo > main(String[])

```java
public class DeadlockDemo {
    Run | Debug
    public static void main(String[] args) {
        Object a = new Object(), b = new Object();
        Thread t1 = new Thread(() -> {
            synchronized (a) {
                sleep();
                synchronized (b) {
                    System.out.println("t1 acquired both");
                }
            }
        });
        Thread t2 = new Thread(() -> {
            synchronized (b) {
                sleep();
                synchronized (a) {
                    System.out.println("t2 acquired both");
                }
            }
        });
        t1.start();
        t2.start();
        System.out.println("Demo may deadlock; stop manually if both threads block.");
    }

    static void sleep() {
        try {
            Thread.sleep(100);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

Code Screenshot 1

ProducerConsumer.java

Part A > Practical 10-A > ProducerConsumer.java > ...

```java
import java.util.concurrent.*;

public class ProducerConsumer {
    Run | Debug
    public static void main(String[] args) throws Exception {
        BlockingQueue<Integer> buffer = new ArrayBlockingQueue<>(3);
        Thread producer = new Thread(() -> {
            try {
                for (int i = 1; i <= 5; i++) {
                    buffer.put(i);
                    System.out.println("Produced " + i);
                }
                buffer.put(-1);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });
        Thread consumer = new Thread(() -> {
            try {
                while (true) {
                    int n = buffer.take();
                    if (n == -1)
                        break;
                    System.out.println("Consumed " + n);
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });
        producer.start();
        consumer.start();
        producer.join();
        consumer.join();
    }
}

```

Code Screenshot 2

ThreadPoolRunner.java

Part A > Practical 10-A > ThreadPoolRunner.java > ...

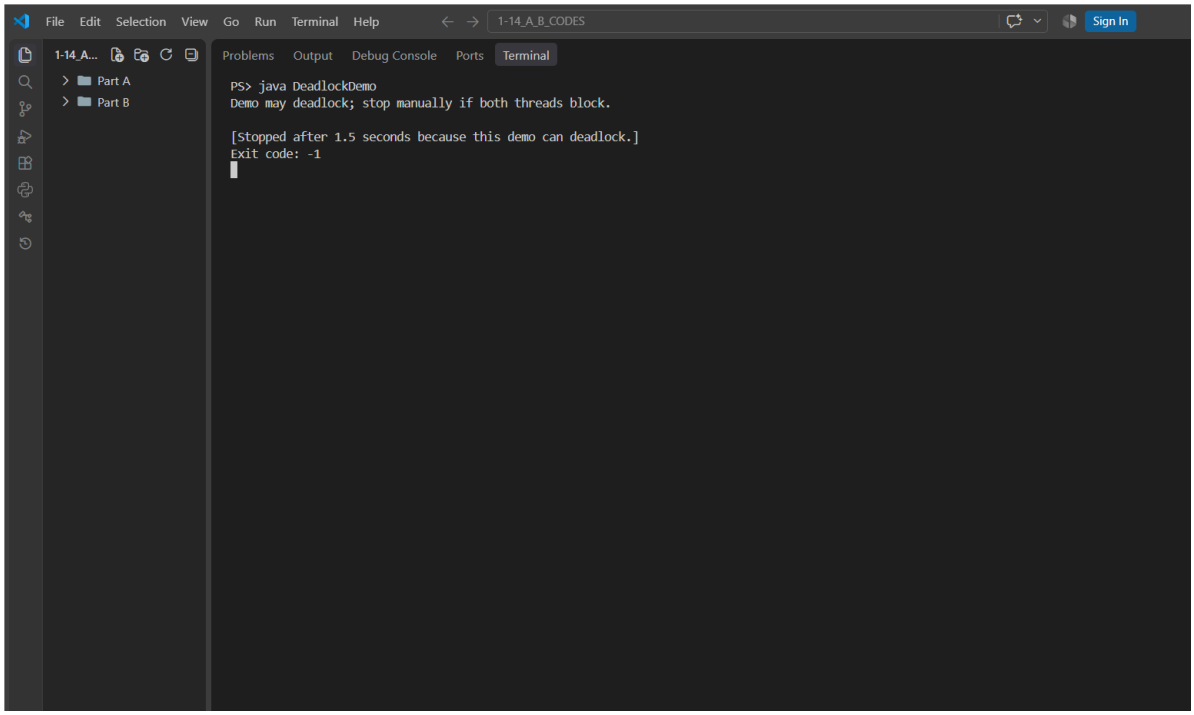
```java
import java.util.concurrent.*;

public class ThreadPoolRunner {
    Run | Debug
    public static void main(String[] args) throws InterruptedException {
        ExecutorService pool = Executors.newFixedThreadPool(3);
        for (int i = 1; i <= 10; i++) {
            final int n = i;
            pool.submit(() -> System.out.println("Task " + n + " on " + Thread.currentThread().getName()));
        }
        pool.shutdown();
        pool.awaitTermination(1, TimeUnit.MINUTES);
    }
}

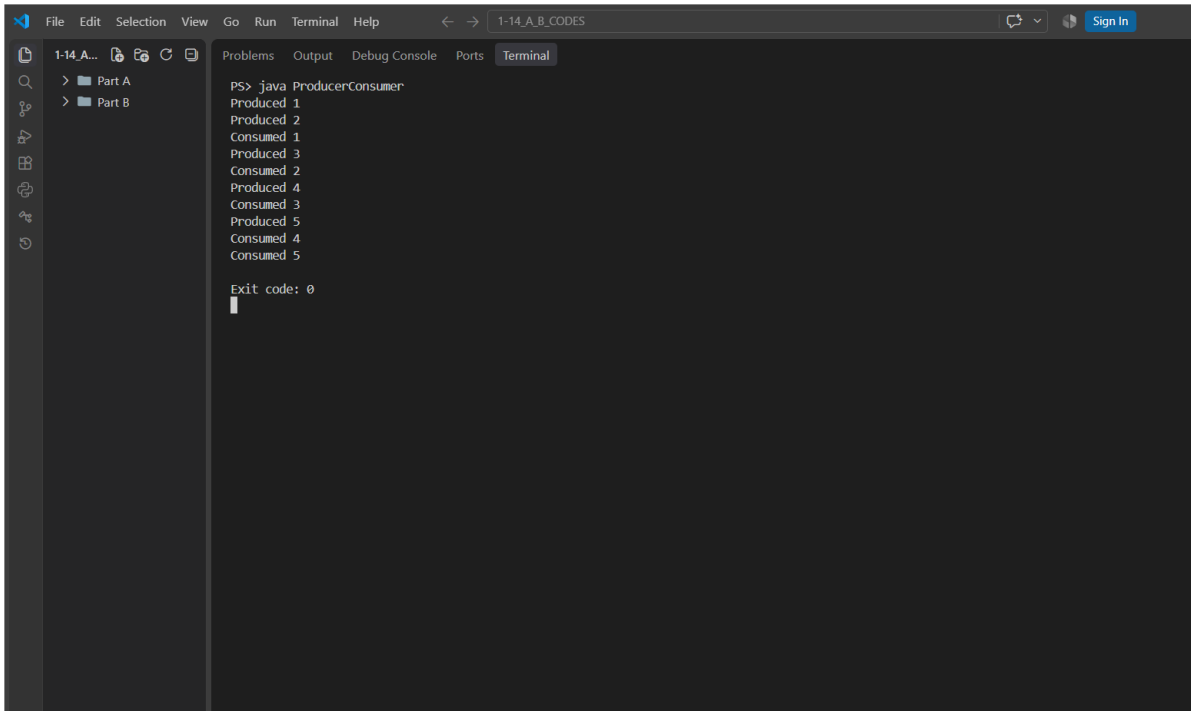
```

Code Screenshot 3

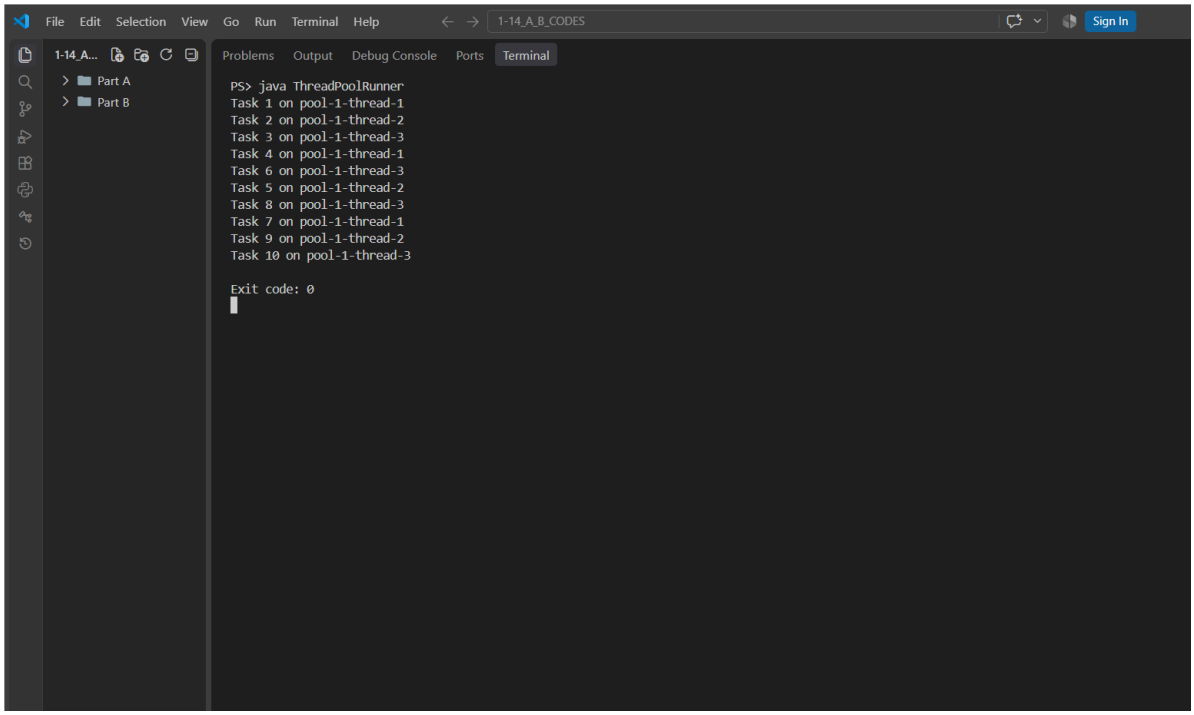
OUTPUT SCREENSHOT:



Output Screenshot 1



Output Screenshot 2



Output Screenshot 3

LEARNING OUTCOMES:

Use executor services and producer-consumer coordination to manage concurrent tasks.

PRACTICAL 11-A

Object Oriented Programming

AIM:

To implement and understand the concepts specified in Practical 11 of the CEUE203 Object Oriented Programming practical list, using Java. The work covers Part A: File Persistence and Report and Part B: Load/save account data and report.

CODE SCREENSHOT:

AccountFileStore.java 1

Part A > Practical 11-A > AccountFileStore.java > ...

```java
import java.io.*;
import java.nio.file.*;
import java.util.*;

public class AccountFileStore {
    Run | Debug
    public static void main(String[] args) throws Exception {
        Path p = Path.of("accounts.csv");
        List<String> rows = List.of("AC0001,Anshul,5000", "AC0002,Riya,2500");
        Files.write(p, rows);
        List<String> loaded = Files.readAllLines(p);
        System.out.println("Persistence verified: " + rows.equals(loaded));
        loaded.forEach(System.out::println);
    }
}

```

Code Screenshot 1

BankReport.java

Part A > Practical 11-A > BankReport.java > ...

```java
import java.util.*;

public class BankReport {
    record Tx(String type, long amount) {
    }

    Run | Debug
    public static void main(String[] args) {
        List<Tx> tx = List.of(new Tx(type: "DEPOSIT", amount: 5000), new Tx(type: "WITHDRAW", amount: 1200), new Tx(type: "DEPOSIT", amount: 300));
        long deposits = tx.stream().filter(t -> t.type.equals("DEPOSIT")).mapToLong(Tx::amount).sum();
        long withdrawals = tx.stream().filter(t -> t.type.equals("WITHDRAW")).mapToLong(Tx::amount).sum();
        System.out.println("Transactions: " + tx.size());
        System.out.println("Deposits: " + deposits);
        System.out.println("Withdrawals: " + withdrawals);
        System.out.println("Net: " + (deposits - withdrawals));
    }
}

```

Code Screenshot 2

PersistenceDemo.java

Part A > Practical 11-A > PersistenceDemo.java > ...

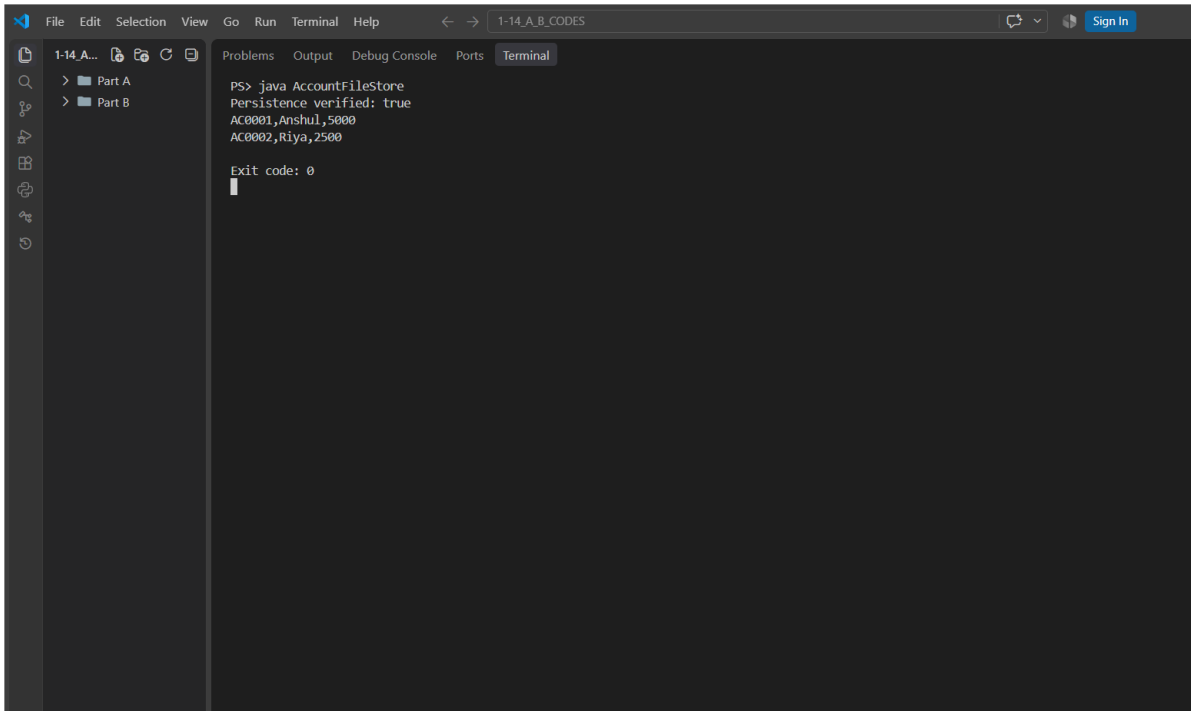
```java
import java.io.*;
import java.nio.file.*;
import java.util.*;

public class PersistenceDemo {
    Run | Debug
    public static void main(String[] args) throws IOException {
        Path path = Path.of("accounts.txt");
        List<String> accounts = Files.exists(path) ? Files.readAllLines(path) : new ArrayList<>();
        if (accounts.isEmpty())
            accounts.add("AC0001,Anshul,5000");
        Files.write(path, accounts);
        System.out.println("Loaded accounts:");
        accounts.forEach(System.out::println);
        System.out.println("Saved to " + path.toAbsolutePath());
    }
}

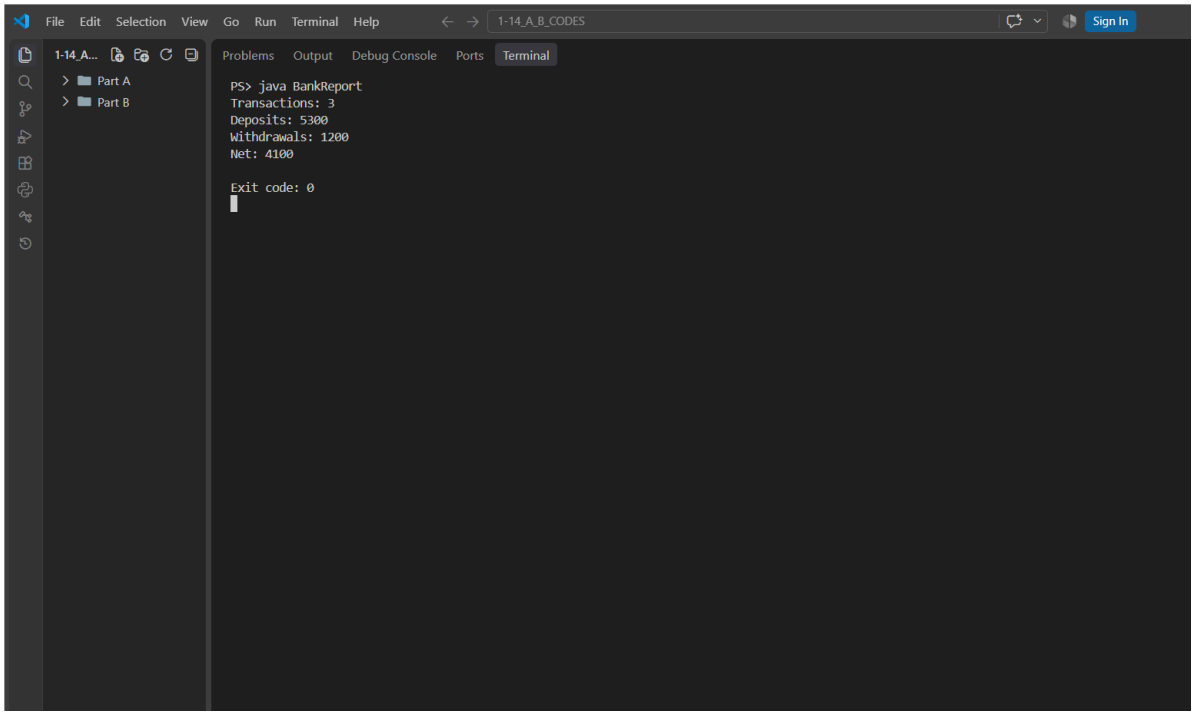
```

Code Screenshot 3

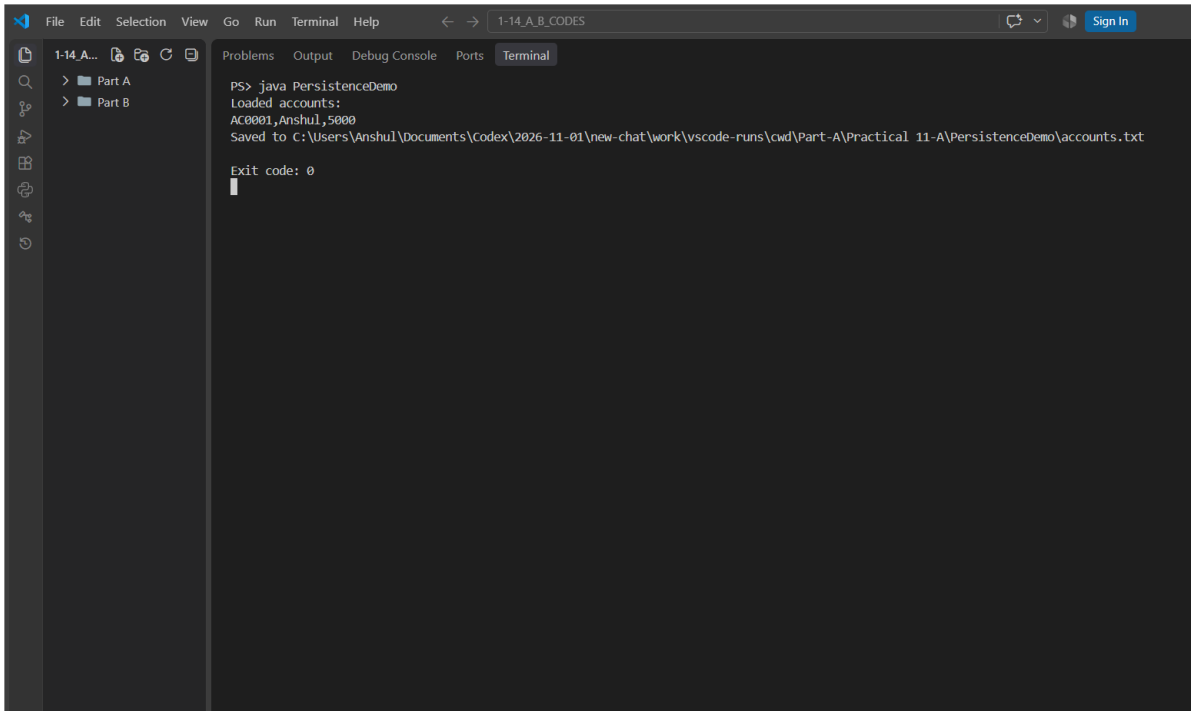
OUTPUT SCREENSHOT:



Output Screenshot 1



Output Screenshot 2



Output Screenshot 3

LEARNING OUTCOMES:

Persist application data to files, reload saved records, and generate a basic report.

PRACTICAL 12-A

Object Oriented Programming

AIM:

To implement and understand the concepts specified in Practical 12 of the CEUE203 Object Oriented Programming practical list, using Java. The work covers Part A: Concurrent Store, Transfer, Thread Lifecycle and Part B: Thread-safe account store.

CODE SCREENSHOT:

ConcurrentStore.java

Part A > Practical 12-A > ConcurrentStore.java > ...

```java
import java.util.concurrent.*;

public class ConcurrentStore {
    Run | Debug
    public static void main(String[] args) throws InterruptedException {
        ConcurrentHashMap<String, Long> balances = new ConcurrentHashMap<>();
        balances.put("AC0001", 0L);
        Thread[] ts = new Thread[10];
        for (int i = 0; i < ts.length; i++) {
            ts[i] = new Thread(() -> {
                for (int j = 0; j < 1000; j++)
                    balances.compute("AC0001", (k, v) -> v + 1);
            });
            ts[i].start();
        }
        for (Thread t : ts)
            t.join();
        System.out.println("Balance: " + balances.get("AC0001"));
    }
}

```

Code Screenshot 1

ConcurrentTransfer.java

Part A > Practical 12-A > ConcurrentTransfer.java > ...

```java
import java.util.concurrent.*;

public class ConcurrentTransfer {
    static class Bank {
        final ConcurrentHashMap<String, Long> b = new ConcurrentHashMap<>();

        synchronized boolean transfer(String from, String to, long amount) {
            if (amount <= 0 || b.get(from) < amount)
                return false;
            b.put(from, b.get(from) - amount);
            b.put(to, b.get(to) + amount);
            return true;
        }
    }

    Run | Debug
    public static void main(String[] args) {
        Bank bank = new Bank();
        bank.b.put("A", 1000L);
        bank.b.put("B", 1000L);
        System.out.println(bank.transfer(from: "A", to: "B", amount: 250));
        System.out.println(bank.b);
    }
}

```

Code Screenshot 2

ThreadLifecycle.java

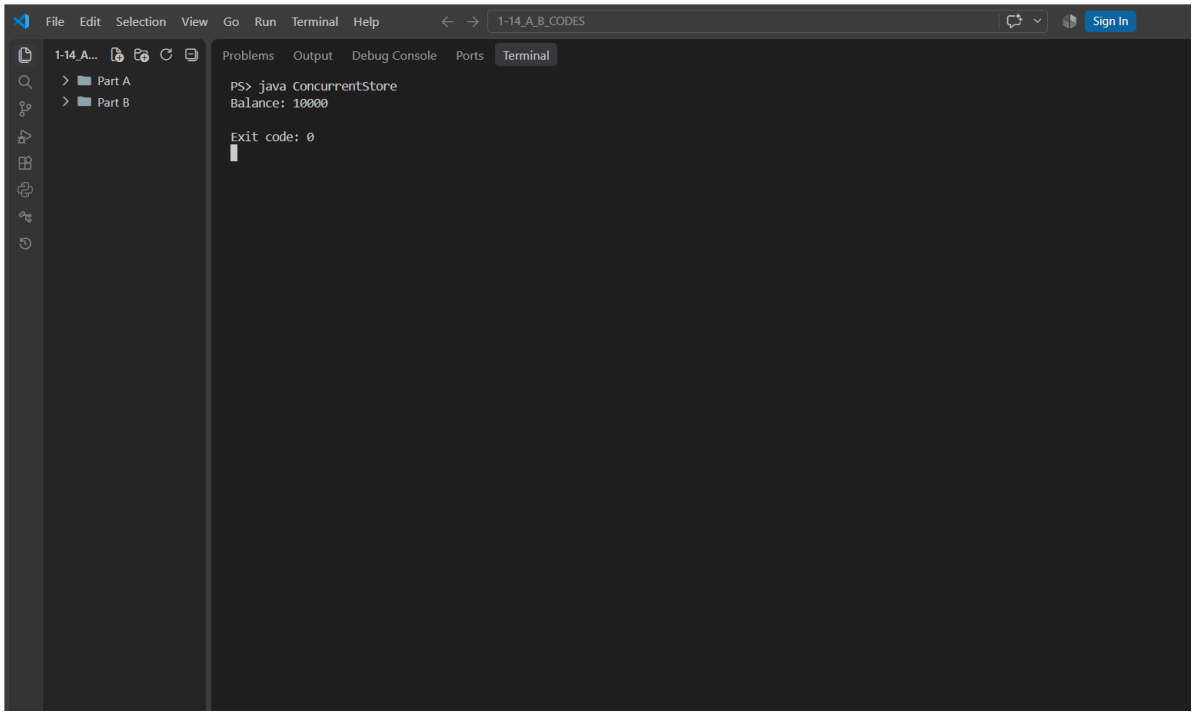
Part A > Practical 12-A > ThreadLifecycle.java > ...

```java
public class ThreadLifecycle {
    Run | Debug
    public static void main(String[] args) throws InterruptedException {
        Thread t = new Thread(() -> System.out.println("Running task"));
        System.out.println(t.getState());
        t.start();
        t.join();
        System.out.println(t.getState());
    }
}

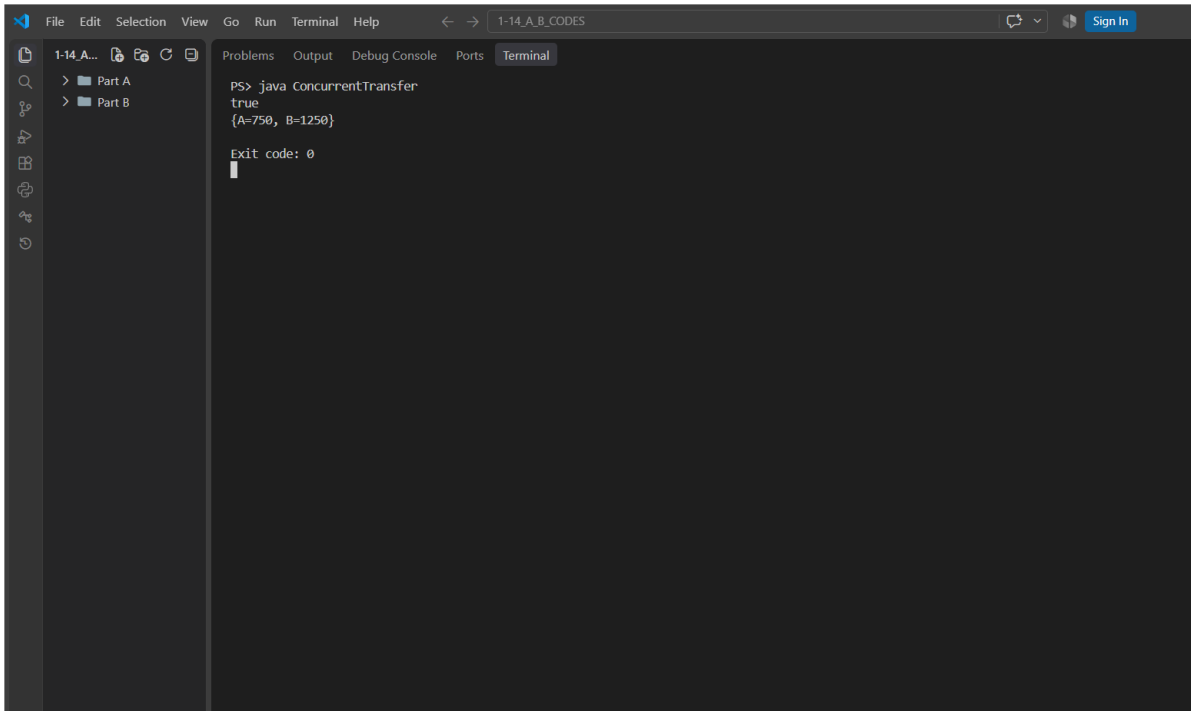
```

Code Screenshot 3

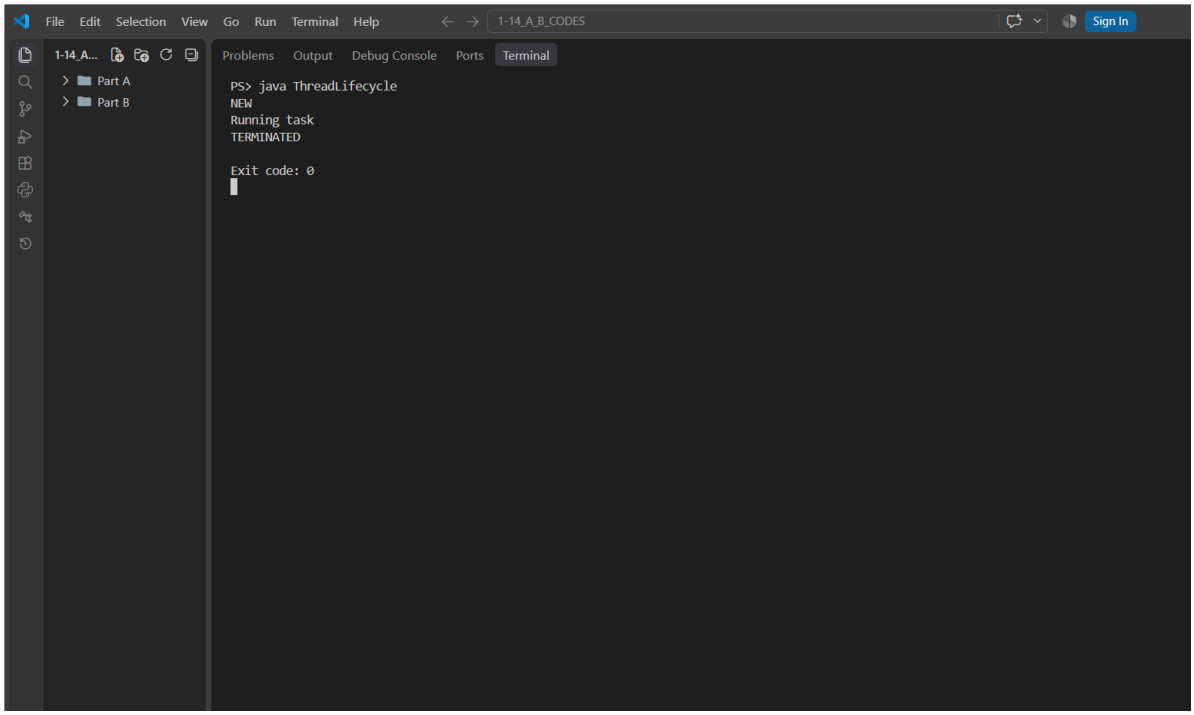
OUTPUT SCREENSHOT:



Output Screenshot 1



Output Screenshot 2



Output Screenshot 3

LEARNING OUTCOMES:

Use concurrent collections and synchronized operations to maintain consistency under concurrent access.

PRACTICAL 13-A

Object Oriented Programming

AIM:

To implement and understand the concepts specified in Practical 13 of the CEUE203 Object Oriented Programming practical list, using Java. The work covers Part A: Generic repository / bounded generics and Part B: Generic Repository<T> using @Id reflection.

CODE SCREENSHOT:

GenericRepository.java

Part A > Practical 13-A > GenericRepository.java > ...

```java
import java.lang.annotation.*;
import java.lang.reflect.*;
import java.util.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface Id {
}

class Account {
    @Id
    private final String accountNumber;
    private final String owner;
    private long balance;

    Account(String n, String o, long b) {
        accountNumber = n;
        owner = o;
        balance = b;
    }

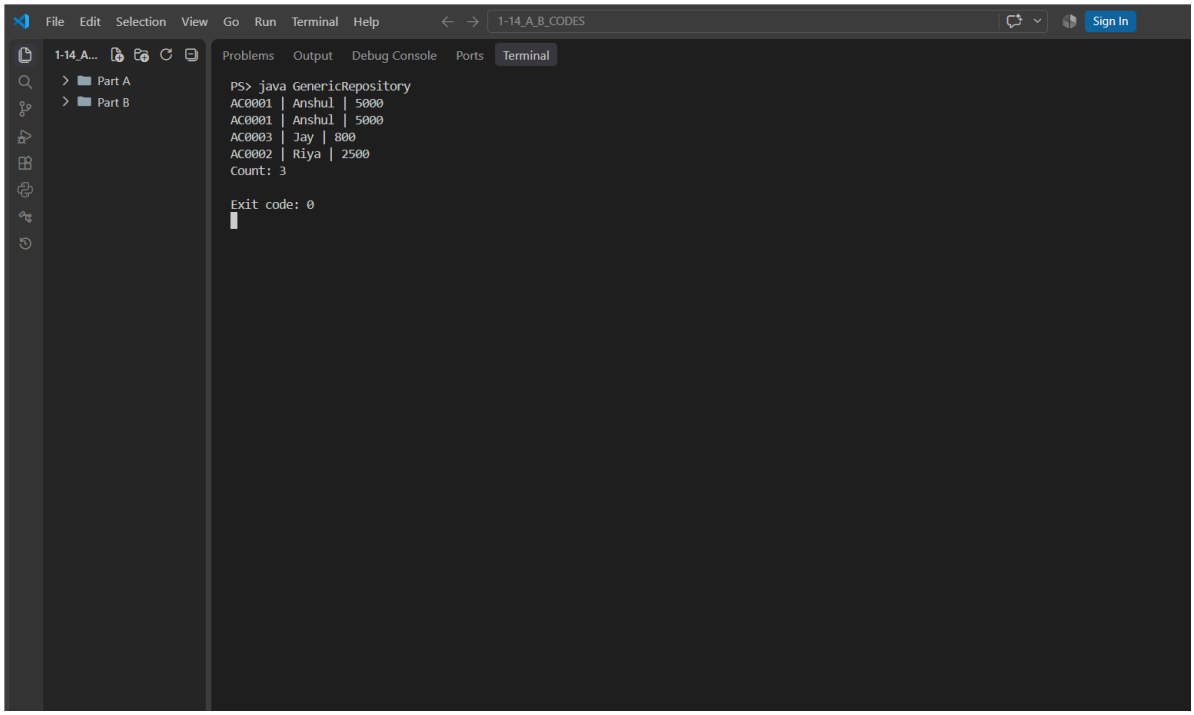
    public String toString() {
        return accountNumber + " | " + owner + " | " + balance;
    }
}

class Repository<T> {
    private final Map<String, T> data = new HashMap<>();

    public T save(T entity) {
        try {
            for (Field f : entity.getClass().getDeclaredFields())
                if (f.isAnnotationPresent(Id.class)) {
                    f.setAccessible(true);
                    data.put(String.valueOf(f.get(entity)), entity);
                    return entity;
                }
            throw new IllegalArgumentException("No @Id field");
        } catch (IllegalAccessException e) {
            throw new RuntimeException(e);
        }
    }

    @SafeVarargs
    public final void saveAll(T... items) {
        for (T item : items)
            save(item);
    }
```

OUTPUT SCREENSHOT:



LEARNING OUTCOMES:

Build reusable type-safe generic components and access entity identifiers using reflection.

PRACTICAL 14-A

Object Oriented Programming

AIM:

To assemble and test a small layered Java application by creating a model, a service that manages data using an ArrayList, and a menu-driven main class; then write JUnit tests to verify successful additions and invalid-action handling.

CODE SCREENSHOT:

LayeredApp.java 1

14-A > LayeredApp.java > { } app

```java
package app;

import service.TaskService;
import java.util.Scanner;

public class LayeredApp {
    public static void main(String[] args) {
        TaskService service = new TaskService();
        try (Scanner sc = new Scanner(System.in)) {
            while (true) {
                System.out.println("\n1. Add task  2. List tasks  0. Exit");
                System.out.print("Choice: ");
                String choice = sc.nextLine().trim();
                try {
                    switch (choice) {
                        case "1" -> {
                            System.out.print("Task title: ");
                            service.performAction("ADD", sc.nextLine());
                            System.out.println("Task added.");
                        }
                        case "2" -> service.performAction("LIST", "");
                        case "0" -> {
                            System.out.println("Goodbye.");
                            return;
                        }
                        default -> service.performAction(choice, "");
                    }
                } catch (IllegalArgumentException ex) {
                    System.out.println(ex.getMessage());
                }
            }
        }
    }
}

```

Code Screenshot 1

Task.java 1 ×

14-A > Task.java > Task

```java
package model;

public record Task(int id, String title) {
    public Task {
        if (title == null || title.isBlank()) {
            throw new IllegalArgumentException("Task title cannot be blank");
        }
    }
}

```

Code Screenshot 2

TaskService.java 1 ×

14-A > TaskService.java > ...

```java
package service;

import model.Task;
import java.util.ArrayList;
import java.util.List;

public class TaskService {
    private final List<Task> tasks = new ArrayList<>();
    private int nextId = 1;

    public Task addTask(String title) {
        Task task = new Task(nextId++, title);
        tasks.add(task);
        return task;
    }

    public int count() {
        return tasks.size();
    }

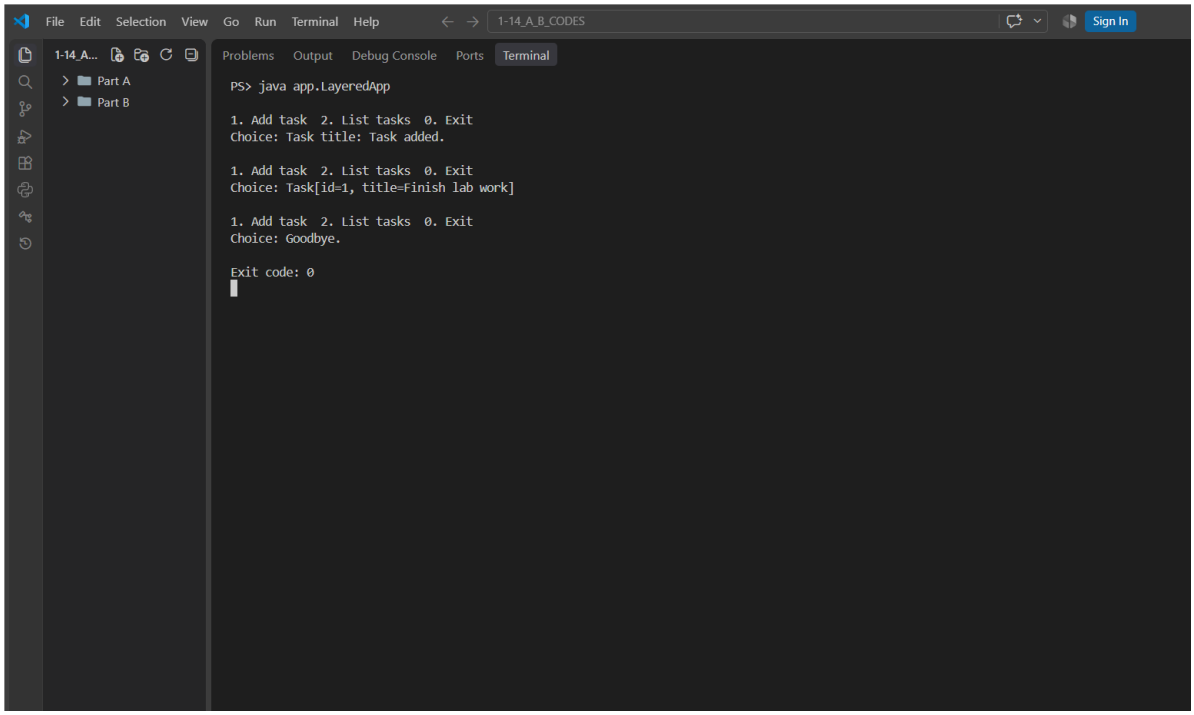
    public List<Task> getAll() {
        return List.copyOf(tasks);
    }

    public void performAction(String action, String value) {
        if ("ADD".equalsIgnoreCase(action)) {
            addTask(value);
        } else if ("LIST".equalsIgnoreCase(action)) {
            getAll().forEach(System.out::println);
        } else {
            throw new IllegalArgumentException("Invalid action: " + action);
        }
    }
}

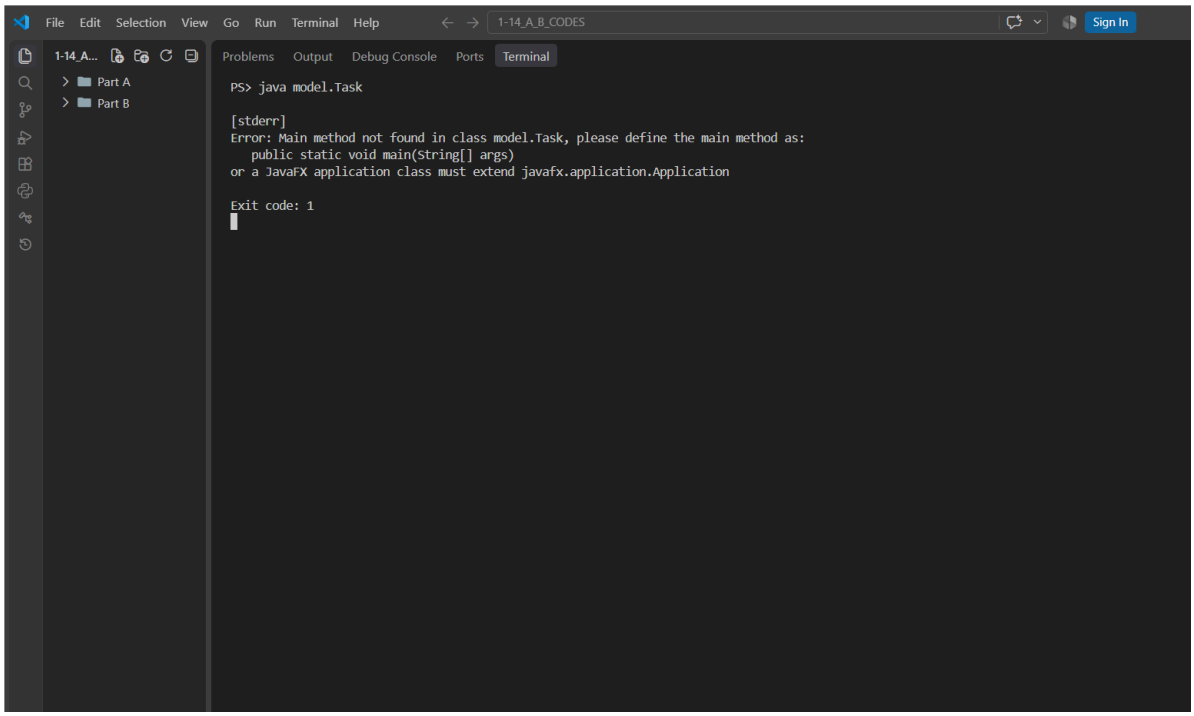
```

Code Screenshot 3

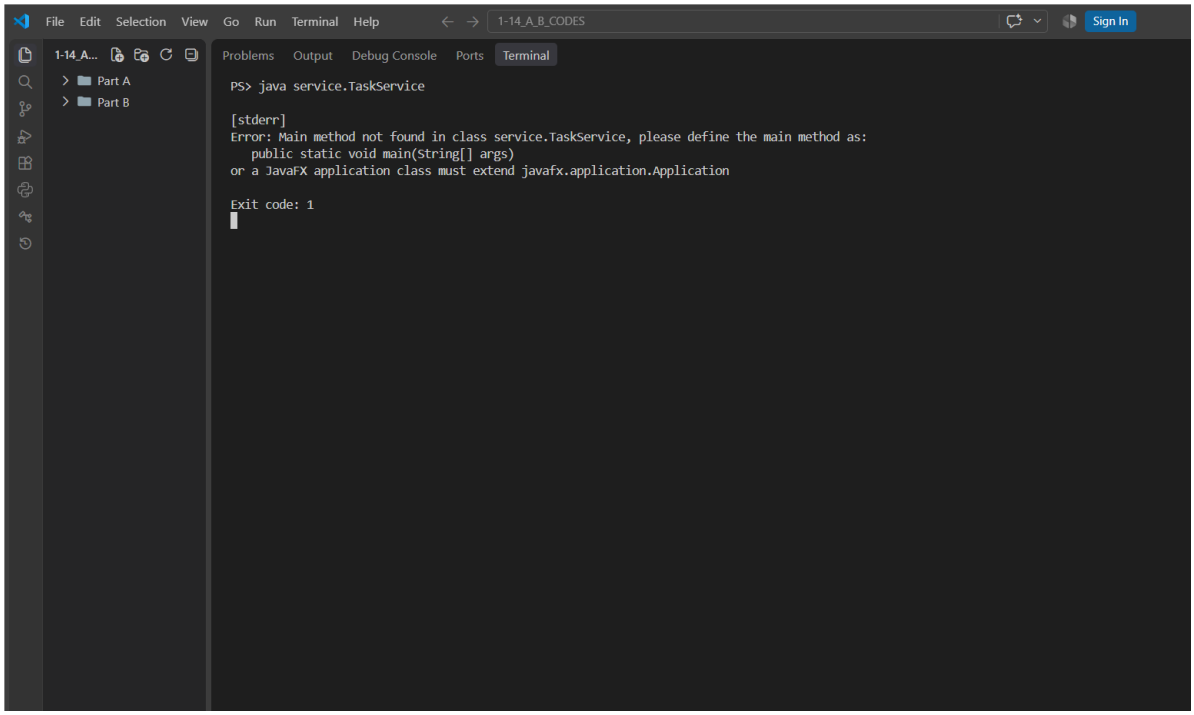
OUTPUT SCREENSHOT:



Output Screenshot 1



Output Screenshot 2



Output Screenshot 3

LEARNING OUTCOMES:

The student can assemble a small layered Java application, write basic JUnit tests for expected behaviour and exceptions, and understand how packaging and persistence tests support reliable application delivery.

Part-B

1 to 14 Practical

Start from here

PRACTICAL 1-B

Object Oriented Programming

AIM:

To develop the MiniBank console application menu shell using a BankInfo record, MenuOption enum, Scanner input and a switch expression.

CODE SCREENSHOT:

MiniBank.java 1

Part B > Practical 1-B > MiniBank.java > ...

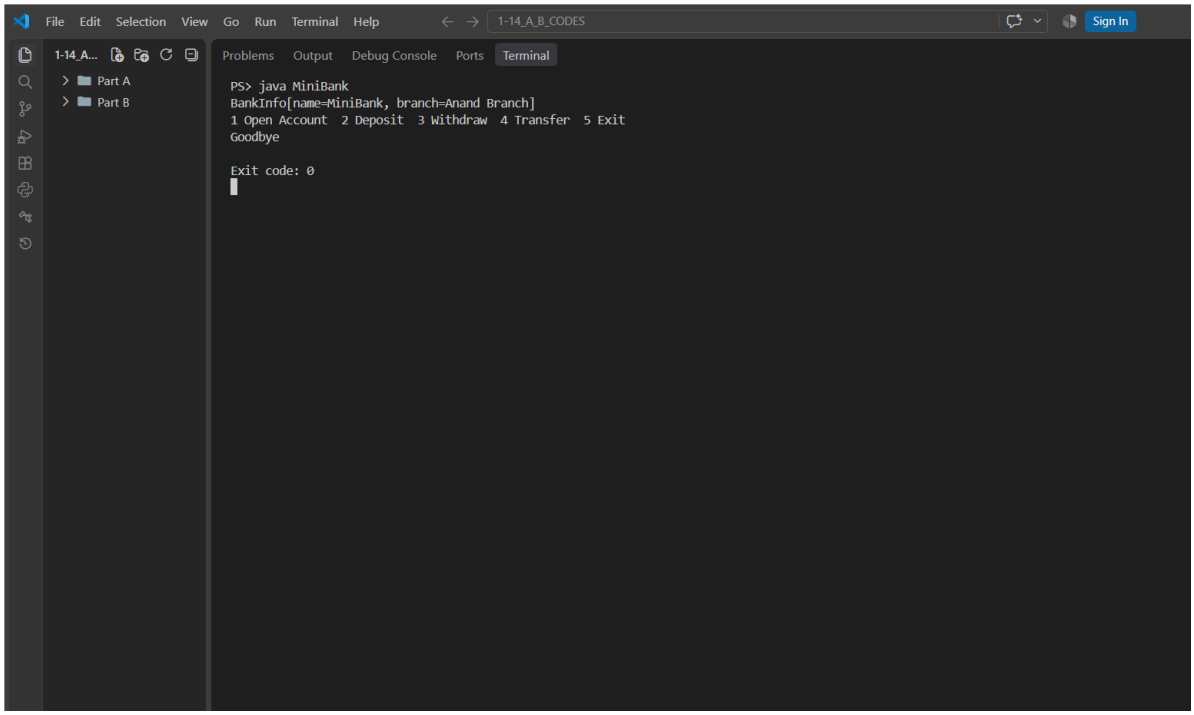
```java
import java.util.*;

public class MiniBank {
    record BankInfo(String name, String branch) {
    }

    enum MenuOption {
        OPEN_ACCOUNT, DEPOSIT, WITHDRAW, TRANSFER, EXIT
    }

    Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println(new BankInfo(name: "MiniBank", branch: "Anand Branch"));
        boolean running = true;
        while (running) {
            System.out.println("1 Open Account  2 Deposit  3 Withdraw  4 Transfer  5 Exit");
            int n = sc.nextInt();
            MenuOption option = switch (n) {
                case 1 -> MenuOption.OPEN_ACCOUNT;
                case 2 -> MenuOption.DEPOSIT;
                case 3 -> MenuOption.WITHDRAW;
                case 4 -> MenuOption.TRANSFER;
                case 5 -> MenuOption.EXIT;
                default -> null;
            };
            if (option == null) {
                System.out.println("Invalid option");
                continue;
            }
            switch (option) {
                case OPEN_ACCOUNT -> System.out.println("Open account — to be implemented in a later lab");
                case DEPOSIT -> System.out.println("Deposit — to be implemented in a later lab");
                case WITHDRAW -> System.out.println("Withdraw — to be implemented in a later lab");
                case TRANSFER -> System.out.println("Transfer — to be implemented in a later lab");
                case EXIT -> {
                    System.out.println("Goodbye");
                    running = false;
                }
            }
        }
    }
}
```

OUTPUT SCREENSHOT:



LEARNING OUTCOMES:

The student can set up the Java toolchain and write, compile, run and version-control a basic interactive program.

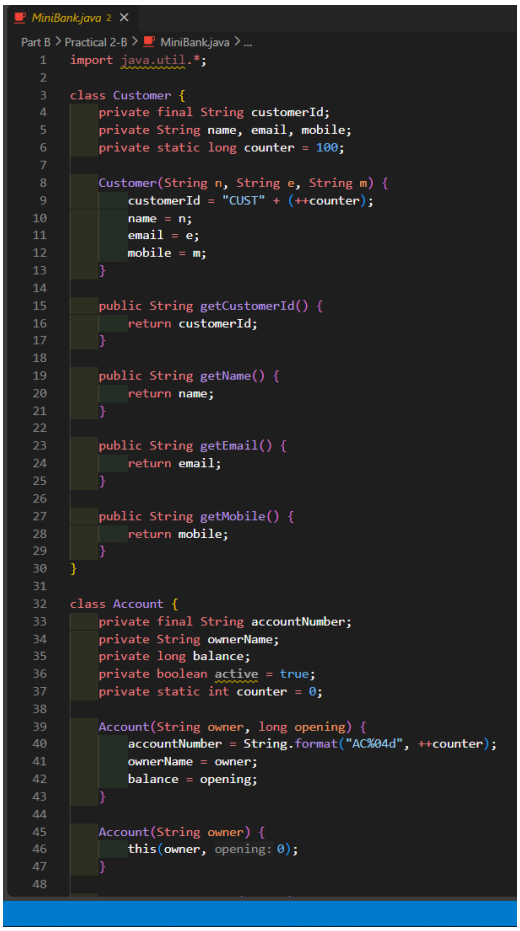
PRACTICAL 2-B

Object Oriented Programming

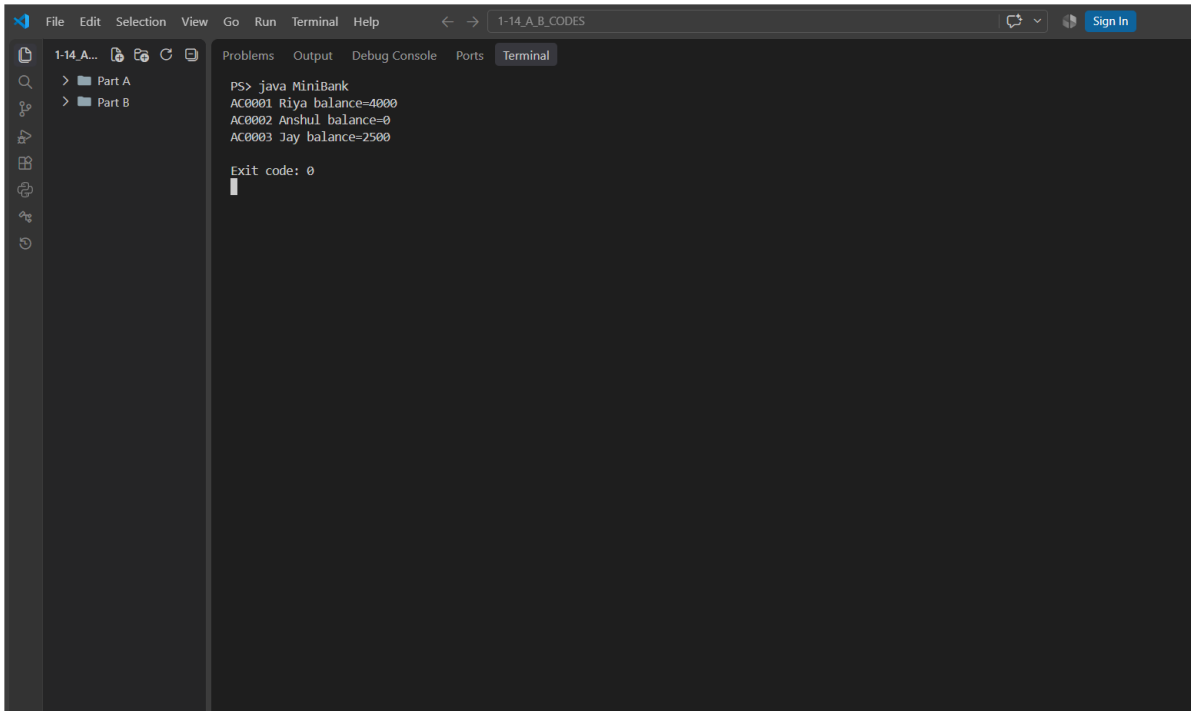
AIM:

To create Customer and Account classes with encapsulated fields, generated IDs, constructors, and controlled deposit and withdrawal operations.

CODE SCREENSHOT:



OUTPUT SCREENSHOT:



LEARNING OUTCOMES:

The student can design well-encapsulated classes and create and manipulate objects safely.

PRACTICAL 3-B

Object Oriented Programming

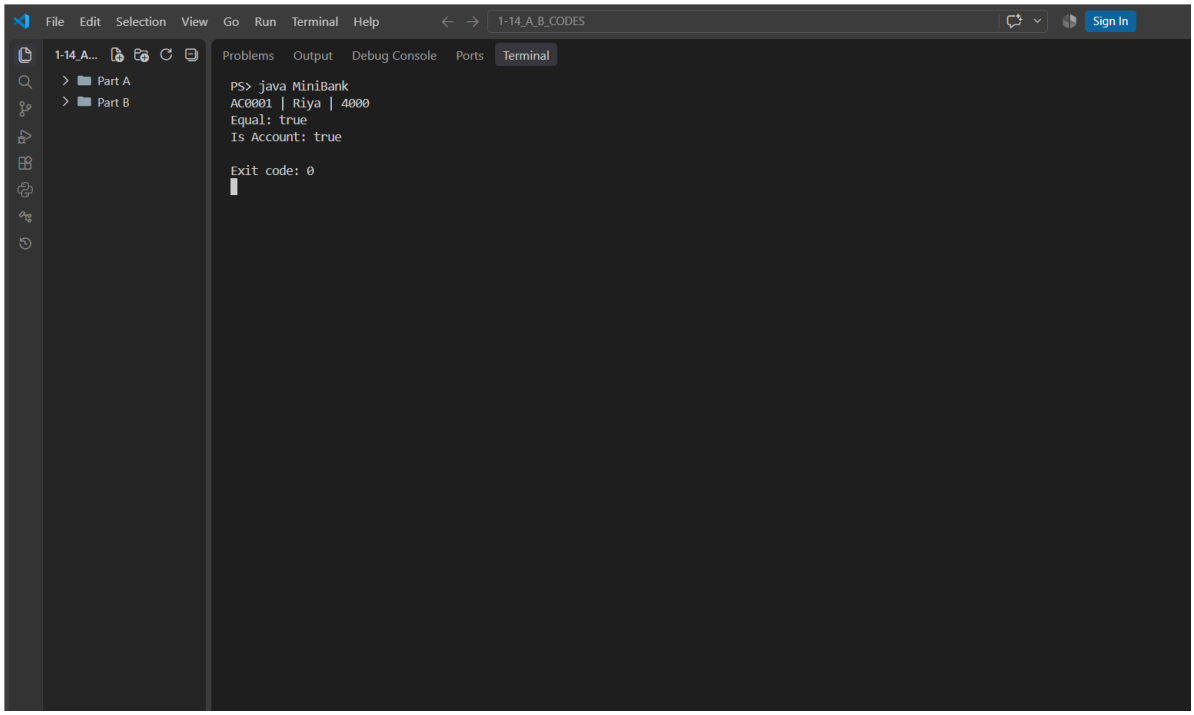
AIM:

To override toString(), equals() and hashCode() for Account, and add a nested Address class and cloning support to Customer.

CODE SCREENSHOT:



OUTPUT SCREENSHOT:



LEARNING OUTCOMES:

The student can make objects behave correctly for printing and comparison and can use nested classes.

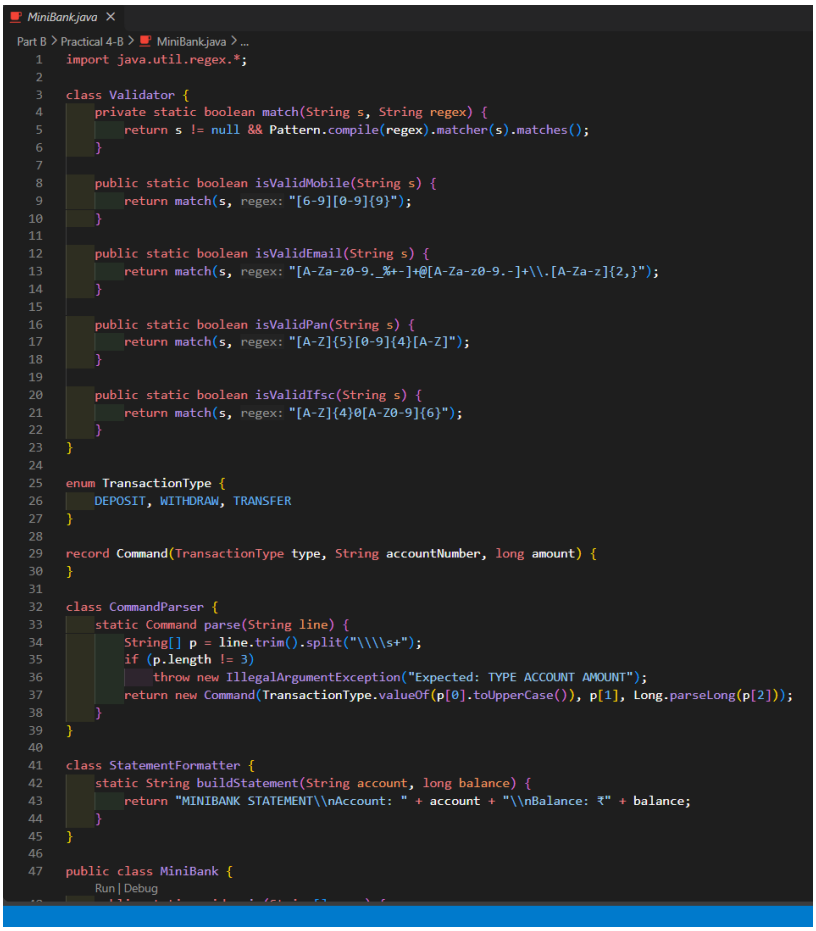
PRACTICAL 4-B

Object Oriented Programming

AIM:

To implement MiniBank input validation and command parsing using regular expressions, Pattern, Matcher, records and StringBuilder.

CODE SCREENSHOT:



```java
import java.util.regex.*;

class Validator {
    private static boolean match(String s, String regex) {
        return s != null && Pattern.compile(regex).matcher(s).matches();
    }

    public static boolean isValidMobile(String s) {
        return match(s, regex: "[6-9][0-9]{9}");
    }

    public static boolean isValidEmail(String s) {
        return match(s, regex: "[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}");
    }

    public static boolean isValidPan(String s) {
        return match(s, regex: "[A-Z]{5}[0-9]{4}[A-Z]");
    }

    public static boolean isValidIfsc(String s) {
        return match(s, regex: "[A-Z]{4}0[A-Z0-9]{6}");
    }
}

enum TransactionType {
    DEPOSIT, WITHDRAW, TRANSFER
}

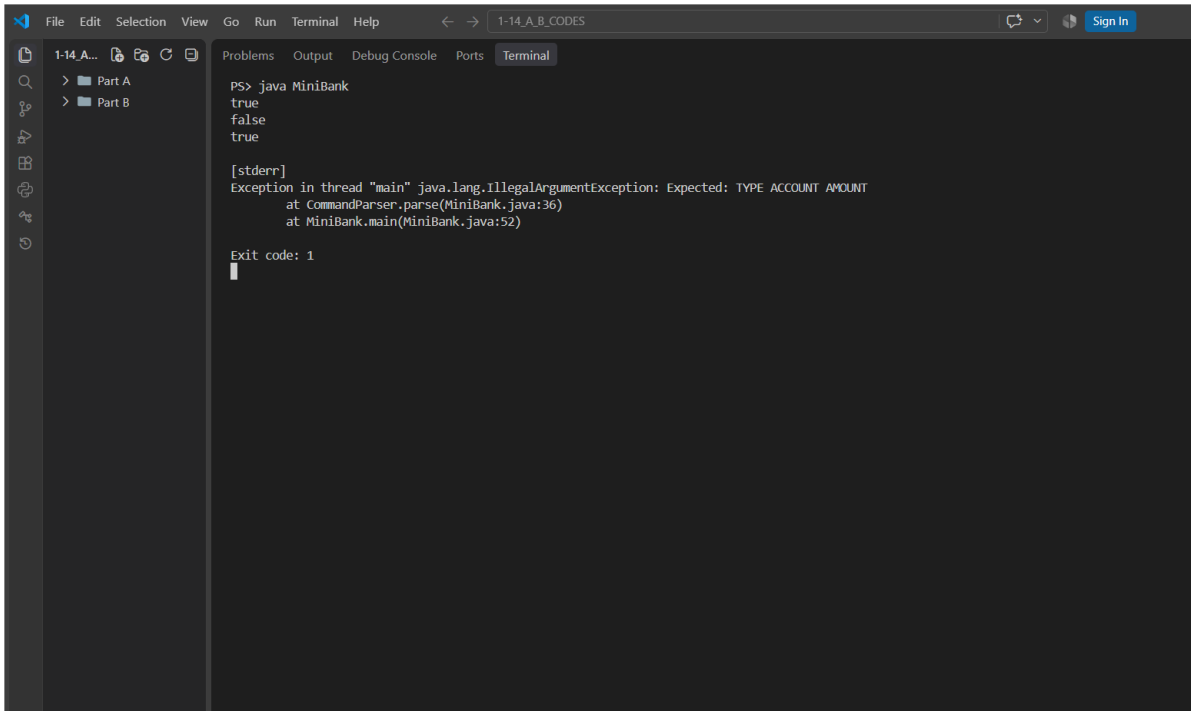
record Command(TransactionType type, String accountNumber, long amount) {
}

class CommandParser {
    static Command parse(String line) {
        String[] p = line.trim().split("\\s+");
        if (p.length != 3)
            throw new IllegalArgumentException("Expected: TYPE ACCOUNT AMOUNT");
        return new Command(TransactionType.valueOf(p[0].toUpperCase()), p[1], Long.parseLong(p[2]));
    }
}

class StatementFormatter {
    static String buildStatement(String account, long balance) {
        return "MINIBANK STATEMENT\\nAccount: " + account + "\\nBalance: ₹" + balance;
    }
}

public class MiniBank {
    Run | Debug
```

OUTPUT SCREENSHOT:



LEARNING OUTCOMES:

The student can validate and parse text input reliably using strings and regular expressions.

PRACTICAL 5-B

Object Oriented Programming

AIM:

To model SavingsAccount, CurrentAccount and FixedDepositAccount through inheritance, abstract methods and run-time polymorphism.

CODE SCREENSHOT:

MiniBank.java

Part B > Practical 5-B > MiniBank.java > ...

```java
abstract class Account {
    protected String accountNumber, ownerName;
    protected long balance;

    Account(String n, String o, long b) {
        accountNumber = n;
        ownerName = o;
        balance = b;
    }

    abstract double interestRate();

    abstract boolean canWithdraw(long amount);

    public long getBalance() {
        return balance;
    }
}

class SavingsAccount extends Account {
    long minBalance;

    SavingsAccount(String n, String o, long b, long m) {
        super(n, o, b);
        minBalance = m;
    }

    double interestRate() {
        return 4.0;
    }

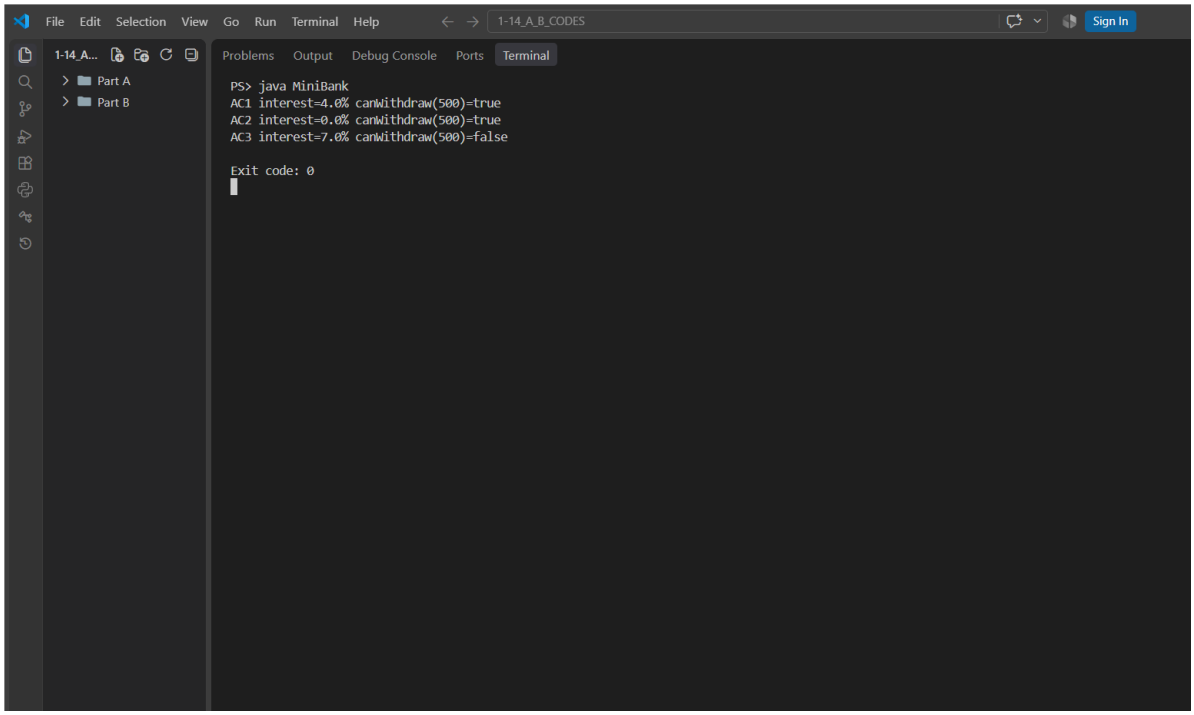
    boolean canWithdraw(long a) {
        return a > 0 && balance - a >= minBalance;
    }
}

class CurrentAccount extends Account {
    long overdraftLimit;

    CurrentAccount(String n, String o, long b, long l) {
        super(n, o, b);
        overdraftLimit = l;
    }

    double interestRate() {
        return 0.0;
    }
```

OUTPUT SCREENSHOT:



LEARNING OUTCOMES:

The student can design a class hierarchy and use polymorphism to write extensible code.

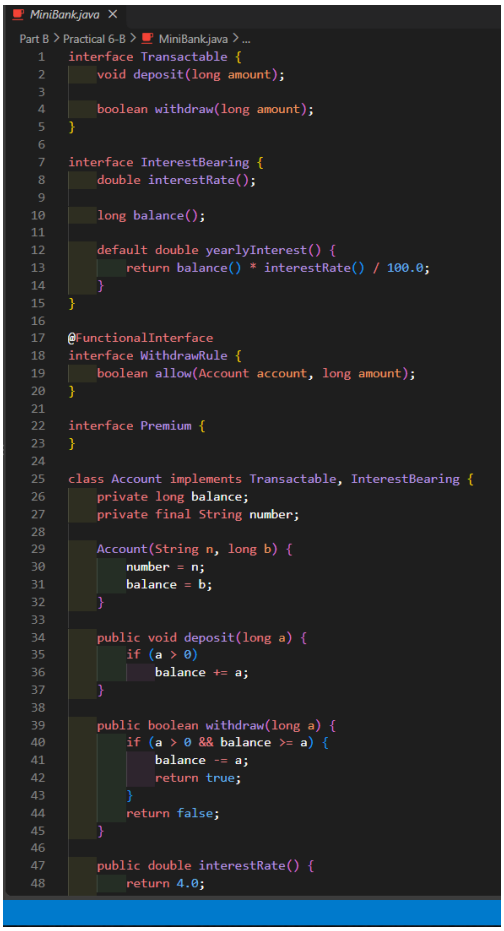
PRACTICAL 6-B

Object Oriented Programming

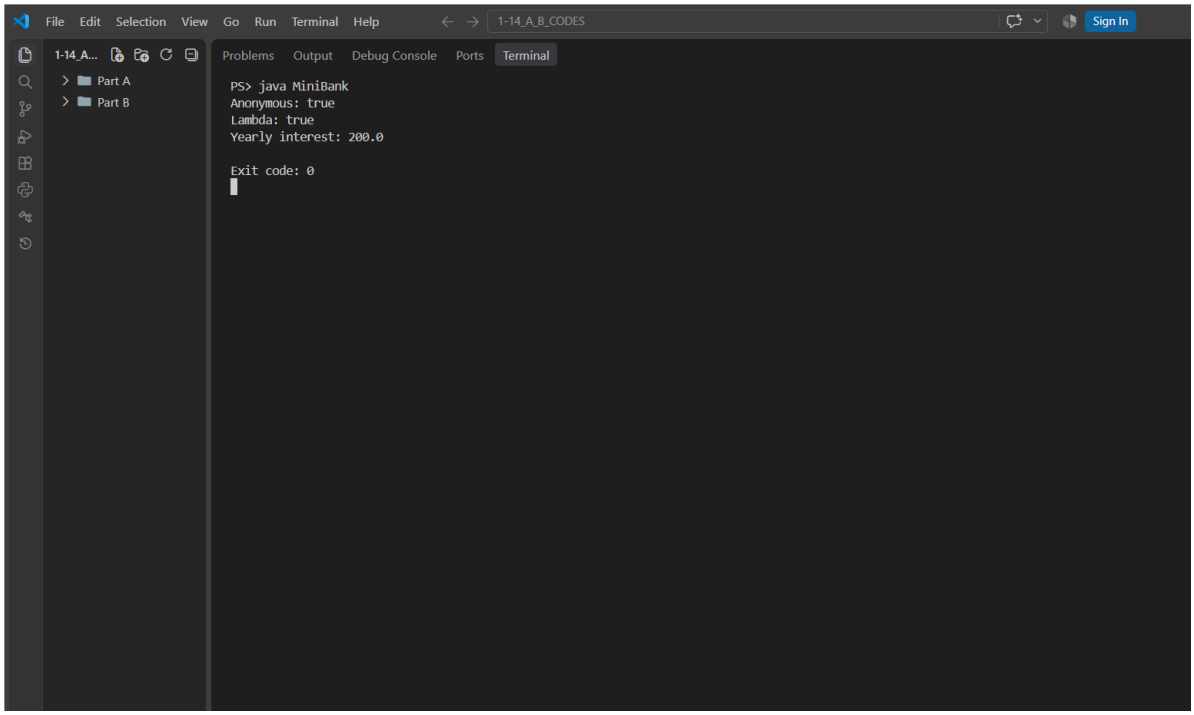
AIM:

To organise MiniBank using interfaces, functional interfaces, lambdas, packages and a runnable JAR file.

CODE SCREENSHOT:



OUTPUT SCREENSHOT:



LEARNING OUTCOMES:

The student can design interface-based code, organise it into packages, and produce a runnable build artifact.

PRACTICAL 7-B

Object Oriented Programming

AIM:

To define custom annotations and use reflection to validate object fields at run time.

CODE SCREENSHOT:



```java
import java.lang.annotation.*;
import java.lang.reflect.*;
import java.util.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface Id {
}

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface Positive {
    String message() default "must be > 0";
}

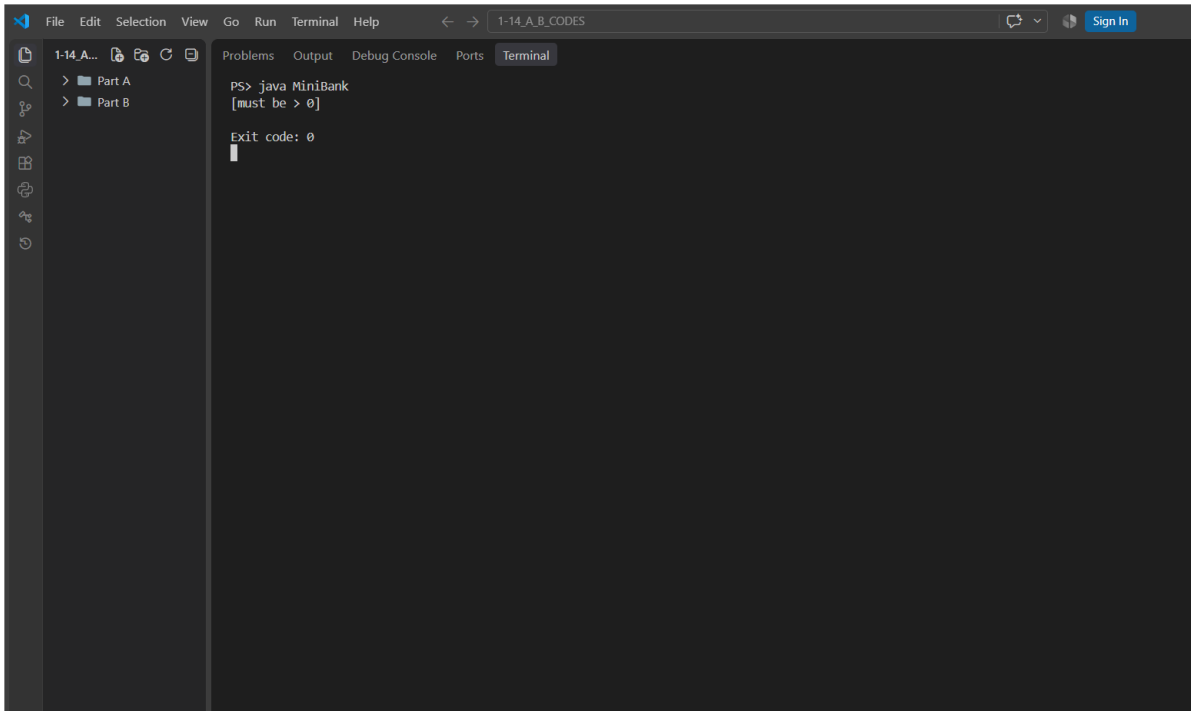
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface MaxLength {
    int value();
}

class Account {
    @Id
    private String accountNumber;
    @Positive
    private long balance;
    @MaxLength(20)
    private String owner;

    Account(String n, long b, String o) {
        accountNumber = n;
        balance = b;
        owner = o;
    }
}

class AnnotationValidator {
    static String[] validate(Object obj) {
        List<String> errors = new ArrayList<>();
        try {
            for (Field f : obj.getClass().getDeclaredFields()) {
                f.setAccessible(true);
                Object v = f.get(obj);
                Positive p = f.getAnnotation(Positive.class);
                if (p != null && v instanceof Number n && n.longValue() <= 0)
                    errors.add(p.message());
                MaxLength m = f.getAnnotation(MaxLength.class);
                if (m != null && v instanceof String s && s.length() > m.value())
```

OUTPUT SCREENSHOT:



LEARNING OUTCOMES:

The student can define custom annotations and process them at run time with reflection.

PRACTICAL 8-B

Object Oriented Programming

AIM:

To implement a custom checked-exception hierarchy and handle invalid amounts, insufficient funds and account transfers safely.

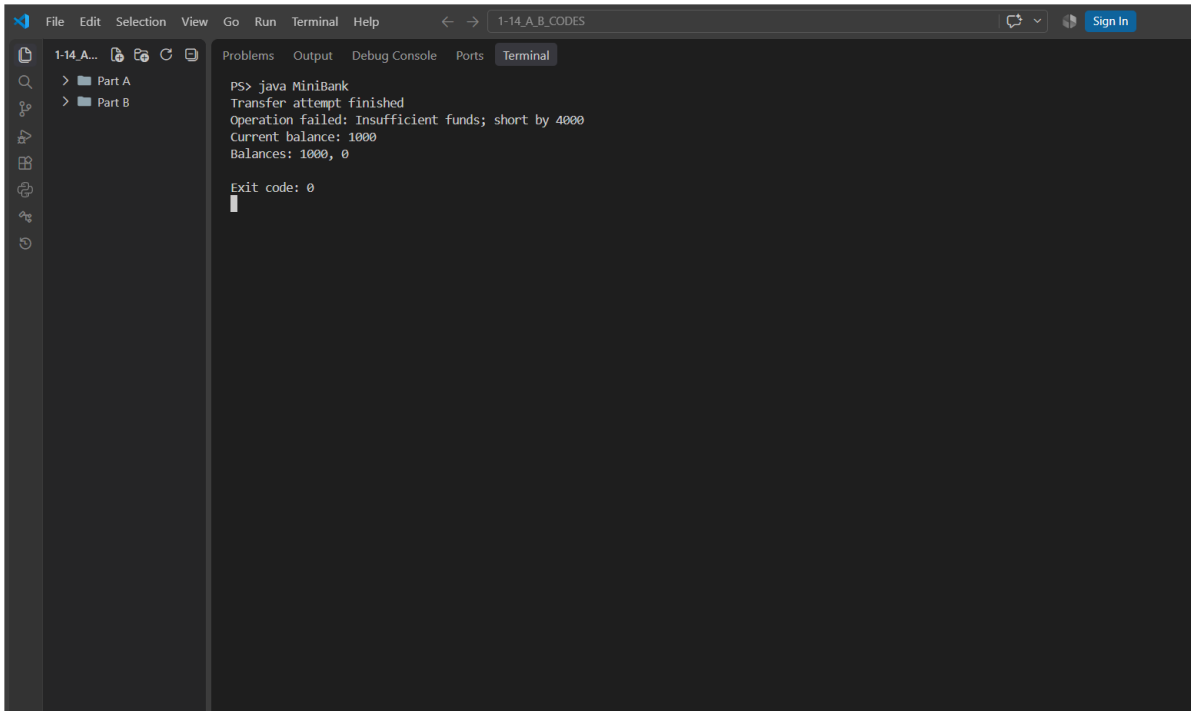
CODE SCREENSHOT:

MiniBank.java

Part B > Practical 8-B > MiniBank.java > ...

```
1  class BankException extends Exception {
2      BankException(String m) {
3          super(m);
4      }
5  }
6
7  class InsufficientFundsException extends BankException {
8      final long shortfall;
9
10     InsufficientFundsException(long s) {
11         super("Insufficient funds; short by " + s);
12         shortfall = s;
13     }
14 }
15
16 class InvalidAmountException extends BankException {
17     InvalidAmountException() {
18         super(m: "Amount must be positive");
19     }
20 }
21
22 class AccountNotFoundException extends BankException {
23     AccountNotFoundException() {
24         super(m: "Account not found");
25     }
26 }
27
28 class Account {
29     private long balance;
30
31     Account(long b) {
32         balance = b;
33     }
34
35     void deposit(long a) throws InvalidAmountException {
36         if (a <= 0)
37             throw new InvalidAmountException();
38         balance += a;
39     }
40
41     void withdraw(long a) throws InsufficientFundsException, InvalidAmountException {
42         if (a <= 0)
43             throw new InvalidAmountException();
44         if (a > balance)
45             throw new InsufficientFundsException(a - balance);
46         balance -= a;
47     }
48
```

OUTPUT SCREENSHOT:



LEARNING OUTCOMES:

The student can build fault-tolerant code using custom exceptions and structured error handling.

PRACTICAL 9-B

Object Oriented Programming

AIM:

To demonstrate race conditions and make shared account operations thread-safe using threads and synchronization.

CODE SCREENSHOT:



```java
class Account {
    private long balance;

    synchronized void deposit(long amount) {
        balance += amount;
    }

    synchronized boolean withdraw(long amount) {
        if (amount <= 0 || amount > balance)
            return false;
        balance -= amount;
        return true;
    }

    synchronized long getBalance() {
        return balance;
    }
}

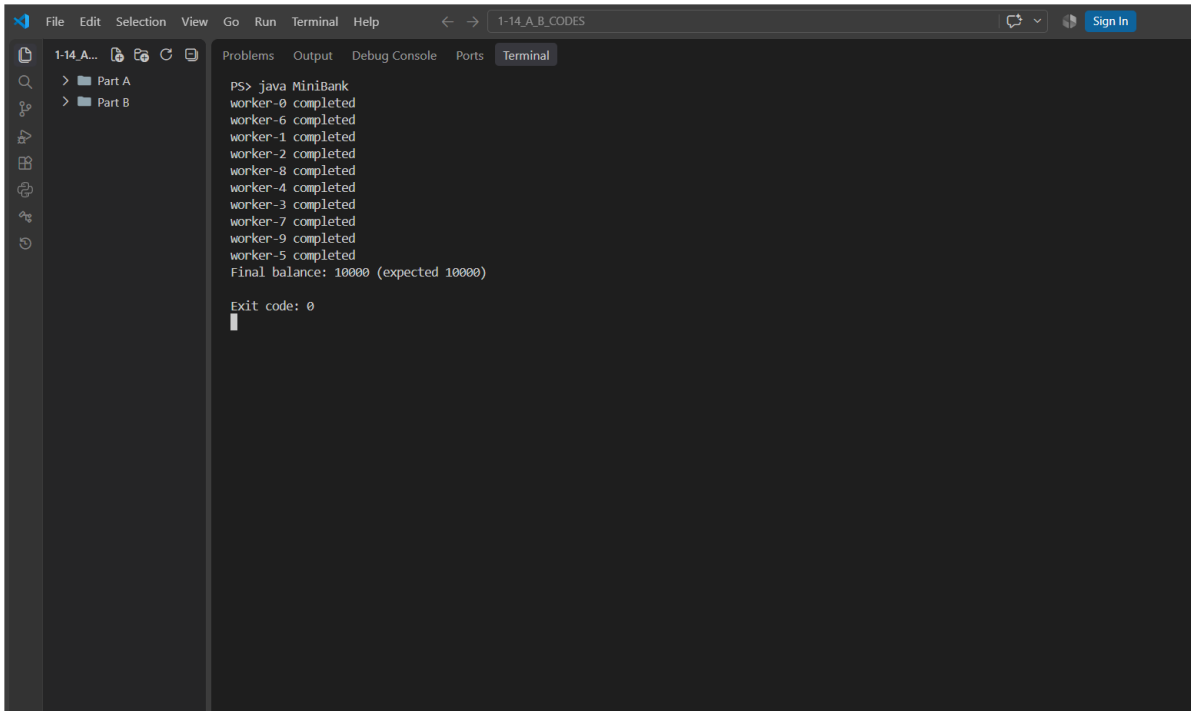
class AccountWorker implements Runnable {
    private final Account account;
    private final int times;
    private final long amount;

    AccountWorker(Account a, int t, long m) {
        account = a;
        times = t;
        amount = m;
    }

    public void run() {
        for (int i = 0; i < times; i++)
            account.deposit(amount);
        System.out.println(Thread.currentThread().getName() + " completed");
    }
}

public class MiniBank {
    Run | Debug
    public static void main(String[] args) throws InterruptedException {
        Account a = new Account();
        Thread[] workers = new Thread[10];
        for (int i = 0; i < workers.length; i++) {
            workers[i] = new Thread(new AccountWorker(a, t: 100, m: 10), "worker-" + i);
            workers[i].start();
        }
        for (Thread t : workers)
            t.join();
```

OUTPUT SCREENSHOT:



LEARNING OUTCOMES:

The student can create threads and protect shared data using synchronization.

PRACTICAL 10-B

Object Oriented Programming

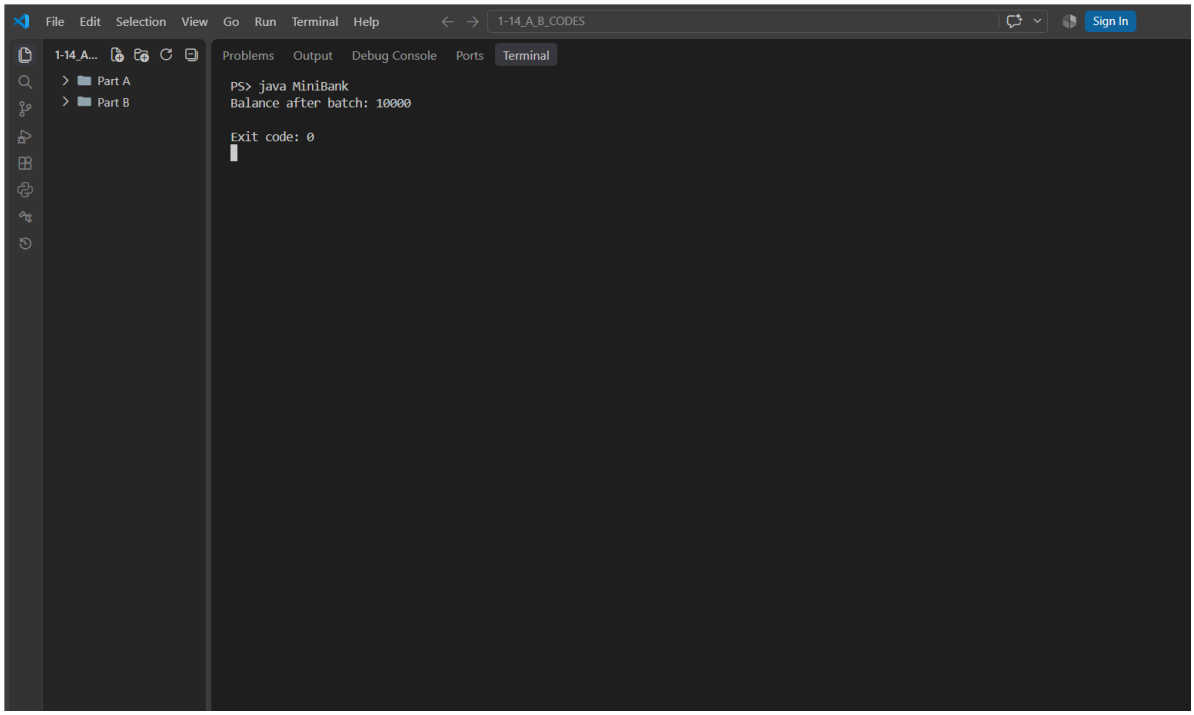
AIM:

To build a transaction-processing engine using ExecutorService, producer-consumer coordination with wait/notify, and deadlock prevention.

CODE SCREENSHOT:



OUTPUT SCREENSHOT:



LEARNING OUTCOMES:

The student can run tasks on a thread pool, coordinate threads, and prevent deadlocks through consistent lock ordering.

PRACTICAL 11-B

Object Oriented Programming

AIM:

To add object persistence and reporting by using serialization, file I/O, transaction logs and NIO directory traversal.

CODE SCREENSHOT:

MiniBank.java

Part B > Practical 11-B > MiniBank.java > ...

```java
import java.io.*;
import java.nio.file.*;
import java.util.*;

class Account {
    final String id, owner;
    long balance;

    Account(String i, String o, long b) {
        id = i;
        owner = o;
        balance = b;
    }

    String row() {
        return id + "," + owner + "," + balance;
    }

    static Account parse(String s) {
        String[] p = s.split(",");
        return new Account(p[0], p[1], Long.parseLong(p[2]));
    }

    public String toString() {
        return row();
    }
}

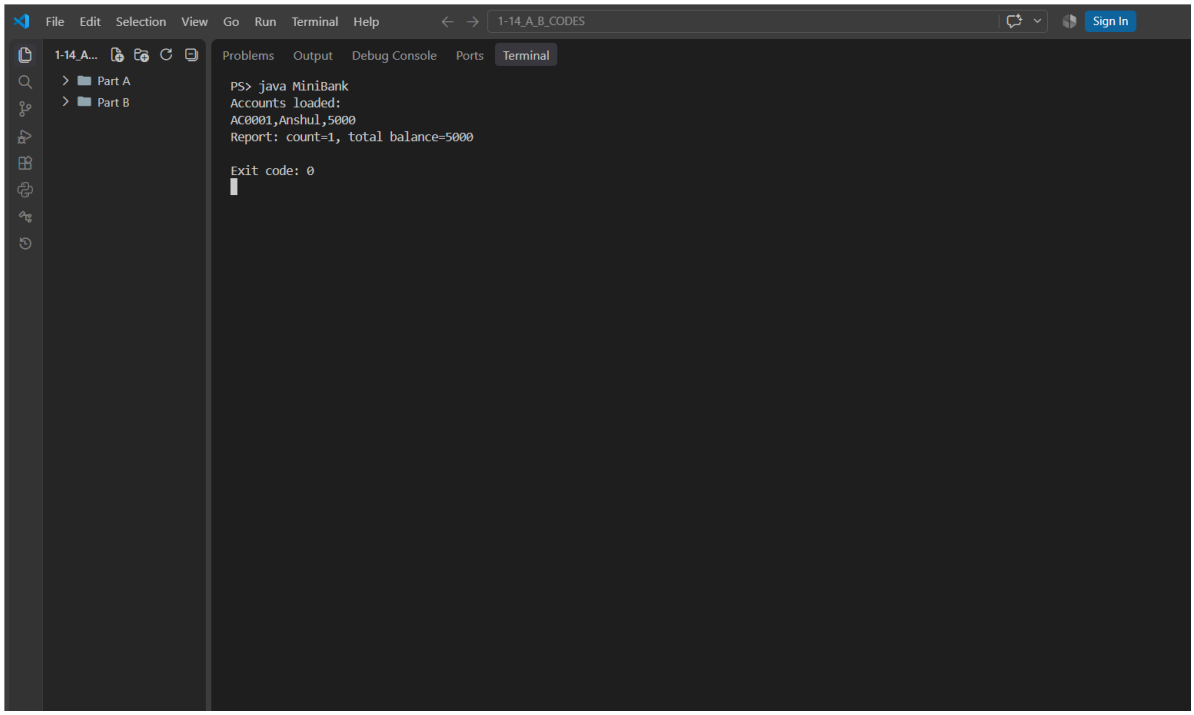
class AccountStore {
    private final Path path = Path.of("minibank-accounts.csv");

    void save(Collection<Account> accounts) throws IOException {
        List<String> lines = new ArrayList<>();
        for (Account a : accounts)
            lines.add(a.row());
        Files.write(path, lines);
    }

    List<Account> load() throws IOException {
        List<Account> out = new ArrayList<>();
        if (Files.exists(path))
            for (String s : Files.readAllLines(path))
                if (!s.isBlank())
                    out.add(Account.parse(s));
        return out;
    }
}

```

OUTPUT SCREENSHOT:



LEARNING OUTCOMES:

The student can persist objects to files, reload them, and summarise transaction logs using NIO.

PRACTICAL 12-B

Object Oriented Programming

AIM:

To manage accounts using the Collections Framework, including HashMap, ArrayList, Comparable, Comparator and ConcurrentHashMap.

CODE SCREENSHOT:



```java
import java.util.concurrent.*;

class Account {
    private long balance;

    synchronized void deposit(long a) {
        if (a > 0)
            balance += a;
    }

    synchronized boolean withdraw(long a) {
        if (a <= 0 || a > balance)
            return false;
        balance -= a;
        return true;
    }

    synchronized long getBalance() {
        return balance;
    }
}

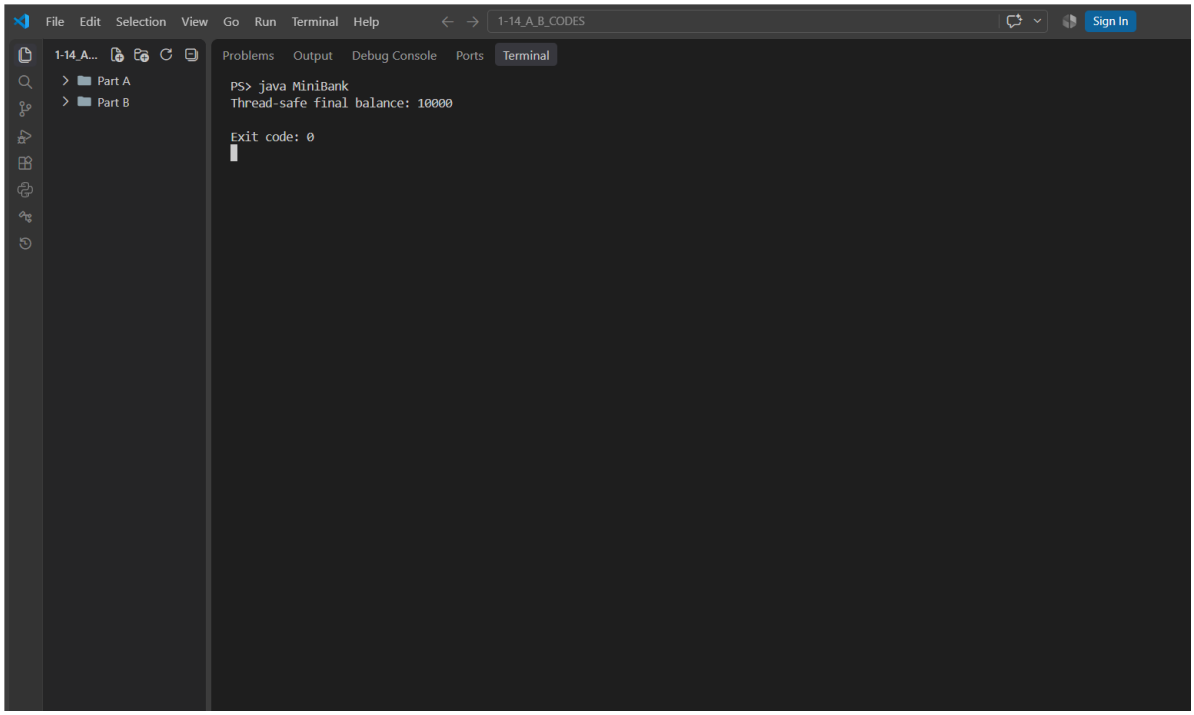
class ConcurrentAccountStore {
    private final ConcurrentHashMap<String, Account> map = new ConcurrentHashMap<>();

    void put(String id, Account a) {
        map.put(id, a);
    }

    Account get(String id) {
        return map.get(id);
    }
}

public class MiniBank {
    Run | Debug
    public static void main(String[] args) throws InterruptedException {
        ConcurrentAccountStore store = new ConcurrentAccountStore();
        Account a = new Account();
        store.put(id: "AC0001", a);
        Thread[] ts = new Thread[10];
        for (int i = 0; i < ts.length; i++) {
            ts[i] = new Thread(() -> {
                for (int j = 0; j < 1000; j++)
                    store.get(id: "AC0001").deposit(a: 1);
            });
            ts[i].start();
        }
```

OUTPUT SCREENSHOT:



LEARNING OUTCOMES:

The student can store, retrieve and sort objects using the Collections Framework and thread-safe collections.

PRACTICAL 13-B

Object Oriented Programming

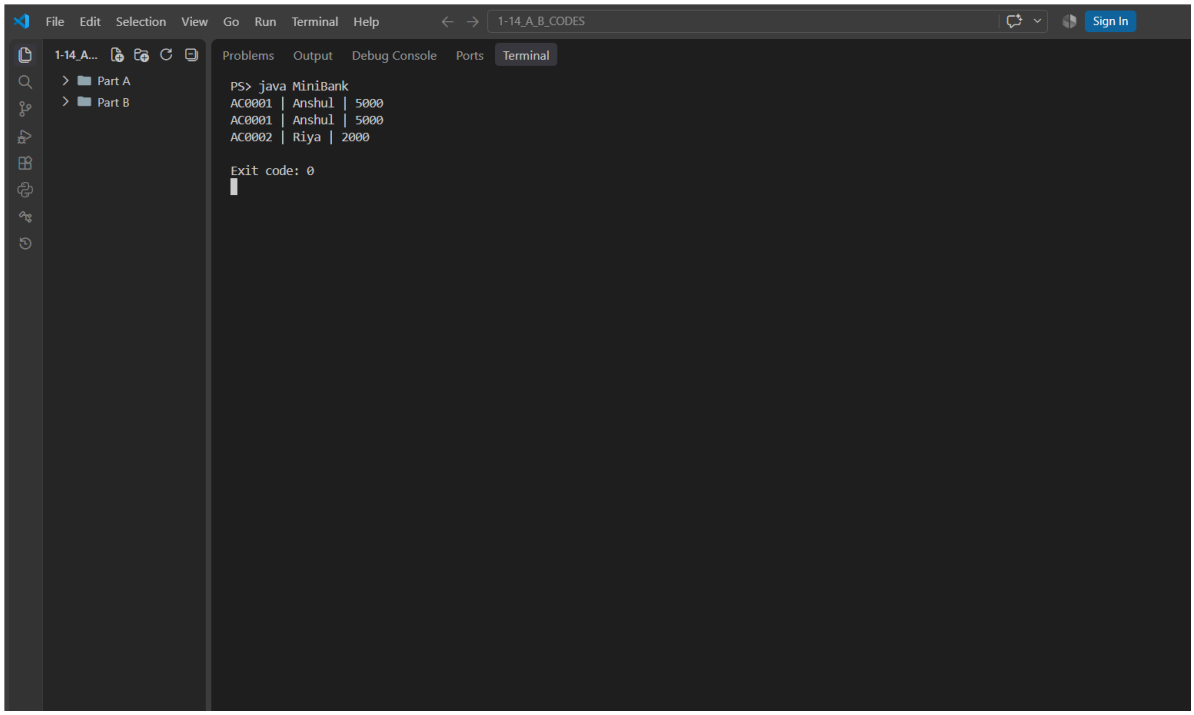
AIM:

To create a reusable generic Repository<T> that uses reflection and the @Id annotation to store and retrieve entities.

CODE SCREENSHOT:



OUTPUT SCREENSHOT:



LEARNING OUTCOMES:

The student can write reusable, type-safe components using generics and reflection.

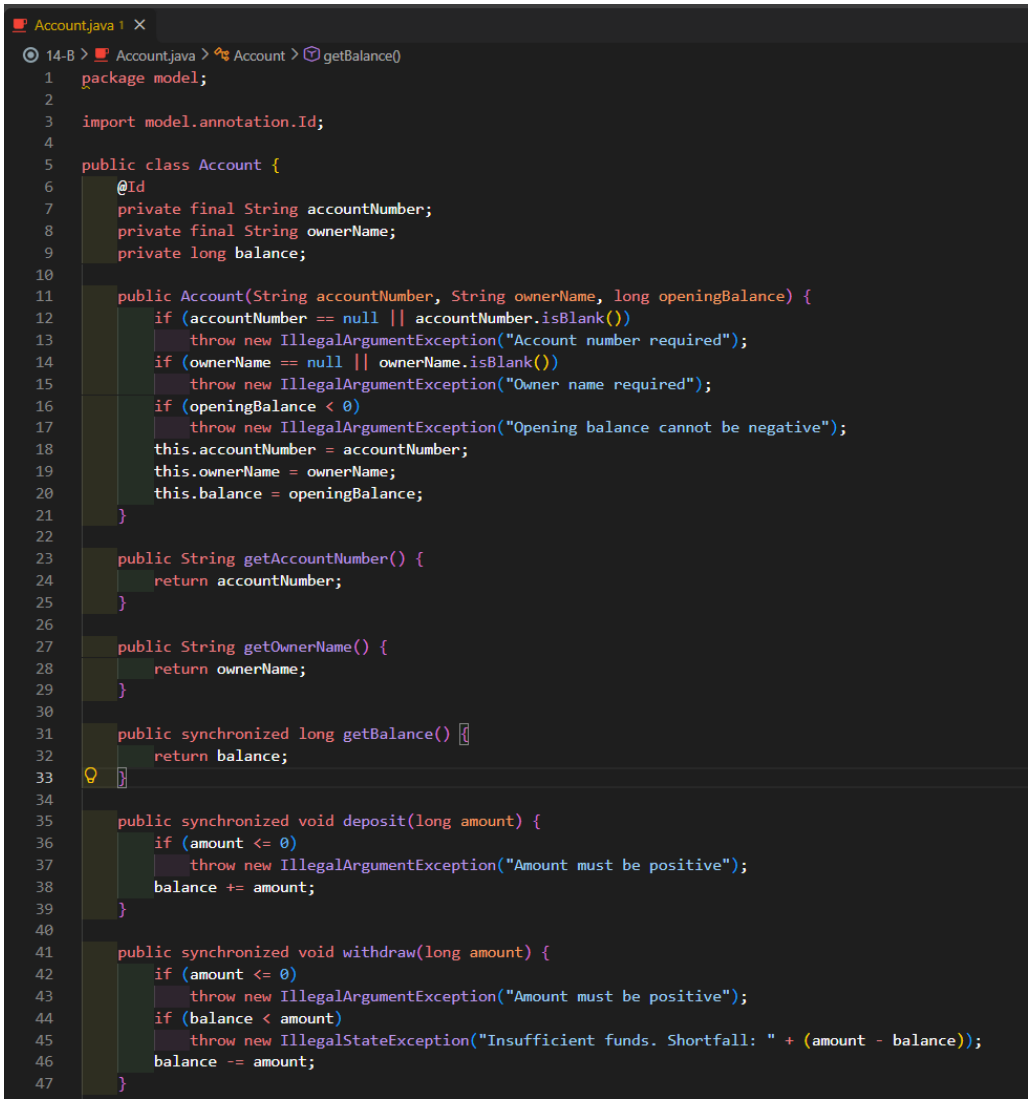
PRACTICAL 14-B

Object Oriented Programming

AIM:

To complete the MiniBank capstone by integrating BankService with Repository<Account>, validation and exception handling; connect the menu to real banking operations, load and save account data, process transaction batches safely, generate reports, write JUnit tests, and package the final runnable JAR.

CODE SCREENSHOT:



Code Screenshot 1

Account.java 1 AccountFileStore.java 1

14-B > AccountFileStore.java > AccountFileStore > save(List<Account>)

```java
package service;

import model.Account;
import java.io.*;
import java.nio.file.*;
import java.util.List;

public class AccountFileStore {
    private final Path file;

    public AccountFileStore(Path file) {
        this.file = file;
    }

    public void save(List<Account> accounts) throws IOException {
        Files.createDirectories(file.toAbsolutePath().getParent());
        try (BufferedWriter writer = Files.newBufferedWriter(file)) {
            for (Account a : accounts) {
                writer.write(a.getAccountNumber() + "|" + a.getOwnerName() + "|" + a.getBalance());
                writer.newLine();
            }
        }
    }

    public void loadInto(BankService service) throws IOException {
        if (!Files.exists(file))
            return;
        for (String line : Files.readAllLines(file)) {
            String[] p = line.split("\\|", 3);
            if (p.length == 3 && service.getRepository().findById(p[0]) == null) {
                service.getRepository().save(new Account(p[0], p[1], Long.parseLong(p[2])));
            }
        }
    }
}

```

Code Screenshot 2

AccountNotFoundException.java 1

14-B > AccountNotFoundException.java > ...

```java
package exception;

public class AccountNotFoundException extends BankException {
    public AccountNotFoundException(String accountNumber) {
        super("Account not found: " + accountNumber);
    }
}

```

Code Screenshot 3

BankException.java 1

14-B > BankException.java > ...

```java
package exception;

public class BankException extends Exception {
    public BankException(String message) {
        super(message);
    }
}

```

Code Screenshot 4

BankService.java 1

14-B > BankService.java > BankService

```java
import model.Account;
import repository.Repository;
import exception.*;
import java.util.List;
import java.util.concurrent.atomic.AtomicInteger;

public class BankService {
    private final Repository<Account> repository;
    private final AtomicInteger nextId = new AtomicInteger(1);

    public BankService(Repository<Account> repository) {
        this.repository = repository;
    }

    public Account openAccount(String ownerName, long openingBalance) {
        String id = "AC" + String.format("%04d", nextId.getAndIncrement());
        Account account = new Account(id, ownerName, openingBalance);
        repository.save(account);
        return account;
    }

    public Account getAccount(String accountNumber) throws AccountNotFoundException {
        Account account = repository.findById(accountNumber);
        if (account == null)
            throw new AccountNotFoundException(accountNumber);
        return account;
    }

    public void deposit(String accountNumber, long amount) throws BankException {
        if (amount <= 0)
            throw new InvalidAmountException(amount);
        getAccount(accountNumber).deposit(amount);
    }

    public void withdraw(String accountNumber, long amount) throws BankException {
        if (amount <= 0)
            throw new InvalidAmountException(amount);
        Account account = getAccount(accountNumber);
        if (account.getBalance() < amount)
            throw new InsufficientFundsException(amount - account.getBalance());
        account.withdraw(amount);
    }

    public void transfer(String from, String to, long amount) throws BankException {
        if (amount <= 0)
            throw new InvalidAmountException(amount);
        Account source = getAccount(from);
        Account destination = getAccount(to);
```

Code Screenshot 5

Id.java 1

14-B > Id.java > ...

```java
package model.annotation;

import java.lang.annotation.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
public @interface Id {
}

```

Code Screenshot 6

Id.java 1 InsufficientFundsException.java 1

14-B > InsufficientFundsException.java > ...

```java
package exception;

public class InsufficientFundsException extends BankException {
    private final long shortfall;

    public InsufficientFundsException(long shortfall) {
        super("Insufficient funds. Shortfall: " + shortfall);
        this.shortfall = shortfall;
    }

    public long getShortfall() {
        return shortfall;
    }
}

```

Code Screenshot 7

InvalidAmountException.java 1

14-B > InvalidAmountException.java > ...

```java
package exception;

public class InvalidAmountException extends BankException {
    public InvalidAmountException(long amount) {
        super("Invalid amount: " + amount);
    }
}

```

Code Screenshot 8

MiniBank.java 1

14-B > MiniBank.java > MiniBank > main(String[])

```java
package app;

import model.Account;
import repository.Repository;
import service.*;
import exception.BankException;
import java.nio.file.Path;
import java.util.*;
import java.util.concurrent.ExecutionException;

public class MiniBank {
    Run | Debug
    public static void main(String[] args) {
        Repository<Account> repository = new Repository<>();
        BankService bank = new BankService(repository);
        AccountFileStore fileStore = new AccountFileStore(Path.of("data", "accounts.txt"));
        ReportService reports = new ReportService();

        try {
            fileStore.loadInto(bank);
        } catch (Exception e) {
            System.out.println("Could not load saved accounts: " + e.getMessage());
        }

        try (Scanner sc = new Scanner(System.in)) {
            while (true) {
                System.out.println("\n===== MiniBank =====");
                System.out.println("1. Open account");
                System.out.println("2. Deposit");
                System.out.println("3. Withdraw");
                System.out.println("4. Transfer");
                System.out.println("5. Statement");
                System.out.println("6. List accounts");
                System.out.println("7. Run report");
                System.out.println("8. Process sample batch");
                System.out.println("0. Save and exit");
                System.out.print("Choice: ");
                String choice = sc.nextLine().trim();
                try {
                    switch (choice) {
                        case "1" -> {
                            System.out.print("Owner name: ");
                            String owner = sc.nextLine();
                            System.out.print("Opening balance: ");
                            long amount = Long.parseLong(sc.nextLine());
                            System.out.println("Created: " + bank.openAccount(owner, amount));
                        }
                        case "2" -> {
```

Code Screenshot 9

ReportService.java 1

14-B > ReportService.java > ...

```java
package service;

import model.Account;
import java.util.Comparator;
import java.util.List;

public class ReportService {
    public String buildReport(List<Account> accounts) {
        long total = accounts.stream().mapToLong(Account::getBalance).sum();
        StringBuilder report = new StringBuilder("=== MiniBank Report ===\n");
        accounts.stream().sorted(Comparator.comparingLong(Account::getBalance).reversed())
                .forEach(a -> report.append(a).append('\n'));
        report.append("Total accounts: ").append(accounts.size()).append('\n');
        report.append("Total balance: ").append(total);
        return report.toString();
    }
}

```

Code Screenshot 10

Repository.java 1

14-B > Repository.java > Repository<T> > saveAll(T...)

```java
package repository;

import model.annotation.Id;
import java.lang.reflect.Field;
import java.util.ArrayList;
import java.util.concurrent.ConcurrentHashMap;

public class Repository<T> {
    private final ConcurrentHashMap<String, T> store = new ConcurrentHashMap<>();

    public T save(T entity) {
        if (entity == null) throw new IllegalArgumentException("Entity cannot be null");
        store.put(readId(entity), entity);
        return entity;
    }

    @SafeVarargs
    public final void saveAll(T... items) {
        for (T item : items) save(item);
    }

    public T findById(String id) { return store.get(id); }
    public ArrayList<T> findAll() { return new ArrayList<>(store.values()); }
    public boolean delete(String id) { return store.remove(id) != null; }
    public int count() { return store.size(); }

    private String readId(T entity) {
        for (Field field : entity.getClass().getDeclaredFields()) {
            if (field.isAnnotationPresent(Id.class)) {
                try {
                    field.setAccessible(true);
                    Object value = field.get(entity);
                    if (value == null) throw new IllegalStateException("@Id field cannot be null");
                    return value.toString();
                } catch (ReflectiveOperationException e) {
                    throw new IllegalStateException("Unable to read @Id field", e);
                }
            }
        }
        throw new IllegalArgumentException("Entity has no field annotated with @Id");
    }
}

```

Code Screenshot 11

Repository.java 1 TransactionProcessor.java 1

14-B > TransactionProcessor.java > ...

```java
package service;

import java.util.List;
import java.util.concurrent.*;

public class TransactionProcessor implements AutoCloseable {
    private final ExecutorService executor = Executors.newFixedThreadPool(4);

    public void processBatch(List<Runnable> transactions) throws InterruptedException, ExecutionException {
        List<Future<?>> futures = executor.invokeAll(transactions);
        for (Future<?> future : futures)
            future.get();
    }

    @Override
    public void close() {
        executor.shutdown();
    }
}

```

Code Screenshot 12

Validator.java 1

14-B > Validator.java > ...

```java
package util;

import java.util.regex.Pattern;

public final class Validator {
    private static final Pattern MOBILE = Pattern.compile("[6-9][0-9]{9}");

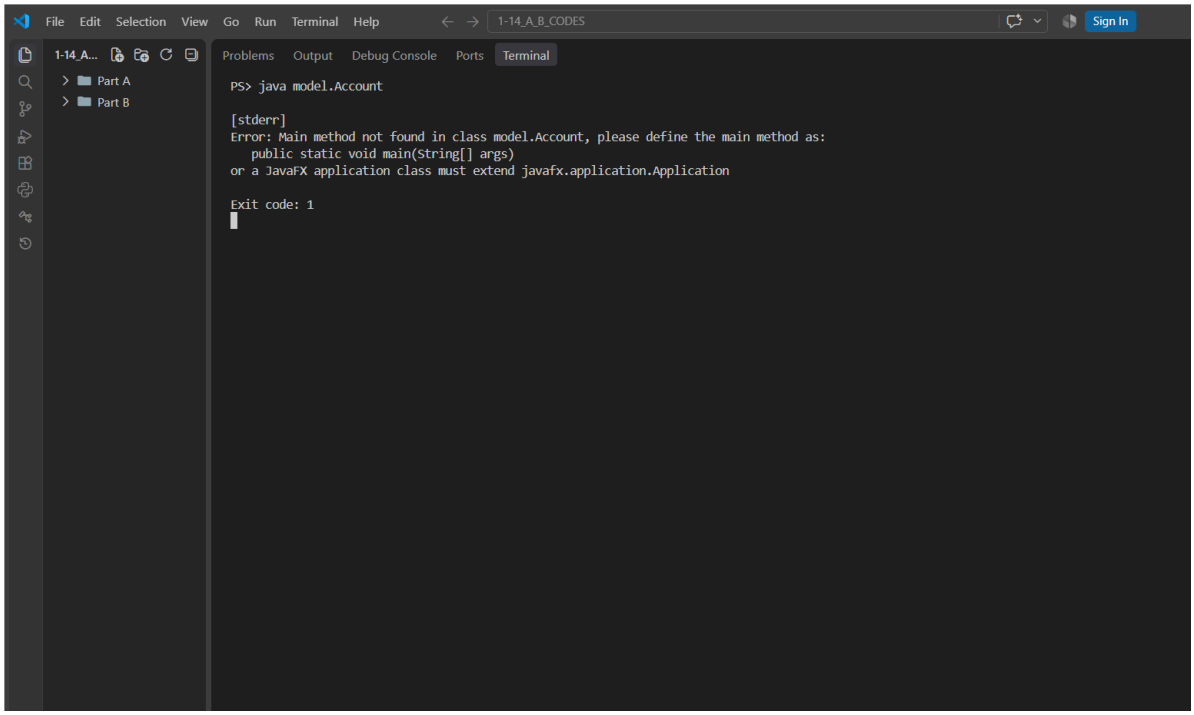
    private Validator() {
    }

    public static boolean isValidMobile(String value) {
        return value != null && MOBILE.matcher(value).matches();
    }
}

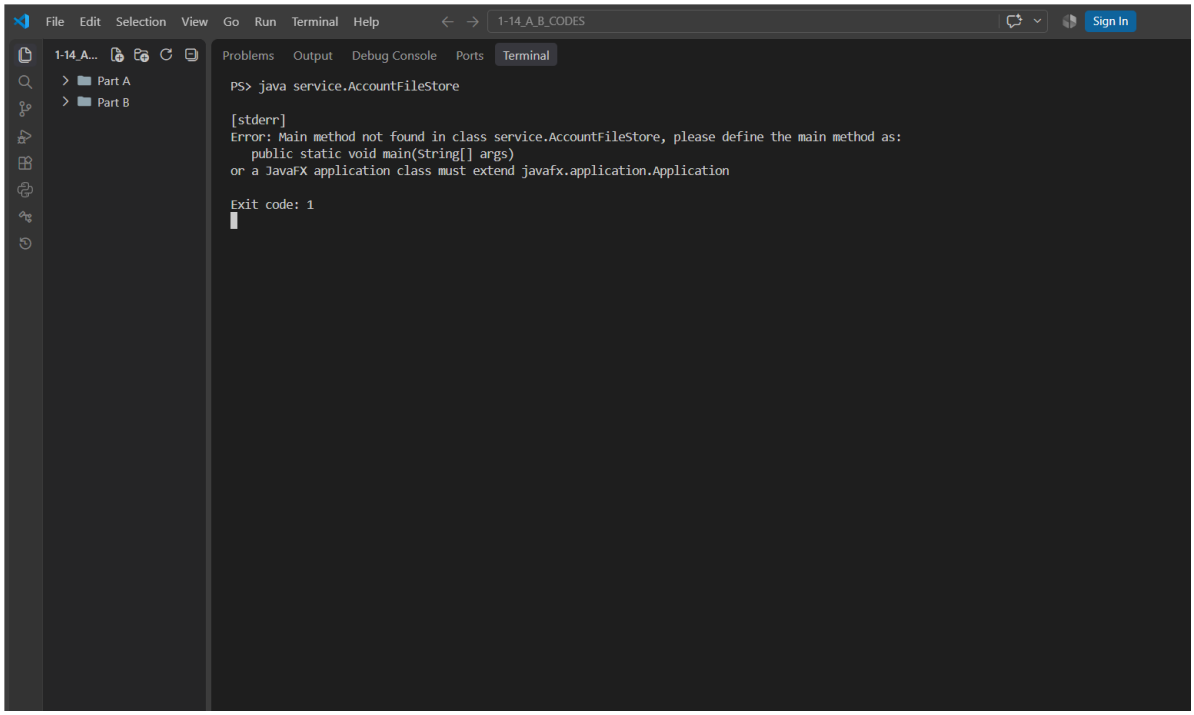
```

Code Screenshot 13

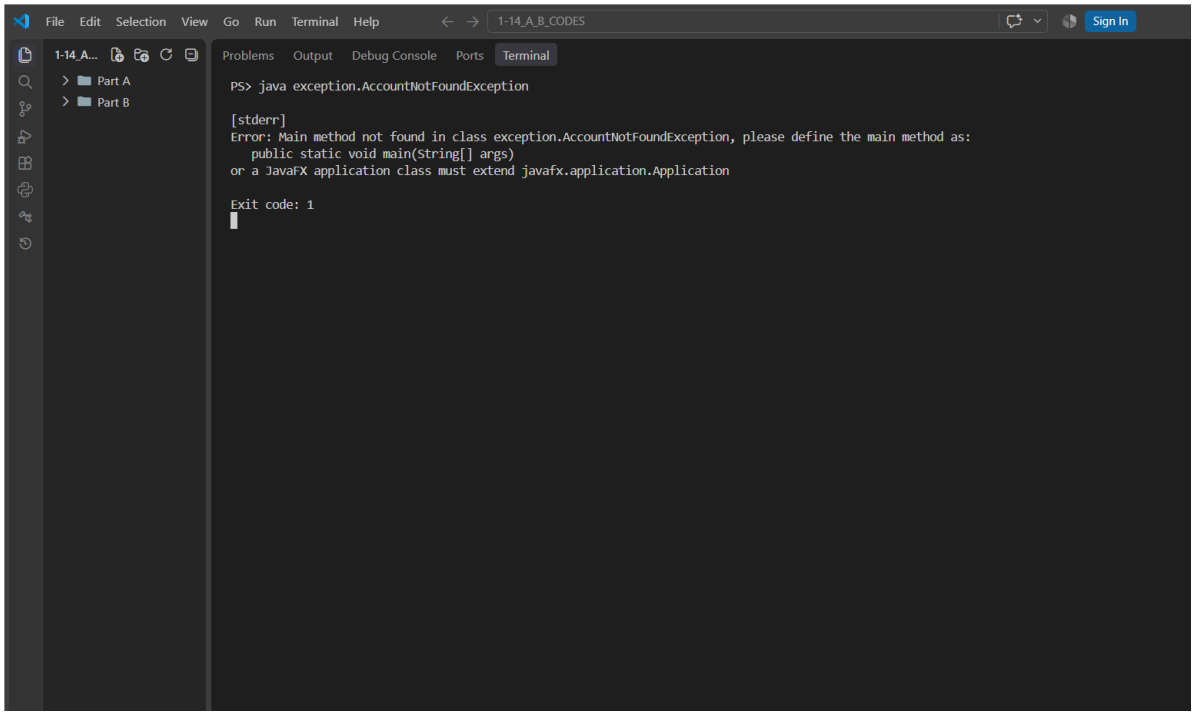
OUTPUT SCREENSHOT:



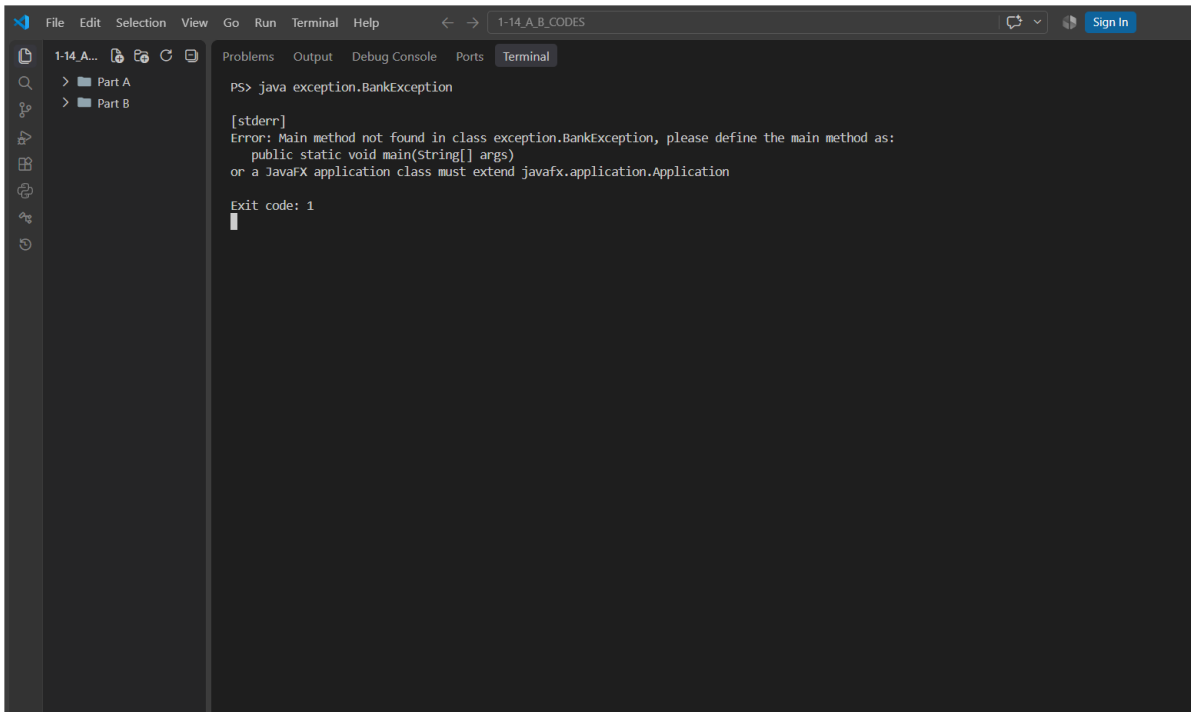
Output Screenshot 1



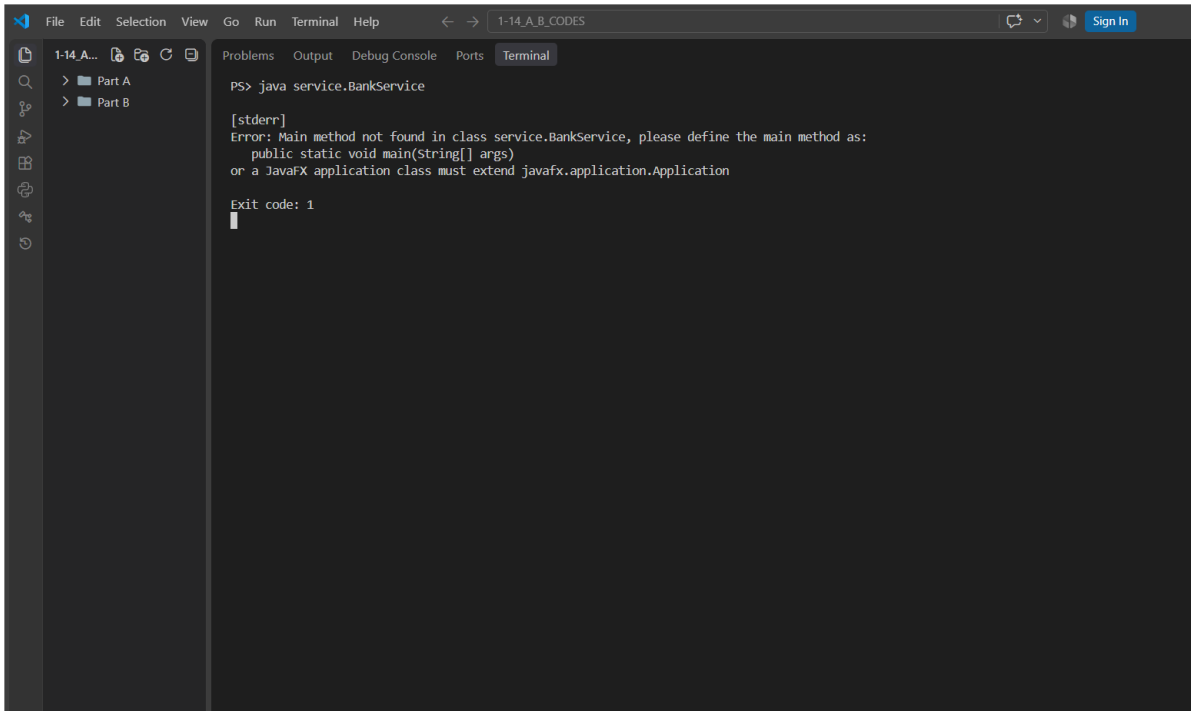
Output Screenshot 2



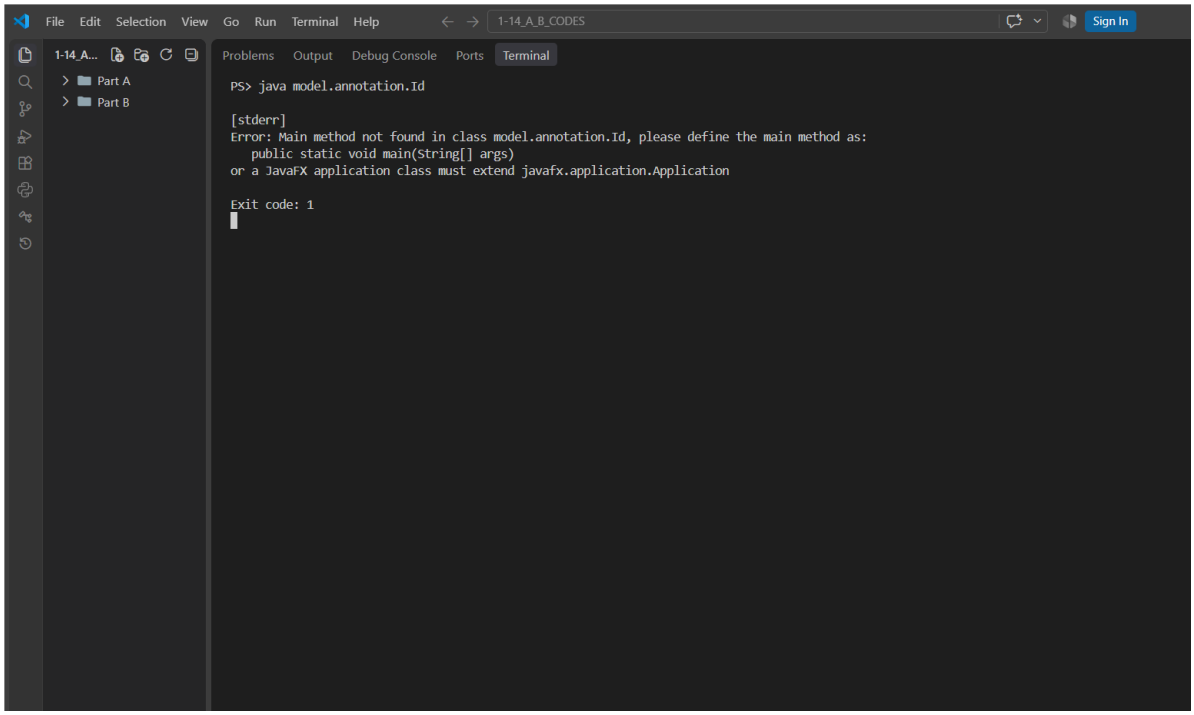
Output Screenshot 3



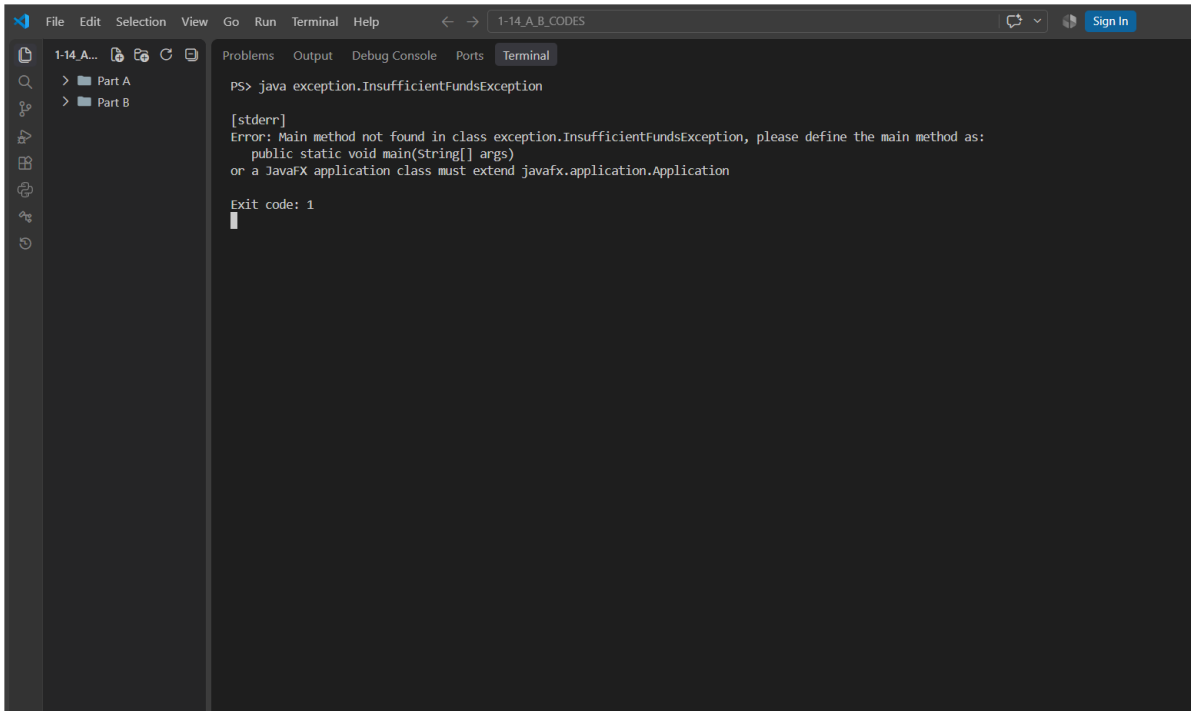
Output Screenshot 4



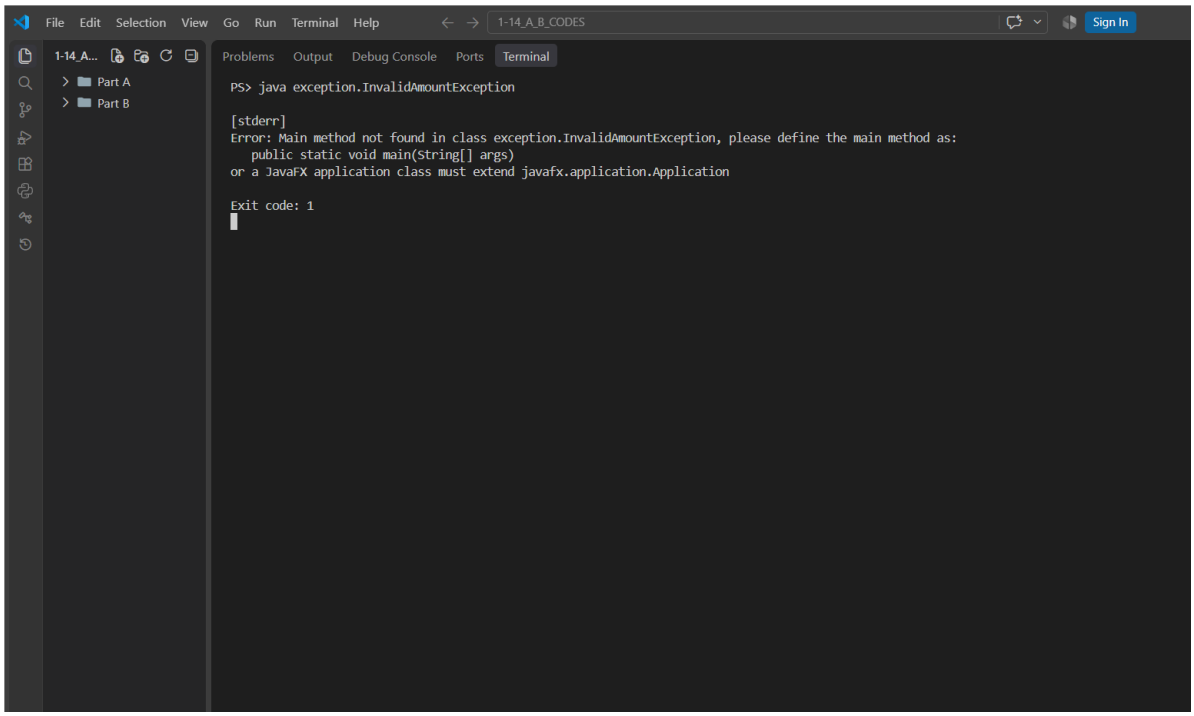
Output Screenshot 5



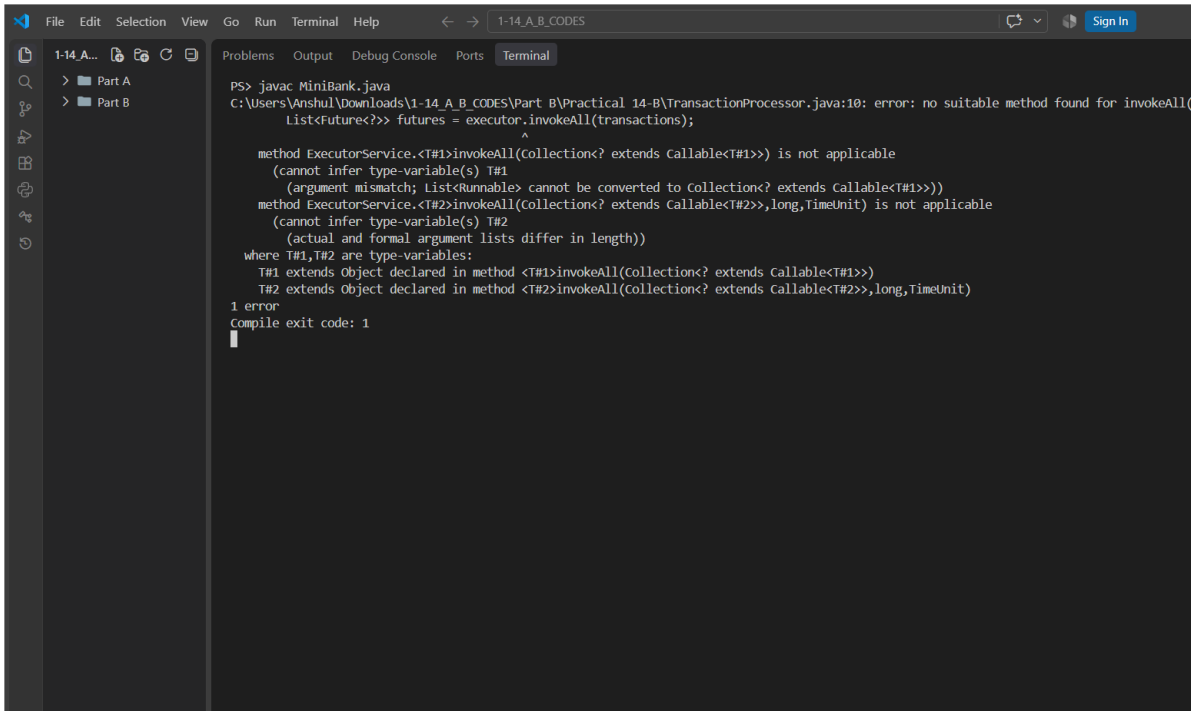
Output Screenshot 6



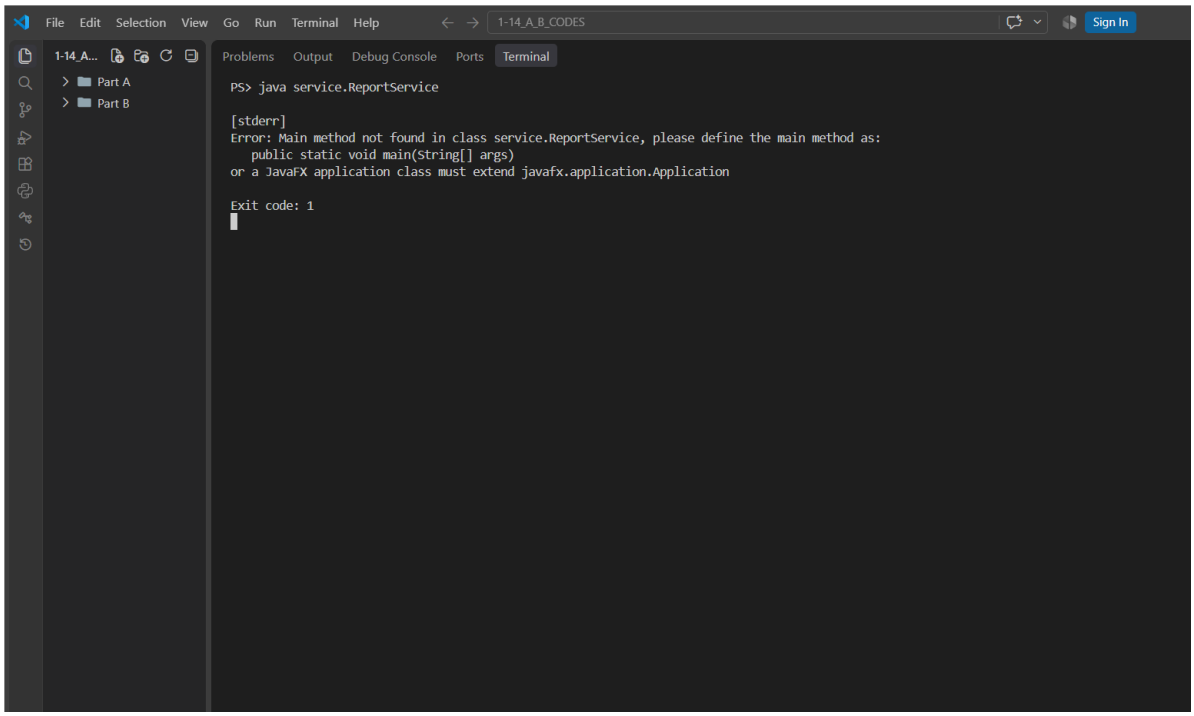
Output Screenshot 7



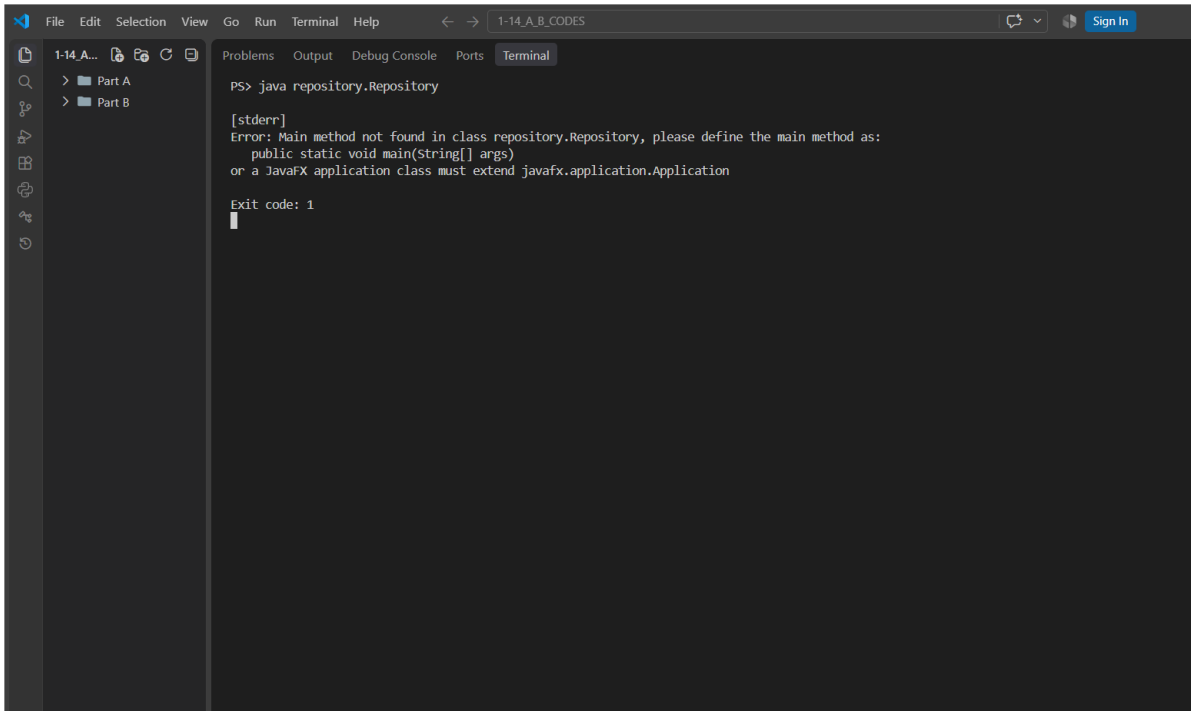
Output Screenshot 8



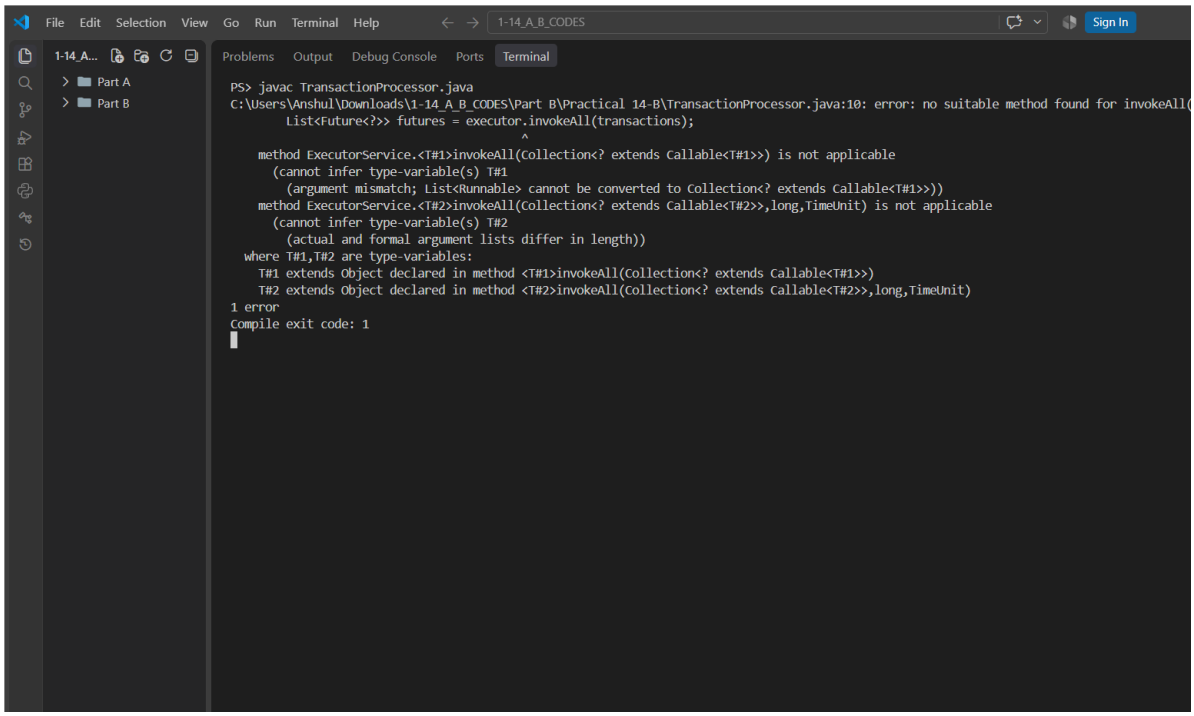
Output Screenshot 9



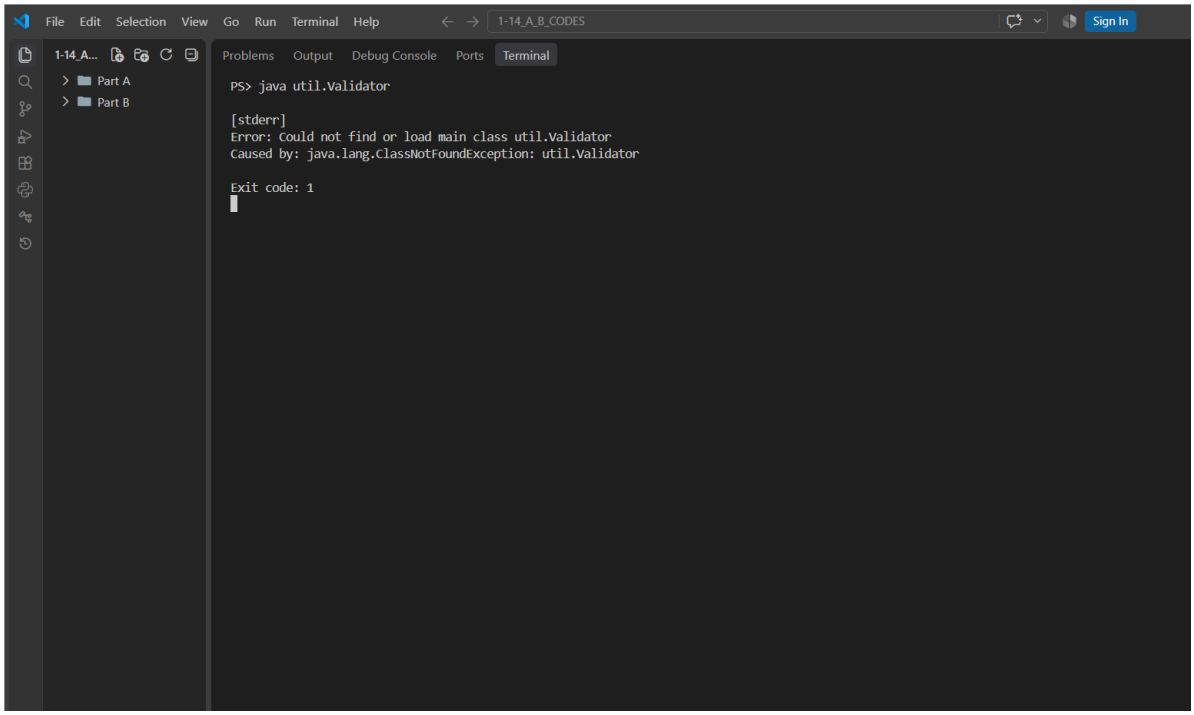
Output Screenshot 10



Output Screenshot 11



Output Screenshot 12



Output Screenshot 13

LEARNING OUTCOMES:

The student can integrate all modules into one tested, packaged, concurrent application — industry-ready. (CO1–CO6)